



ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
FACULTAD DE INGENIERÍA EN ELECTRICIDAD Y
COMPUTACIÓN
SOFG1007 – INGENIERÍA DE SOFTWARE I

ESPECIFICACIÓN DE PROYECTO FINAL
Segundo Parcial

LEODEGA
Storage Marketplace

Integrantes del equipo:

No.	Nombre completo	Matrícula
1	Darwin Leonardo Díaz González	[ID]
2	Geovanny Jahir Díaz Llor	[ID]
3	Carla Alejandra Gutiérrez Córdova	[ID]
4	Johao Carlos Dorado Quinde	[ID]

Profesor: Mgti. Vanessa Jurado Alexandra Vite
Cliente / Representante: Leonardo

Período académico: PAO II – 2026
14 de agosto de 2026

Acta de Conformidad del Cliente

Importante: La ausencia de esta acta firmada implica una penalidad de -100 según la rúbrica de calificación. Adjunte el documento escaneado/firmado.

*Insertar aquí el acta de conformidad firmada por el representante del cliente.
(Reemplazar por: `imagenes/acta_conformidad.pdf` usando `includepdf`,
o insertar una imagen escaneada con `includegraphics`)*

Tabla de Contenido

Acta de Conformidad del Cliente	1
1. Información General del Proyecto	8
1.1. Descripción General	8
1.2. Objetivos del Proyecto	9
1.3. Alcance	9
2. Sección A: Gestión del Proyecto	12
2.1. Gestión de Riesgos	12
2.1.1. Marco de Identificación y Análisis	12
2.1.2. Registro de Riesgos del Proyecto	13
2.2. Sprint Backlogs	18
2.2.1. Sprint 1 — Fundamentos de Autenticación y Gestión de Almacenes	18
2.2.2. Sprint 2 — Catálogo, Reservas y Pago	20
2.2.3. Sprint 3 — Organizaciones (Multiempresa)	22
2.2.4. Sprint 4 — Mensajería, Reportes y Reseñas	24
2.2.5. Sprint 5 — Integración de Inventario y Métricas	26
2.3. Cronograma del Proyecto	28
2.3.1. Tabla de Actividades	28
2.3.2. Diagrama de Red del Proyecto (Activity-on-Arrow)	30
2.3.3. Planificación con Recursos Limitados	30
2.3.4. Diagrama de Gantt	32
3. Sección B: Modelamiento Estático del Sistema	33
3.1. Diagrama de Casos de Uso	33
3.1.1. Actores del Sistema	33
3.1.2. Diagrama General (Contexto)	35
3.1.3. Casos de Uso 01 – Autenticación, Cuenta y Comunicación	36
3.1.4. Casos de Uso 02 – Cliente: Catálogo, Reservas, Pagos y Calificaciones	38
3.1.5. Casos de Uso 03 – Arrendador: Gestión de Bodegas y Reservas	
Recibidas	40
3.1.6. Casos de Uso 04 – Administrador: Moderación, Reportes y Cuentas	42
3.1.7. Casos de Uso 05 – Módulos Nuevos: Organizaciones e Inventario	
Externo	44
3.1.8. Catálogo de Casos de Uso	46
3.1.9. Documentación Detallada de Casos de Uso Modulares	49

3.2.	Diagrama de Clases	61
3.2.1.	Cumplimiento de Principios SOLID	64
3.2.2.	Patrones de Diseño Aplicados	65
3.2.3.	Malos Olores Identificados (Oportunidades de Mejora)	67
3.3.	Diagramas de Objetos	69
3.3.1.	Diagrama de Objetos 01 – Reserva confirmada con pago	69
3.3.2.	Diagrama de Objetos 02 – Reporte de incidente sobre una bodega	71
3.4.	Diagrama de Componentes	72
3.5.	Diagrama de Despliegue	75
4.	Sección C: Modelamiento del Comportamiento del Sistema	77
4.1.	Diagramas de Actividad	77
4.1.1.	Diagrama de Actividad 01 – [Nombre del proceso]	77
4.1.2.	Diagrama de Actividad 02 – [Nombre del proceso]	79
4.1.3.	Diagrama de Actividad 03 – [Nombre del proceso]	80
4.2.	Diagramas de Secuencia	82
4.2.1.	Diagrama de Secuencia 1 – [Nombre del proceso transaccional 1]	83
4.2.2.	Diagrama de Secuencia 2 – [Nombre del proceso transaccional 2]	84
4.2.3.	Diagrama de Secuencia 3 – [Nombre del proceso transaccional 3]	85
4.2.4.	Diagrama de Secuencia 4 – [Nombre del proceso transaccional 4]	86
4.2.5.	Diagrama de Secuencia 5 – [Nombre del proceso transaccional 5]	87
4.2.6.	Diagrama de Secuencia 6 – [Nombre del proceso transaccional 6]	88
4.2.7.	Diagrama de Secuencia 7 – [Nombre del proceso transaccional 7]	89
4.2.8.	Diagrama de Secuencia 8 – [Nombre del proceso transaccional 8]	90
4.2.9.	Diagrama de Secuencia 9 – [Nombre del proceso transaccional 9]	91
4.2.10.	Diagrama de Secuencia 10 – [Nombre del proceso transaccional 10]	92
4.2.11.	Diagrama de Secuencia 11 – [Nombre del proceso transaccional 11]	93
4.2.12.	Diagrama de Secuencia 12 – [Nombre del proceso transaccional 12]	94
4.2.13.	Diagrama de Secuencia 13 – [Nombre del proceso transaccional 13]	95
4.2.14.	Diagrama de Secuencia 14 – [Nombre del proceso transaccional 14]	96
4.2.15.	Diagrama de Secuencia 15 – [Nombre del proceso transaccional 15]	97
4.2.16.	Diagrama de Secuencia 16 – [Nombre del proceso transaccional 16]	98
4.2.17.	Diagrama de Secuencia 17 – [Nombre del proceso transaccional 17]	99
4.2.18.	Diagrama de Secuencia 18 – [Nombre del proceso transaccional 18]	100
4.2.19.	Diagrama de Secuencia 19 – [Nombre del proceso transaccional 19]	101
4.2.20.	Diagrama de Secuencia 20 – [Nombre del proceso transaccional 20]	102
4.2.21.	Diagrama de Secuencia 21 – [Nombre del proceso transaccional 21]	103
4.2.22.	Diagrama de Secuencia 22 – [Nombre del proceso transaccional 22]	104
4.2.23.	Diagrama de Secuencia 23 – [Nombre del proceso transaccional 23]	105

4.2.24. Diagrama de Secuencia 24 – [Nombre del proceso transaccional 24]	106
4.2.25. Diagrama de Secuencia 25 – [Nombre del proceso transaccional 25]	107
4.2.26. Diagrama de Secuencia 26 – [Nombre del proceso transaccional 26]	108
4.2.27. Diagrama de Secuencia 27 – [Nombre del proceso transaccional 27]	109
4.2.28. Diagrama de Secuencia 28 – [Nombre del proceso transaccional 28]	110
4.2.29. Diagrama de Secuencia 29 – [Nombre del proceso transaccional 29]	111
4.2.30. Diagrama de Secuencia 30 – [Nombre del proceso transaccional 30]	112
4.2.31. Diagrama de Secuencia 31 – [Nombre del proceso transaccional 31]	113
4.2.32. Diagrama de Secuencia 32 – [Nombre del proceso transaccional 32]	114
4.3. Diagramas de Colaboración / Comunicación	115
4.3.1. Diagrama de Comunicación 01 – [Escenario ilustrado, ej. correspondiente a Secuencia #1]	115
4.3.2. Diagrama de Comunicación 02 – [Escenario ilustrado]	116
4.3.3. Diagrama de Comunicación 03 – [Escenario ilustrado]	117
4.4. Diagramas de Estado	119
4.4.1. Diagrama de Estado – Clase Reservations	119
4.4.2. Diagrama de Estado – Clase StoreRooms	121
4.4.3. Diagrama de Estado – Clase Reports	123
4.4.4. Diagrama de Estado – Clase Payments	125
4.4.5. Diagrama de Estado – Clase User	127
Contribuciones Individuales	129
A. Especificación de Requerimientos de Software (Versión Mejorada)	132
A.1. Prototipo de Alta Fidelidad	133
A.1.1. Flujo de Ventanas del Prototipo	133
A.1.2. Capturas de Pantalla	134
A.1.3. Pantalla 01 – [Nombre de la pantalla]	134
A.1.4. Pantalla 02 – [Nombre de la pantalla]	135
A.2. Acta de Conformidad del Anexo (ERS y Prototipo)	137

Índice de Figuras

2.1. Diagrama de red del proyecto (Activity-on-Arrow) con ruta crítica	30
2.2. Planificación con recursos limitados sin optimizar (límite = 4 personas, duración total = 87 días)	31
2.3. Planificación con recursos limitados optimizada (límite = 4 personas, duración total = 85 días)	31
2.4. Cronograma del proyecto (diagrama de Gantt)	32
3.1. Diagrama general de casos de uso del sistema (visión de contexto)	35
3.2. Casos de uso de Autenticación, Gestión de Cuenta y Comunicación	36
3.3. Casos de uso del Cliente (arrendatario)	38
3.4. Casos de uso del Arrendador (storage manager)	40
3.5. Casos de uso del Administrador de Plataforma	42
3.6. Casos de uso de Organizaciones e Integración de Inventario (planificados)	44
3.7. Diagrama de clases del sistema (lógica de negocio)	61
3.8. Diagrama de Objetos 01 – Reserva confirmada con pago	69
3.9. Diagrama de Objetos 02 – Reporte de incidente sobre una bodega	71
3.10. Diagrama de Componentes del Sistema	72
3.11. Diagrama de Despliegue del Sistema	75
4.1. Diagrama de Actividad 01 – [Nombre del proceso]	77
4.2. Diagrama de Actividad 02 – [Nombre del proceso]	79
4.3. Diagrama de Actividad 03 – [Nombre del proceso]	80
4.4. Diagrama de Secuencia 1 – [Nombre del proceso transaccional 1]	83
4.5. Diagrama de Secuencia 2 – [Nombre del proceso transaccional 2]	84
4.6. Diagrama de Secuencia 3 – [Nombre del proceso transaccional 3]	85
4.7. Diagrama de Secuencia 4 – [Nombre del proceso transaccional 4]	86
4.8. Diagrama de Secuencia 5 – [Nombre del proceso transaccional 5]	87
4.9. Diagrama de Secuencia 6 – [Nombre del proceso transaccional 6]	88
4.10. Diagrama de Secuencia 7 – [Nombre del proceso transaccional 7]	89
4.11. Diagrama de Secuencia 8 – [Nombre del proceso transaccional 8]	90
4.12. Diagrama de Secuencia 9 – [Nombre del proceso transaccional 9]	91
4.13. Diagrama de Secuencia 10 – [Nombre del proceso transaccional 10]	92
4.14. Diagrama de Secuencia 11 – [Nombre del proceso transaccional 11]	93
4.15. Diagrama de Secuencia 12 – [Nombre del proceso transaccional 12]	94
4.16. Diagrama de Secuencia 13 – [Nombre del proceso transaccional 13]	95
4.17. Diagrama de Secuencia 14 – [Nombre del proceso transaccional 14]	96

4.18. Diagrama de Secuencia 15 – [Nombre del proceso transaccional 15]	97
4.19. Diagrama de Secuencia 16 – [Nombre del proceso transaccional 16]	98
4.20. Diagrama de Secuencia 17 – [Nombre del proceso transaccional 17]	99
4.21. Diagrama de Secuencia 18 – [Nombre del proceso transaccional 18]	100
4.22. Diagrama de Secuencia 19 – [Nombre del proceso transaccional 19]	101
4.23. Diagrama de Secuencia 20 – [Nombre del proceso transaccional 20]	102
4.24. Diagrama de Secuencia 21 – [Nombre del proceso transaccional 21]	103
4.25. Diagrama de Secuencia 22 – [Nombre del proceso transaccional 22]	104
4.26. Diagrama de Secuencia 23 – [Nombre del proceso transaccional 23]	105
4.27. Diagrama de Secuencia 24 – [Nombre del proceso transaccional 24]	106
4.28. Diagrama de Secuencia 25 – [Nombre del proceso transaccional 25]	107
4.29. Diagrama de Secuencia 26 – [Nombre del proceso transaccional 26]	108
4.30. Diagrama de Secuencia 27 – [Nombre del proceso transaccional 27]	109
4.31. Diagrama de Secuencia 28 – [Nombre del proceso transaccional 28]	110
4.32. Diagrama de Secuencia 29 – [Nombre del proceso transaccional 29]	111
4.33. Diagrama de Secuencia 30 – [Nombre del proceso transaccional 30]	112
4.34. Diagrama de Secuencia 31 – [Nombre del proceso transaccional 31]	113
4.35. Diagrama de Secuencia 32 – [Nombre del proceso transaccional 32]	114
4.36. Diagrama de Comunicación 01 – [Escenario ilustrado, ej. correspondiente a Secuencia #1]	115
4.37. Diagrama de Comunicación 02 – [Escenario ilustrado]	116
4.38. Diagrama de Comunicación 03 – [Escenario ilustrado]	117
4.39. Diagrama de Estado – Clase Reservations	119
4.40. Diagrama de Estado – Clase StoreRooms	121
4.41. Diagrama de Estado – Clase Reports	123
4.42. Diagrama de Estado – Clase Payments	125
4.43. Diagrama de Estado – Clase User	127
 A.1. Diagrama de flujo de ventanas del prototipo	 133
A.2. Pantalla 01 – [Nombre de la pantalla]	134
A.3. Pantalla 02 – [Nombre de la pantalla]	135

Índice de Tablas

2.1. Matriz de riesgo 5×5 (Probabilidad × Impacto)	12
2.2. Identificación y evaluación de riesgos	13
2.3. Plan de respuesta a riesgos	15
2.4. Sprint Backlog – Sprint 1	18
2.5. Sprint Backlog – Sprint 2	20
2.6. Sprint Backlog – Sprint 3	22
2.7. Sprint Backlog – Sprint 4	24
2.8. Sprint Backlog – Sprint 5	26
2.9. Actividades, duraciones, dependencias y análisis CPM del proyecto	28
3.1. Actores identificados y su clasificación	33
3.7. Catálogo consolidado de casos de uso	46
3.20. Aplicación de principios SOLID en el diseño de clases	64
3.21. Patrones de diseño utilizados en el sistema [22]	65
3.22. Code smells detectados en la revisión del código real	67
4.6. Contribuciones individuales de los integrantes del equipo	129

Capítulo 1

Información General del Proyecto

1.1. Descripción General

Leodega es un *marketplace* colaborativo de espacios de almacenamiento que conecta a propietarios de espacios —arrendadores, en adelante **Storage Managers**— con personas y organizaciones que necesitan almacenar sus bienes —arrendatarios, en adelante **Cientes**—, bajo la supervisión de un **Administrador** del sistema. El proyecto parte de una plataforma web *full-stack* ya existente, desarrollada bajo metodología Scrum, la cual se encontraba en fase de integración y pruebas al momento de ser entregada al equipo como punto de partida.

La plataforma heredada está construida sobre la siguiente base tecnológica:

- **Backend:** Laravel 12 (PHP 8.2+), autenticación basada en tokens con Laravel Sanctum, ORM Eloquent y base de datos PostgreSQL en producción (SQLite en desarrollo local), contenerizada con Docker Compose y desplegable en Railway.
- **Frontend:** React 19 con TypeScript y Vite, React Router DOM con rutas protegidas por rol, Tailwind CSS, Axios con interceptor de token Bearer, mapas con Leaflet/React-Leaflet, e internacionalización con i18next; desplegable en Vercel.
- **Capacidades existentes:** autenticación y registro de usuarios, control de acceso basado en roles (administrador, arrendador, arrendatario), publicación y moderación de espacios de almacenamiento, búsqueda y vistas de detalle, reservas con detección de conflictos, calificaciones, reportes, notificaciones dentro de la aplicación, conversaciones y mensajería, registro de pagos, y paneles (*dashboards*) por rol.

El presente proyecto tiene como propósito evolucionar Leodega para que soporte cuentas organizacionales (empresas), un acompañante móvil denominado **Leodeguita**, integración con sistemas de inventario externos, importación de inventario desde archivos Excel, validación del permiso del cuerpo de bomberos, un panel de control administrativo, y moderación de cuentas de usuario. Estas capacidades están especificadas mediante historias de usuario, las cuales constituyen la fuente primaria de los requerimientos funcionales del sistema (ver Anexo A). El diseño presentado en las Secciones A, B y C de este documento es consistente con dicha especificación y con el prototipo de alta fidelidad aprobado por el cliente.

1.2. Objetivos del Proyecto

Objetivo General

Evolucionar la plataforma Leodega hacia un *marketplace* de almacenamiento completo, confiable y accesible desde dispositivos móviles, que conecte a Storage Managers y Clientes —de forma individual o bajo organizaciones— a través de un sistema moderado y confiable que integre capacidades de reserva, pago, comunicación, reporte de incidentes e integración de inventario.

Objetivos Específicos

1. **Gestión de almacenamiento.** Permitir a los Storage Managers registrar, editar y eliminar bodegas —incluyendo el permiso obligatorio del cuerpo de bomberos—, así como visualizar, cancelar y comunicarse sobre sus reservas. (Referencias: HUG-01 a HUG-08.)
2. **Experiencia del cliente.** Permitir a los Clientes buscar y filtrar bodegas, ver su detalle, reservarlas y pagarlas, gestionar sus reservas, comunicarse con los gestores, reportar incidentes, calificar arriendos finalizados, e integrar sus sistemas de inventario externos o importar inventario desde archivos Excel. (Referencias: HUC-01 a HUC-14.)
3. **Administración de la plataforma.** Permitir a los Administradores validar permisos del cuerpo de bomberos y moderar publicaciones, bloquear y reactivar cuentas de usuario, monitorear un panel de control con métricas clave, y revisar y resolver reportes. (Referencias: HUA-01 a HUA-04.)
4. **Cuentas organizacionales.** Habilitar la creación de organizaciones, la invitación y aceptación de clientes miembros, el cambio entre contexto personal y organizacional, las reservas a nivel empresarial, y la administración y auditoría de la actividad organizacional. (Referencias: HUE-01 a HUE-06.)
5. **Acompañante móvil (Leodeguita).** Proveer acceso móvil para autenticarse con las credenciales de Leodega, seleccionar la empresa operadora, publicar bodegas, y explorar bodegas disponibles. (Referencias: HUL-01 a HUL-06.)

1.3. Alcance

El alcance del sistema está definido por los requerimientos funcionales derivados de las historias de usuario, resumidos en la Sección B de este documento y detallados en el Anexo A.

Alcance funcional por rol de usuario

- **Storage Manager (arrendador).** Recibir y responder consultas previas a la reserva; recibir notificaciones de confirmación de pago y ver contratos activos; chatear con clientes durante arriendos activos; registrar una bodega con dimensiones, ubicación, fotografías, tarifa mensual y permiso del cuerpo de bomberos; ver reservas con filtros y detalle; cancelar reservas futuras pagadas con motivo obligatorio; eliminar bodegas sin reservas activas o futuras; y editar la información de una bodega. (HUG-01 a HUG-08.)
- **Cliente (arrendatario).** Buscar y filtrar bodegas por ubicación, tamaño y precio, con vista de mapa; ver el detalle de una bodega; reservar bodegas disponibles con validación de conflicto de fechas; completar un pago simulado para confirmar una reserva; comunicarse con los gestores antes y durante el arriendo; ver y gestionar reservas activas e históricas; cancelar reservas futuras confirmadas; reportar incidentes; calificar arriendos finalizados; aceptar o rechazar invitaciones a organizaciones; registrar un endpoint de inventario externo y su mapeo de campos; ver el inventario sincronizado; e importar inventario desde un archivo Excel. (HUC-01 a HUC-14.)
- **Administrador.** Validar el permiso del cuerpo de bomberos y aprobar o rechazar publicaciones de bodegas; bloquear y reactivar cuentas de usuario con registro de auditoría; monitorear un panel de control con métricas de bodegas, usuarios, transacciones, reservas y reportes sin resolver; y revisar, responder y resolver reportes. (HUA-01 a HUA-04.)
- **Administrador de Organización y miembros.** Crear una organización; invitar clientes como miembros; acceder a un panel administrativo empresarial con miembros, reservas e historial; y eliminar reservas empresariales con motivo obligatorio. Los miembros clientes pueden cambiar entre contexto personal y organizacional, reservar en nombre de la organización, y compartir visibilidad de las reservas empresariales, sin poder eliminarlas. (HUE-01 a HUE-06.)
- **Usuarios de Leodeguita (todos los roles).** El acompañante móvil permite a cualquier usuario de Leodega autenticarse con sus credenciales existentes; permite a un cliente seleccionar la empresa operadora; permite a un Storage Manager publicar una bodega para revisión y ver sus bodegas publicadas; y permite a un cliente explorar y ver el detalle de bodegas disponibles. (HUL-01 a HUL-06.)

Fuera de alcance

- **Procesamiento de pagos reales.** El flujo de pago es simulado; no se requiere integración con pasarelas de pago o procesadores bancarios reales.

- **Integración con ERP y facturación electrónica.** Forman parte de la visión del producto pero no están implementadas ni cubiertas por ninguna historia de usuario; quedan excluidas de esta especificación.
- **Aplicación móvil React Native de propósito general.** Se planifica para versiones futuras; solo los flujos del acompañante Leodeguita (HUL-01 a HUL-06) están dentro del alcance.
- **Funcionalidades heredadas no re-especificadas** (por ejemplo, favoritos, gestión de tokens de sesión y recuperación de contraseña) se mantienen de la plataforma heredada tal como están y no se re-especifican como requerimientos funcionales en este documento.

Capítulo 2

Sección A: Gestión del Proyecto

2.1. Gestión de Riesgos

2.1.1. Marco de Identificación y Análisis

Los riesgos del proyecto se identificaron mediante tres técnicas complementarias: listas de chequeo de riesgos típicos de proyectos de software, categorías documentadas de riesgo (técnico, de alcance, de recursos/equipo, de cronograma, de seguridad y externo/regulatorio) y sesiones de tormenta de ideas del equipo, contrastadas contra los artefactos de planificación del proyecto (historias de usuario, cronograma PERT/CPM y estimación funcional COSMIC, ver Sección 2.3).

Cada riesgo identificado se evalúa según dos dimensiones cualitativas de 5 niveles:

- **Probabilidad de ocurrencia:** (1) No probable [1–20 %], (2) Posible [20–40 %], (3) Poco probable [40–60 %], (4) Probable [60–80 %], (5) Muy probable [80–100 %].
- **Impacto potencial:** (1) Mínimo, (2) Obstáculo menor — podría no afectar la satisfacción del cliente, presupuesto, cronograma o desempeño—, (3) Obstáculo — potencial insatisfacción del cliente, sobrecosto, retraso o degradación del sistema—, (4) Obstáculo mayor — efecto significativo sobre cliente, cronograma, presupuesto o desempeño—, (5) Sería amenaza — severa insatisfacción del cliente, sobrecosto grave, retraso grave o degradación severa del sistema.

El nivel de riesgo resultante (Probabilidad $\times$ Impacto) se clasifica de acuerdo con la matriz de calor 5 $\times$ 5 de la Tabla 2.1, la cual determina el orden de prioridad de atención: primero los riesgos de alta probabilidad y alto impacto, luego los de alto impacto con baja probabilidad, después los de bajo impacto con alta probabilidad, y por último los de bajo impacto y baja probabilidad.

Tabla 2.1: Matriz de riesgo 5 $\times$ 5 (Probabilidad $\times$ Impacto)

Prob. ↓ / Impacto →	Insign. (1)	Menor (2)	Signif. (3)	Mayor (4)	Severo (5)
5 – Casi seguro	Medio 5	Alto 10	Muy alto 15	Extremo 20	Extremo 25
4 – Probable	Medio 4	Medio 8	Alto 12	Muy alto 16	Extremo 20
3 – Moderado	Bajo 3	Medio 6	Medio 9	Alto 12	Muy alto 15
2 – Poco probable	Muy bajo 2	Bajo 4	Medio 6	Medio 8	Alto 10
1 – Raro	Muy bajo 1	Muy bajo 2	Bajo 3	Medio 4	Medio 5

Para el manejo de cada riesgo se aplica una (o una combinación) de las siguientes seis estrategias:

1. **Evitar el riesgo:** remover por completo la causa del riesgo.
2. **Reducción de la probabilidad:** disminuir la probabilidad de que el riesgo ocurra.
3. **Reducción de impacto:** disminuir la severidad de las consecuencias si el riesgo ocurre.
4. **Transferencia del riesgo:** trasladar el impacto (típicamente financiero) a un tercero, p. ej. mediante librerías/servicios ya auditados o subcontratación.
5. **Planes de contingencia:** acciones predefinidas que se ejecutan si la amenaza se materializa.
6. **Aceptación del riesgo:** riesgos de bajo impacto y baja probabilidad que se monitorean sin actuar activamente sobre ellos, salvo que su probabilidad o impacto cambie.

2.1.2. Registro de Riesgos del Proyecto

La Tabla 2.2 presenta los riesgos identificados para Leodega, ordenados de mayor a menor nivel de riesgo. Los riesgos R-01 y R-02 están directamente vinculados a la concentración de tamaño funcional revelada por la estimación COSMIC del proyecto (el módulo Cliente concentra 120 de 324 puntos de función, el 37 % del total) y a la ausencia de holgura en la ruta crítica del cronograma (Sección 2.3).

Tabla 2.2: Identificación y evaluación de riesgos

ID	Riesgo	Categoría	Prob.	Impacto	Nivel
R-01	Retraso o falla en la integración con los sistemas de inventario externos de los clientes (HUC-10 a HUC-13): Leodega depende de APIs de terceros no controladas por el equipo y de un modelo de sincronización <i>pull</i> cuyo comportamiento real se desconoce hasta la integración.	Técnico / Integración	4 – Probable	4 – Obstáculo mayor	Muy alto (16)

continúa en la siguiente página

Tabla 2.2 (continuación)

ID	Riesgo	Categoría	Prob.	Impacto	Nivel
R-02	Subestimación del esfuerzo del módulo Cliente, que concentra el 37 % del tamaño funcional total del sistema (120 de 324 CFP según la estimación COSMIC), lo que puede provocar desbordes de cronograma en los Sprints 2, 4 y 5.	Técnico / Complejidad	4 – Probable	4 – Obstáculo mayor	Muy alto (16)
R-03	Retraso en cascada sobre la ruta crítica del proyecto si cualquiera de sus actividades se atrasa, dado que ninguna tiene holgura (float = 0).	Cronograma	3 – Poco probable	5 – Seria amenaza	Muy alto (15)
R-04	Indisponibilidad o rotación de un integrante del equipo, dado que la planificación con recursos limitados ya asume el uso máximo de personal disponible sin holgura adicional en las tareas críticas.	Recursos / Equipo	3 – Poco probable	4 – Obstáculo mayor	Alto (12)
R-05	Curva de aprendizaje del equipo con la tecnología del acompañante móvil Leodeguita, no utilizada previamente en la plataforma heredada (100 % web).	Técnico / Aprendizaje	4 – Probable	3 – Obstáculo	Alto (12)
R-06	Vulnerabilidades en la autenticación compartida entre la plataforma web y Leodeguita (mismas credenciales, tokens Bearer) si el almacenamiento de tokens en el cliente móvil no es seguro.	Seguridad	2 – Posible	5 – Seria amenaza	Alto (10)

continúa en la siguiente página

Tabla 2.2 (continuación)

ID	Riesgo	Categoría	Prob.	Impacto	Nivel
R-07	Demoras o inconsistencias en la validación del permiso del cuerpo de bomberos por parte del Administrador, al depender de un proceso de revisión documental no automatizado.	Externo / Regulatorio	3 – Poco probable	3 – Obstáculo	Medio (9)
R-08	Cambios o ambigüedades en los requerimientos de las historias de usuario durante el desarrollo que obliguen a retrabajo.	Alcance	2 – Posible	3 – Obstáculo	Medio (6)
R-09	Expectativa del cliente/evaluador de un procesamiento de pago real, dado que el flujo de pago está simulado por diseño (fuera de alcance).	Alcance	2 – Posible	3 – Obstáculo	Medio (6)

Tabla 2.3: Plan de respuesta a riesgos

ID	Estrategia	Acciones de mitigación	Plan de contingencia	Responsable
R-01	Reducción de impacto + Plan de contingencia	Diseñar el conector de inventario con una capa de adaptación (patrón <i>Adapter</i>) desacoplada del núcleo y definir contratos de API simulados (<i>mocks</i>) desde etapas tempranas para pruebas de integración anticipadas.	Si la integración real no está disponible a tiempo, degradar al modo de importación manual vía Excel (HUC-13) para cumplir la fecha de entrega, e integrar el conector en un sprint posterior.	Equipo Sprint 5

continúa en la siguiente página

Tabla 2.3 (continuación)

ID	Estrategia	Acciones de mitigación	Plan de contingencia	Responsable
R-02	Reducción de impacto	Usar la estimación funcional COSMIC para re-balancear el personal asignado por sprint y descomponer las historias más grandes (HUC-10 a HUC-13) en subtareas verificables.	Priorizar las funcionalidades de mayor valor de negocio del módulo Cliente (búsqueda, reserva, pago) sobre las de menor prioridad (integración de inventario) si el tiempo se agota.	Líder técnico
R-03	Planes de contingencia + Reducción de probabilidad	Monitorear semanalmente el avance de las actividades de la ruta crítica y reforzar con personal adicional ante cualquier desviación temprana.	Usar la holgura ganada en la optimización de recursos y, de ser necesario, negociar con el cliente una reducción de alcance en funcionalidades no críticas para proteger la fecha de entrega.	Líder de proyecto
R-04	Planes de contingencia + Reducción de probabilidad	Distribuir el conocimiento de los módulos críticos entre al menos dos integrantes (reducir el <i>bus factor</i>) y mantener documentación técnica continuamente actualizada.	Reasignar las tareas del integrante ausente al miembro con menor carga según el cronograma nivelado, aceptando una extensión acotada antes de escalar al profesor/cliente.	Todo el equipo
R-05	Reducción de probabilidad + Reducción de impacto	Capacitación express del equipo antes del sprint del acompañante móvil, reutilizando la lógica de negocio y los endpoints REST ya validados en el backend.	Reducir el alcance de Leodeguita a las historias de mayor prioridad (login y exploración) si el aprendizaje consume el margen disponible.	Equipo Sprint 5 (móvil)

continúa en la siguiente página

Tabla 2.3 (continuación)

ID	Estrategia	Acciones de mitigación	Plan de contingencia	Responsable
R-06	Reducción de impacto + Transfere- rencia	Usar almacenamiento seguro nativo para los tokens, expiración corta de sesión y cumplimiento de los requerimientos de seguridad del ERS (RNF-03/RNF-04).	Revocar de forma centralizada los tokens comprometidos desde el backend y forzar reautenticación de los usuarios afectados.	Responsable de seguridad/backend
R-07	Aceptación + Reducción de impacto	Definir casos de prueba con documentos de permiso simulados (válidos, vencidos, ilegibles) para validar el flujo sin depender de una autoridad real.	Documentar un criterio de validación manual de respaldo si la automatización no cubre todos los casos.	QA
R-08	Reducción de probabilidad	Validar cada historia de usuario con el representante del cliente antes de iniciar el sprint correspondiente y mantener trazabilidad HU → RF (Sección B).	Congelar el alcance del sprint en curso y trasladar el cambio solicitado al backlog del siguiente sprint.	Product Owner
R-09	Reducción de probabilidad	Documentar explícitamente el alcance y las exclusiones del proyecto (Capítulo 1) y validarlo con el cliente en el acta de conformidad.	Aclarar en la demostración del sistema que el pago es simulado por diseño.	Product Owner

2.2. Sprint Backlogs

El proyecto se organizó en **5 sprints**, con una duración total planificada de 52 días según el cronograma PERT/CPM (Sección 2.3), gestionados mediante un tablero ágil tipo Scrum en *[herramienta de gestión ágil utilizada, p. ej. Jira, Trello, Azure DevOps]*. Cada historia de usuario se prioriza con la escala MoSCoW (Must/Should/Could Have) y se estima en puntos de historia.

2.2.1. Sprint 1 — Fundamentos de Autenticación y Gestión de Almacenes

Objetivo del Sprint: Establecer el inicio de sesión compartido web/móvil y las operaciones básicas de registro, edición, eliminación y moderación de almacenes.

Duración: Día 1 – Día 11 (11 días)

Tabla 2.4: Sprint Backlog – Sprint 1

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUG-04	Como Gestor de Almacenamiento, quiero registrar un nuevo almacén (dimensiones, ubicación, fotos, tarifa y permiso de bomberos vigente), para ofrecer mi espacio en alquiler.	5	Equipo Leodega	Pendiente
HUG-07	Como Gestor, quiero eliminar un almacén sin reservas activas o futuras, para retirar espacios que ya no ofrezco.	3	Equipo Leodega	Pendiente
HUG-08	Como Gestor, quiero editar la información de un almacén ya publicado, para mantener actualizada mi oferta sin recrearlo.	5	Equipo Leodega	Pendiente
HUA-01	Como Administrador, quiero revisar y validar el permiso de bomberos subido por el Gestor, para asegurar el cumplimiento legal antes de habilitar el espacio.	5	Equipo Leodega	Pendiente

continúa en la siguiente página

Tabla 2.4 (continuación)

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUA-03	Como Administrador, quiero bloquear o reactivar cuentas de usuario, para actuar como moderador ante incumplimientos de las políticas de uso.	5	Equipo Leodega	Pendiente
HUL-01	Como usuario (Administrador, Gestor o Cliente), quiero iniciar sesión en Leodeguita con mis credenciales de Leodega, para acceder a mis datos sin crear una cuenta aparte.	3	Equipo Leodega	Pendiente
HUL-03	Como Gestor, quiero registrar un almacén disponible con el mínimo de pasos, para publicarlo rápidamente desde mi dispositivo móvil.	5	Equipo Leodega	Pendiente
HUL-04	Como Gestor, quiero ver la lista de almacenes que he publicado, para monitorear mis espacios y su estado actual.	2	Equipo Leodega	Pendiente

2.2.2. Sprint 2 — Catálogo, Reservas y Pago

Objetivo del Sprint: Habilitar la búsqueda y reserva de almacenes por parte del cliente, el pago simulado y la gestión de reservas por parte del Gestor.

Duración: Día 12 – Día 21 (10 días)

Tabla 2.5: Sprint Backlog – Sprint 2

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUG-02	Como Gestor, quiero recibir una notificación cuando un cliente confirme el pago de una reserva, para saber en tiempo real qué contratos se formalizaron.	3	Equipo Leodega	Pendiente
HUG-05	Como Gestor, quiero ver todas las reservas de mis almacenes con fecha, cliente y estado de pago, para gestionar el uso de mi espacio.	3	Equipo Leodega	Pendiente
HUG-06	Como Gestor, quiero cancelar una reserva pagada indicando un motivo obligatorio, para liberar mi espacio ante imprevistos manteniendo transparencia con el cliente.	3	Equipo Leodega	Pendiente
HUC-01	Como Cliente, quiero buscar y filtrar almacenes disponibles por ubicación, tamaño y precio, para encontrar el espacio que mejor se ajuste a mis necesidades y presupuesto.	5	Equipo Leodega	Pendiente
HUC-02	Como Cliente, quiero ver el detalle completo de un almacén antes de reservar, para evaluar si cumple mis requisitos de condiciones, capacidad, precio y ubicación.	3	Equipo Leodega	Pendiente
HUC-03	Como Cliente, quiero reservar un almacén disponible seleccionando el período de alquiler deseado, para asegurar el espacio que necesito en las fechas requeridas.	5	Equipo Leodega	Pendiente

continúa en la siguiente página

Tabla 2.5 (continuación)

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUC-05	Como Cliente, quiero completar un proceso de pago simulado para confirmar y activar mi reserva, para formalizar el acuerdo de alquiler.	5	Equipo Leodega	Pendiente
HUL-06	Como Cliente, quiero ver la información detallada de un espacio disponible desde el móvil, para evaluar si cumple mis necesidades antes de alquilarlo.	2	Equipo Leodega	Pendiente

2.2.3. Sprint 3 — Organizaciones (Multiempresa)

Objetivo del Sprint: Incorporar la operación multiempresa: creación de organizaciones, invitación de miembros, cambio de contexto y reservas a nombre de la organización.

Duración: Día 22 – Día 32 (11 días)

Tabla 2.6: Sprint Backlog – Sprint 3

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUC-14	Como Cliente, quiero aceptar o rechazar una invitación de una organización para unirme como miembro, para decidir si opero bajo su nombre en la plataforma.	3	Equipo Leodega	Pendiente
HUE-01	Como Administrador de Organización, quiero invitar a usuarios clientes a unirse a mi organización como miembros, para que mis colaboradores operen bajo el nombre de la empresa.	5	Equipo Leodega	Pendiente
HUE-02	Como Cliente perteneciente a una organización, quiero cambiar entre mi contexto personal y el de cada organización desde un selector, para operar bajo el nombre correcto sin cerrar sesión.	5	Equipo Leodega	Pendiente
HUE-03	Como Administrador de Organización, quiero acceder a un panel administrativo con miembros, reservas activas e historial, para tener visibilidad y control total de las operaciones de mi empresa.	5	Equipo Leodega	Pendiente
HUE-04	Como Administrador de Organización, quiero crear una organización desde mi cuenta de cliente ingresando sus datos (razón social, RUC y correo), para gestionar alquileres a nombre de mi organización.	5	Equipo Leodega	Pendiente

continúa en la siguiente página

Tabla 2.6 (continuación)

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUE-05	Como Cliente perteneciente a una organización, quiero reservar un almacén disponible a nombre de la organización, para asegurar espacio de almacenamiento para la empresa.	5	Equipo Leodega	Pendiente
HUE-06	Como Cliente perteneciente a una organización, quiero que el sistema me impida eliminar almacenes reservados por la organización, para proteger los contratos empresariales de eliminaciones no autorizadas.	3	Equipo Leodega	Pendiente
HUL-02	Como Cliente, quiero seleccionar la empresa bajo la cual operar dentro de Leodeguita, para ver solo los almacenes e ítems correspondientes a esa empresa.	3	Equipo Leodega	Pendiente

2.2.4. Sprint 4 — Mensajería, Reportes y Reseñas

Objetivo del Sprint: Habilitar la comunicación cliente–gestor (consultas y chat de reserva activa), el reporte de incidentes y el sistema de calificaciones.

Duración: Día 33 – Día 42 (10 días)

Tabla 2.7: Sprint Backlog – Sprint 4

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUG-01	Como Gestor, quiero recibir y responder mensajes de consulta previa de clientes interesados, para brindar información oportuna y generar confianza antes de la reserva formal.	5	Equipo Leodega	Pendiente
HUG-03	Como Gestor, quiero tener un canal de chat activo con mis clientes durante el período de alquiler vigente, para coordinar accesos y resolver incidencias sin salir de la plataforma.	5	Equipo Leodega	Pendiente
HUC-04	Como Cliente, quiero enviar un mensaje al Gestor de un almacén antes de reservar para aclarar dudas, para obtener información que no está en la descripción.	3	Equipo Leodega	Pendiente
HUC-06	Como Cliente, quiero comunicarme con el Gestor mediante un chat vinculado a mi reserva activa, para coordinar accesos y resolver situaciones operativas menores.	5	Equipo Leodega	Pendiente
HUC-07	Como Cliente, quiero ver y gestionar mis reservas activas e históricas desde mi panel personal, para tener visibilidad y control sobre mis contratos de alquiler vigentes.	3	Equipo Leodega	Pendiente

continúa en la siguiente página

Tabla 2.7 (continuación)

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUC-08	Como Cliente, quiero reportar un problema relacionado con un almacén o una reserva activa desde mi panel personal, para notificar al Administrador incumplimientos, daños o situaciones anómalas.	3	Equipo Leodega	Pendiente
HUC-09	Como Cliente, quiero calificar y dejar una reseña sobre el Gestor y el almacén al finalizar mi alquiler, para compartir mi experiencia y aportar a la reputación del Gestor.	3	Equipo Leodega	Pendiente
HUA-04	Como Administrador, quiero revisar y dar seguimiento a los reportes que envían los clientes, pudiendo responder y cambiar su estado, para gestionar incidencias y mantener la calidad del servicio.	3	Equipo Leodega	Pendiente

2.2.5. Sprint 5 — Integración de Inventario y Métricas

Objetivo del Sprint: Incorporar la integración con sistemas de inventario externos (conexión, mapeo y visualización), la importación vía Excel y el panel de métricas del administrador.

Duración: Día 43 – Día 52 (10 días)

Tabla 2.8: Sprint Backlog – Sprint 5

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUC-10	Como Cliente, quiero registrar la URL del endpoint de mi sistema de inventario externo junto con la credencial de acceso, para que Leodega consulte automáticamente los productos almacenados sin modificar mi sistema.	8	Equipo Leodega	Pendiente
HUC-11	Como Cliente, quiero configurar la correspondencia entre los campos devueltos por el endpoint de mi sistema externo y los campos de Leodega, para que mis datos de inventario se registren correctamente.	8	Equipo Leodega	Pendiente
HUC-12	Como Cliente, quiero ver la lista actualizada de productos almacenados en mi espacio reservado, obtenida directamente de mi sistema externo, para controlar mi inventario sin depender de canales externos.	5	Equipo Leodega	Pendiente
HUC-13	Como Cliente, quiero subir un archivo Excel con la lista de mis productos directamente a la plataforma, para registrar mi inventario rápidamente cuando mi sistema externo no expone un endpoint.	5	Equipo Leodega	Pendiente

continúa en la siguiente página

Tabla 2.8 (continuación)

ID	Descripción (Historia de Usuario)	Puntos	Responsable	Estado
HUA-02	Como Administrador, quiero ver un panel de control con métricas clave del estado de la plataforma, para tomar decisiones informadas sobre la operación del negocio y monitorear la rentabilidad mensual.	8	Equipo Leodega	Pendiente
HUL-05	Como Cliente, quiero ver todos los espacios de almacenamiento disponibles en una sola lista desde el móvil, para encontrar un espacio que se ajuste a mis necesidades.	5	Equipo Leodega	Pendiente

2.3. Cronograma del Proyecto

El cronograma se construyó a partir de 38 actividades (**HST-001** a **HST-038**), derivadas de las historias de usuario de la Sección 2.2¹, con sus dependencias, duraciones en días y personal asignado. Para cada actividad se calculó el tiempo más próximo del evento (**EET**), la holgura total y el tiempo más lejano del evento (**LET**); las actividades con holgura cero pertenecen a la ruta crítica (**RC**) y se marcan con ●.

2.3.1. Tabla de Actividades

Tabla 2.9: Actividades, duraciones, dependencias y análisis CPM del proyecto

ID	Sprint	Predec.	Dur.	Personal	EET	Holgura	LET	RC
HST-001	1	—	3	1	0	0	3	●
HST-002	1	HST-001	5	2	3	0	8	●
HST-003	1	HST-001	5	2	3	0	8	●
HST-004	1	HST-002	5	2	8	39	52	
HST-005	1	HST-002	3	1	8	41	52	
HST-006	1	HST-002, HST-003	2	1	8	42	52	
HST-007	1	HST-002, HST-003	5	2	8	0	13	●
HST-008	1	HST-001	5	2	3	36	44	
HST-009	2	HST-007	5	2	13	0	18	●
HST-010	2	HST-007	3	1	13	34	50	
HST-011	2	HST-009	3	1	18	0	21	●
HST-012	2	HST-010	2	1	16	34	52	
HST-013	2	HST-011	5	2	21	0	26	●
HST-014	2	HST-013	5	2	26	0	31	●
HST-015	2	HST-014	3	1	31	18	52	
HST-016	2	HST-014	3	1	31	15	49	
HST-017	2	HST-016	3	1	34	15	52	

continúa en la siguiente página

¹Los identificadores HST-XXX numeran secuencialmente las actividades del análisis PERT/CPM y del diagrama Activity-on-Arrow; no coinciden con los códigos HUG/HUC/HUA/HUE/HUL de las historias de usuario porque responden a una convención de numeración independiente, propia del análisis de cronograma.

Tabla 2.9 (continuación)

ID	Sprint	Predec.	Dur.	Personal	EET	Holgura	LET	RC
HST-018	3	HST-001	5	2	3	28	36	
HST-019	3	HST-018	5	2	8	31	44	
HST-020	3	HST-018	3	1	8	41	52	
HST-021	3	HST-018	5	2	8	28	41	
HST-022	3	HST-021	3	1	13	28	44	
HST-023	3	HST-019, HST-022	5	2	16	23	49	
HST-024	3	HST-019, HST-014	5	2	31	13	49	
HST-025	3	HST-023, HST-024	3	1	36	13	52	
HST-026	4	HST-011	3	1	21	23	47	
HST-027	4	HST-026	5	2	24	23	52	
HST-028	4	HST-014	5	2	31	11	47	
HST-029	4	HST-028	5	2	36	11	52	
HST-030	4	HST-014	3	1	31	18	52	
HST-031	4	HST-014	3	1	31	15	49	
HST-032	4	HST-031	3	1	34	15	52	
HST-033	4	HST-014	3	1	31	18	52	
HST-034	5	HST-014	8	3	31	0	39	●
HST-035	5	HST-034	8	3	39	0	47	●
HST-036	5	HST-035	5	2	47	0	52	●
HST-037	5	HST-014	5	2	31	16	52	
HST-038	5	HST-007, HST-008, HST-014	8	3	31	13	52	

2.3.2. Diagrama de Red del Proyecto (Activity-on-Arrow)

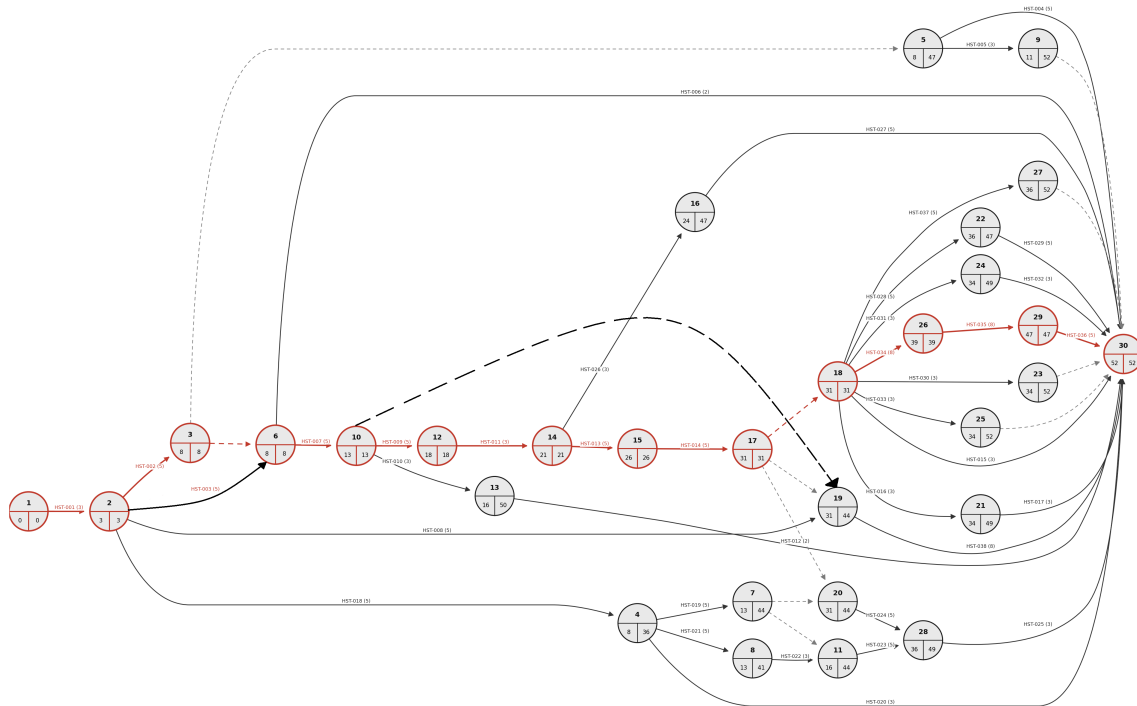


Figura 2.1: Diagrama de red del proyecto (Activity-on-Arrow) con ruta crítica

Ruta crítica: HST-001 → HST-002 → HST-007 → HST-009 → HST-011 → HST-013 → HST-014 → HST-034 → HST-035 → HST-036, con una duración total de **52 días** bajo el supuesto de recursos ilimitados. Esta secuencia va desde la autenticación y el registro de almacenes, pasa por la moderación del Administrador, la búsqueda y el detalle del almacén, la reserva y el pago simulado, y finaliza en el módulo de integración de inventario (conexión externa, mapeo de campos y visualización), que concentra las tres actividades más largas del proyecto (8, 8 y 5 días).

2.3.3. Planificación con Recursos Limitados

Dado que la ruta crítica de 52 días asume disponibilidad ilimitada de personal, se elaboró además una planificación con recursos limitados, restringida a un máximo de **4 integrantes** trabajando en paralelo (el tamaño real del equipo). Una primera versión sin optimizar extiende la duración total del proyecto a **87 días**. Tras nivelar la carga de trabajo y reordenar las actividades no críticas dentro de su holgura disponible, se obtuvo un cronograma optimizado de **85 días**, que ahorra 2 días respecto a la versión sin optimizar sin exceder el límite de 4 personas ni retrasar ninguna actividad de la ruta crítica.

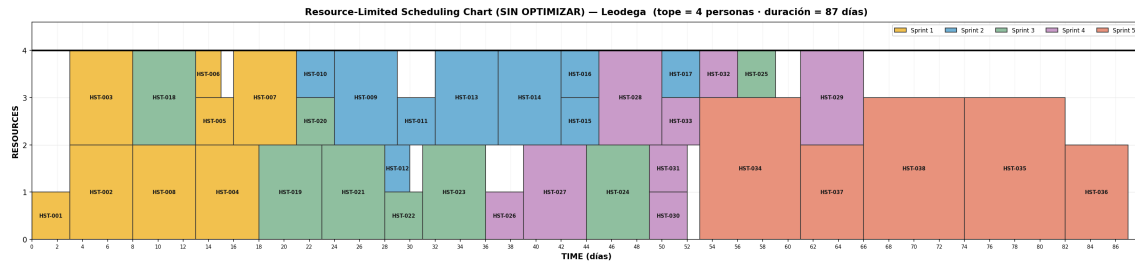


Figura 2.2: Planificación con recursos limitados sin optimizar (límite = 4 personas, duración total = 87 días)

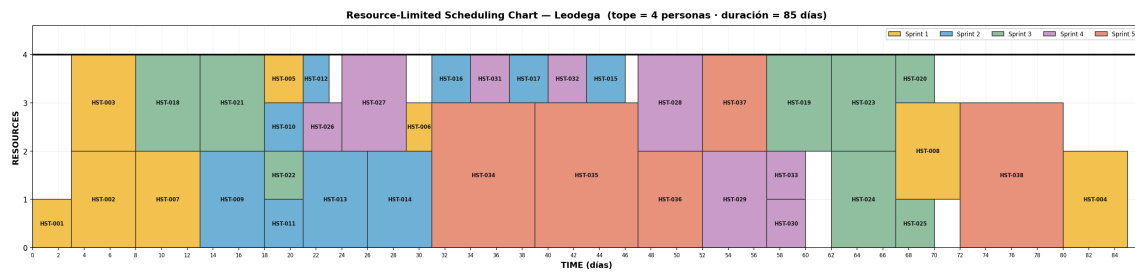


Figura 2.3: Planificación con recursos limitados optimizada (límite = 4 personas, duración total = 85 días)

2.3.4. Diagrama de Gantt

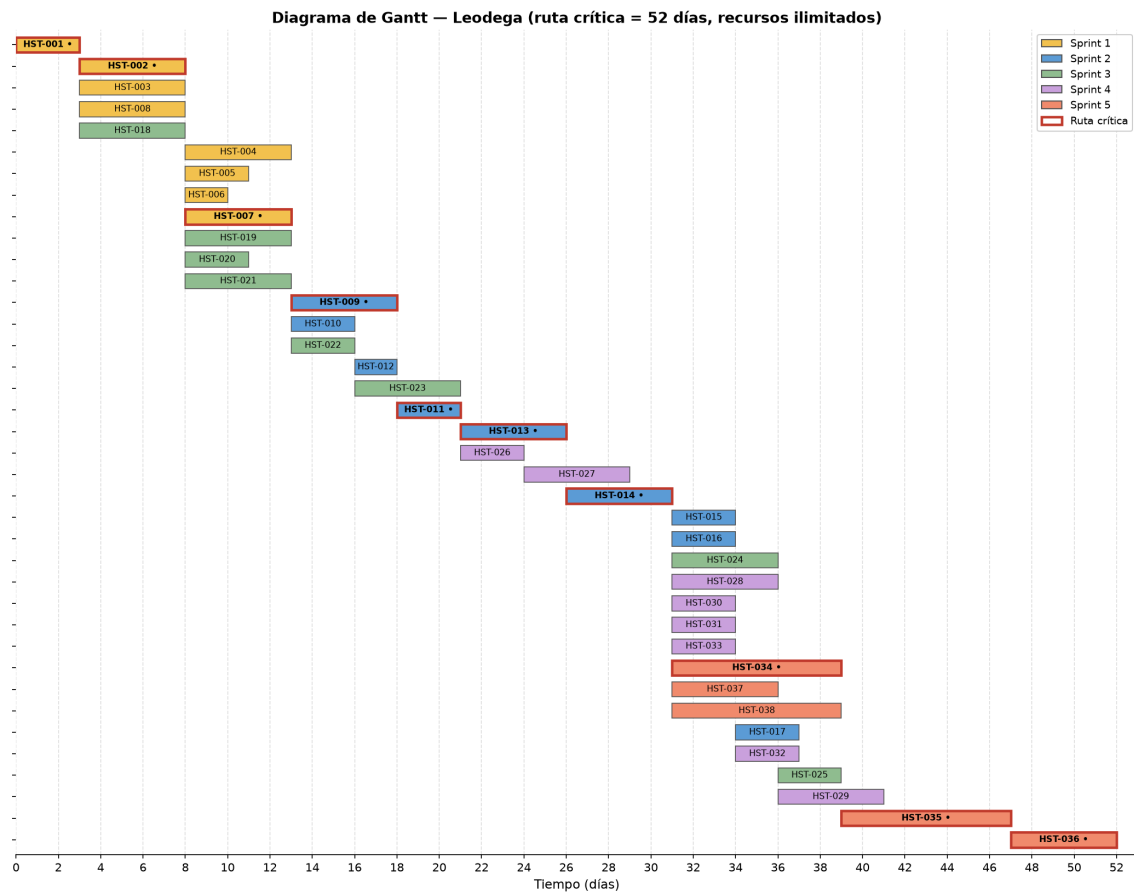


Figura 2.4: Cronograma del proyecto (diagrama de Gantt)

Herramienta utilizada: Generado programáticamente (Python/Matplotlib) a partir de los datos EET/duración/ruta crítica de la Tabla 2.9.

Capítulo 3

Sección B: Modelamiento Estático del Sistema

3.1. Diagrama de Casos de Uso

El modelado de casos de uso abarca el **sistema completo** (heredado + nuevo), diferenciando con color cada elemento en Heredado (entregado por el equipo anterior) y Nuevo/planificado (agregado por este equipo, aún sin código, con fuente en el ERS). Dada la cantidad de casos de uso, se presenta un **diagrama general de contexto** y cinco **diagramas detallados por subsistema**, para preservar la legibilidad exigida por la rúbrica. Todos los elementos fueron verificados contra el código real ([recursos/leodega-codigo](#)) o contra el ERS [21]; la tabla de trazabilidad de cada diagrama cita la fuente exacta.

3.1.1. Actores del Sistema

Tabla 3.1: Actores identificados y su clasificación

Actor	Tipo	Descripción / Fuente
Visitante (no autenticado)	Principal	Usuario sin sesión. Accede a rutas públicas: registro, login, recuperación de contraseña y catálogo público (routes/api.php).
Usuario Registrado	Principal (abstracto)	Generaliza a Cliente, Arrendador y Administrador. Concentra la funcionalidad común autenticada: perfil, contraseña, sesiones, mensajería y notificaciones.
Cliente (Arrendatario)	Principal	Rol tenant . Busca, reserva, paga, califica, reporta e integra inventario (HUC-01..14).
Arrendador (Storage Manager)	Principal	Rol landlord . Publica y administra bodegas y gestiona las reservas recibidas (HUG-01..08).
Administrador de Plataforma	Principal	Rol admin . Modera bodegas, resuelve reportes y gestiona cuentas (HUA-01..04).

continúa en la siguiente página

Tabla 3.1 (continuación)

Actor	Tipo	Descripción / Fuente
Administrador de Organización	Principal (nuevo)	Especialización de Cliente. Crea y administra organizaciones e invita miembros (HUE-01..06). Sin código; fuente ERS.
Servicio de Correo Electrónico	Secundario	Actor externo. Recibe los envíos de correo de recuperación de contraseña (<code>ResetPasswordNotification</code>) y, planificado, las invitaciones a organizaciones.
Sistema de Inventario Externo	Secundario (nuevo)	Actor externo del cliente. Origen/destino de la sincronización de inventario (HUC-10..12). Sin código; fuente ERS.

Nota: el pago es **simulado** (ERS HUC-05, declarado *fuera de alcance* para pasarelas reales), por lo que *no* se modela una pasarela de pago como actor externo. Los flujos móviles de **Leodeguita** (HUL-01..06) reutilizan los mismos casos de uso sobre el backend compartido, por lo que se documentan como un canal de acceso y no como casos de uso duplicados.

3.1.2. Diagrama General (Contexto)

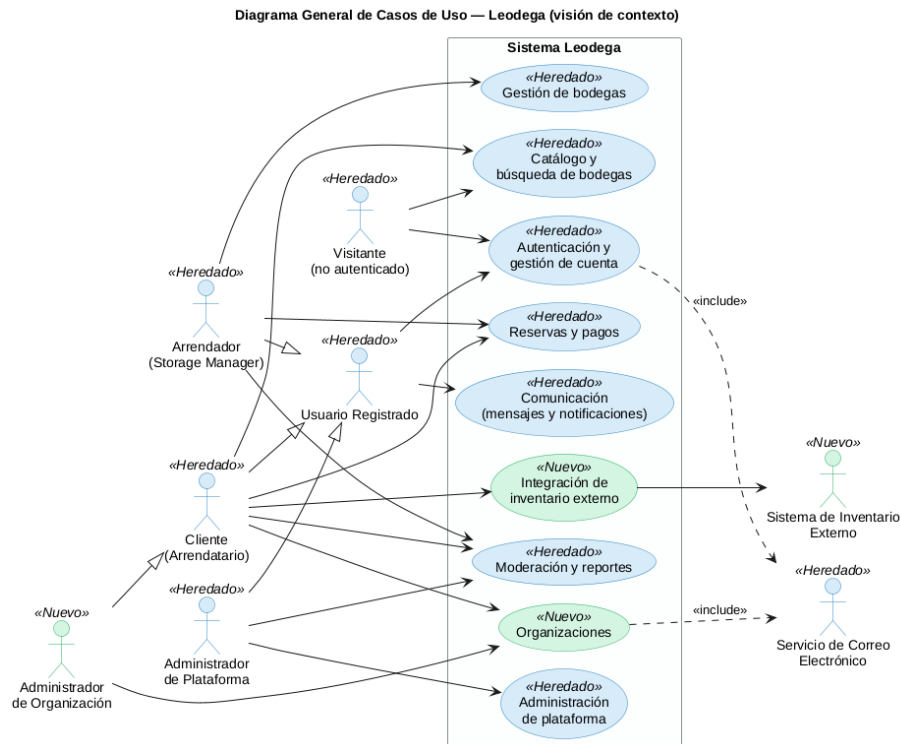


Figura 3.1: Diagrama general de casos de uso del sistema (visión de contexto)

Descripción: Vista de contexto que sitúa a los ocho actores frente a los nueve subsistemas del sistema. Muestra la **jerarquía de actores** (Cliente, Arrendador y Administrador especializan al actor abstracto **Usuario Registrado**; el Administrador de Organización especializa a Cliente), los dos actores externos secundarios (Correo e Inventario Externo, vinculados mediante «include») y la separación heredado/nuevo. Cada subsistema se desglosa en los diagramas de las secciones siguientes.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (renderizado local con `plantuml.jar` sobre OpenJDK). Fuente: [recursos/diagramas_uml/diagrama_casos_uso.puml](#).

3.1.3. Casos de Uso 01 – Autenticación, Cuenta y Comunicación

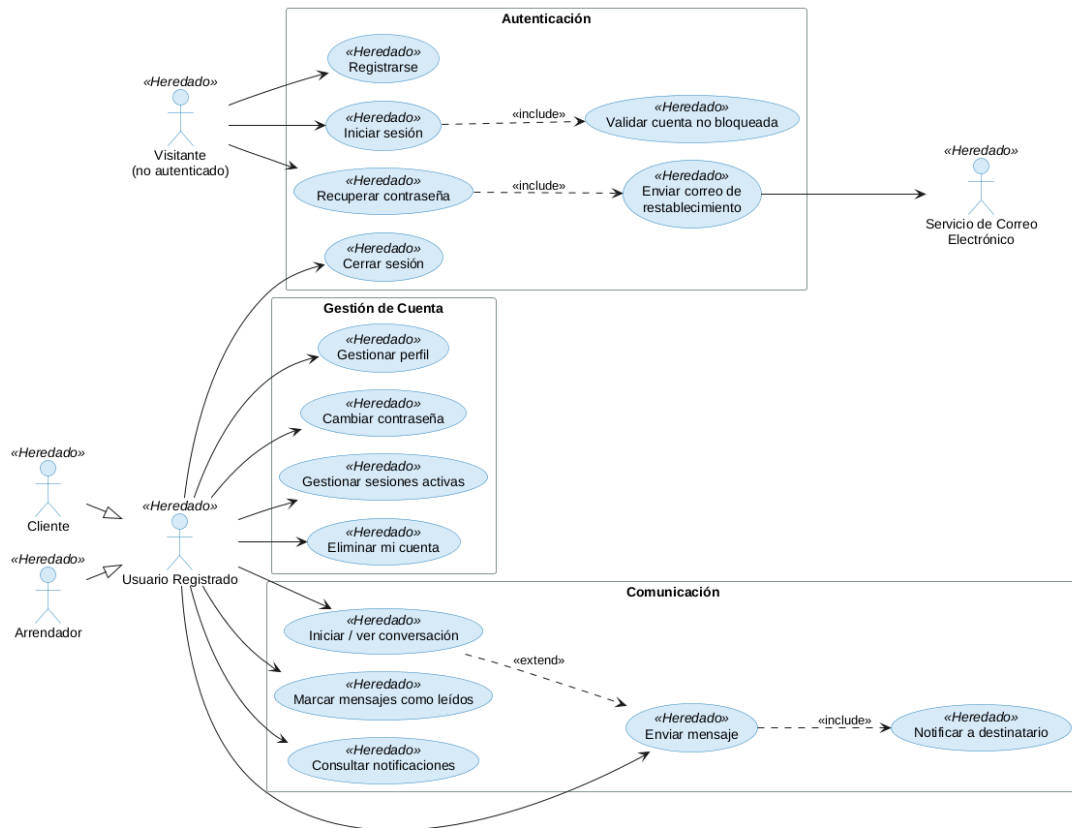


Figura 3.2: Casos de uso de Autenticación, Gestión de Cuenta y Comunicación

Descripción: El Visitante se registra ([routes/api.php](#), POST /user y /register) e inicia sesión; el login «include» la verificación de que la cuenta no esté bloqueada (AuthController::login() lanza AccountBlockedException si state = blocked). La recuperación de contraseña «include» el envío de correo hacia el actor externo (ResetPasswordNotification). El actor abstracto Usuario Registrado agrupa lo común a los tres roles: gestión de perfil, contraseña y sesiones, eliminación de cuenta y la mensajería, donde Enviar mensaje «include» Notificar a destinatario (tipo MESSAGE del enum NotificationType).

Herramienta de modelado utilizada: *PlantUML v1.2026.6*. Fuente: [recursos/diagramas_uml/casos_uso_01.puml](#).

Tabla de trazabilidad

Caso de uso	Fuente (ruta / clase / método)	Estado
Registrarse, Iniciar sesión, Cerrar sesión	<code>backend/routes/api.php;</code> <code>AuthController,</code> <code>UserRegistrationService</code>	Heredado
Validar cuenta no bloqueada	<code>AuthController::login();</code> <code>backend/app/Exceptions/</code> <code>AccountBlockedException.php</code>	Heredado
Recuperar contraseña / Enviar correo	<code>PasswordResetController;</code> <code>backend/app/Notifications/</code> <code>ResetPasswordNotification.php</code>	Heredado
Gestionar perfil / contraseña / sesiones / eliminar cuenta	<code>ProfileController,</code> <code>SecurityController,</code> <code>SessionController,</code> <code>UserController::destroySelf</code>	Heredado
Conversación, Enviar mensaje, Marcar leídos, Notificaciones	<code>ConversationController,</code> <code>MessageController,</code> <code>NotificationsController;</code> <code>backend/</code> <code>app/Enums/NotificationType.php</code>	Heredado

3.1.4. Casos de Uso 02 – Cliente: Catálogo, Reservas, Pagos y Calificaciones

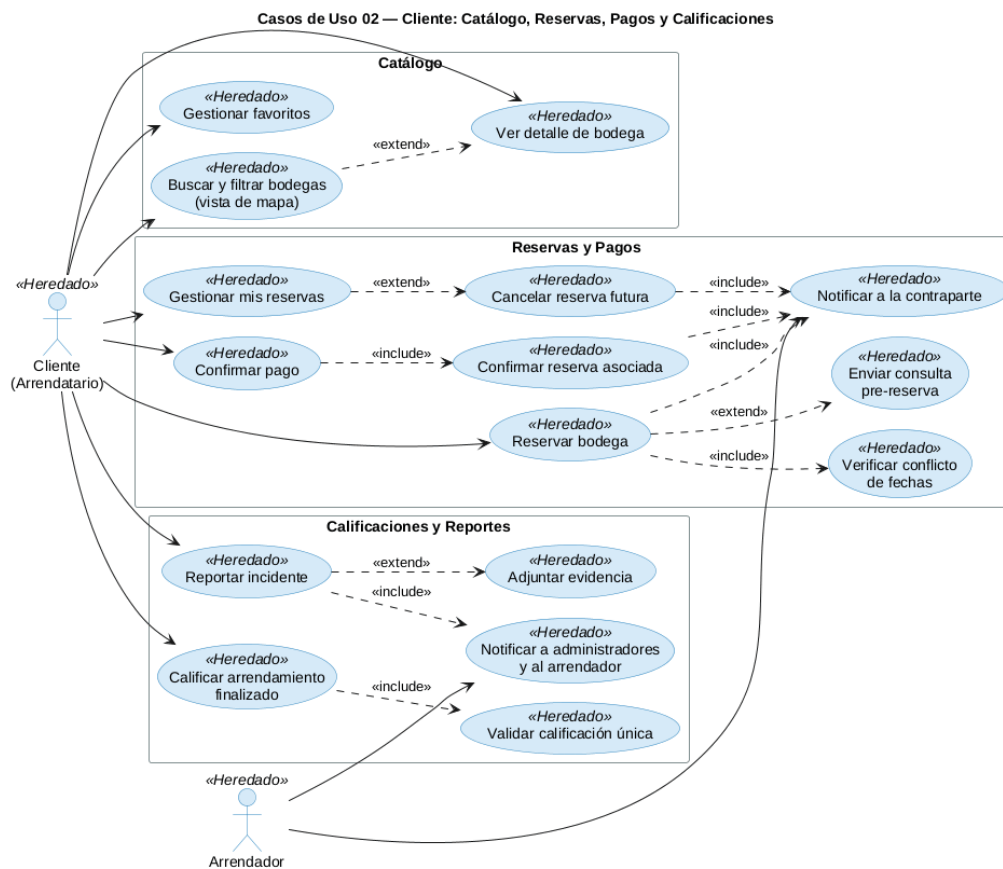


Figura 3.3: Casos de uso del Cliente (arrendatario)

Descripción: Núcleo transaccional del arrendatario. Reservar bodega «include» Verificar conflicto de fechas y Notificar, y «extend» el envío de una consulta pre-reserva (ReservationService). Confirmar pago «include» Confirmar reserva asociada, disparada por PaymentService cuando el pago queda en paid, que a su vez notifica. Calificar arrendamiento «include» Validar calificación única (RatingsService, DuplicateRatingException). Reportar incidente «include» la notificación en cascada a administradores y al arrendador dueño (ReportService::create()).

Herramienta de modelado utilizada: PlantUML v1.2026.6. Fuente: [recursos/diagramas_uml/casos_uso_02.puml](#).

Tabla de trazabilidad

Caso de uso	Fuente (ruta / clase / método)	Estado
Buscar/filtrar bodegas, Ver detalle, Favoritos	<code>StoreRoomsController::index/detail</code> , <code>FavoritesController</code> ; HUC-01/02	Heredado
Reservar bodega / Verificar conflicto de fechas	<code>ReservationsController::store</code> ; <code>backend/app/Services/ReservationService.php</code>	Heredado
Confirmar pago / Confirmar reserva asociada	<code>PaymentsController::store</code> ; <code>backend/app/Services/PaymentService.php</code> → <code>ReservationService::confirm</code>	Heredado
Gestionar mis reservas / Cancelar reserva futura	<code>ReservationsController::tenantIndex</code> ; HUC-07	Heredado
Calificar arrendamiento / Validar calificación única	<code>RatingsController</code> ; <code>backend/app/Services/RatingsService.php</code> ; <code>DuplicateRatingException</code>	Heredado
Reportar incidente / Notificar en cascada	<code>ReportsController::store</code> ; <code>backend/app/Services/ReportService.php</code>	Heredado

3.1.5. Casos de Uso 03 – Arrendador: Gestión de Bodegas y Reservas Recibidas

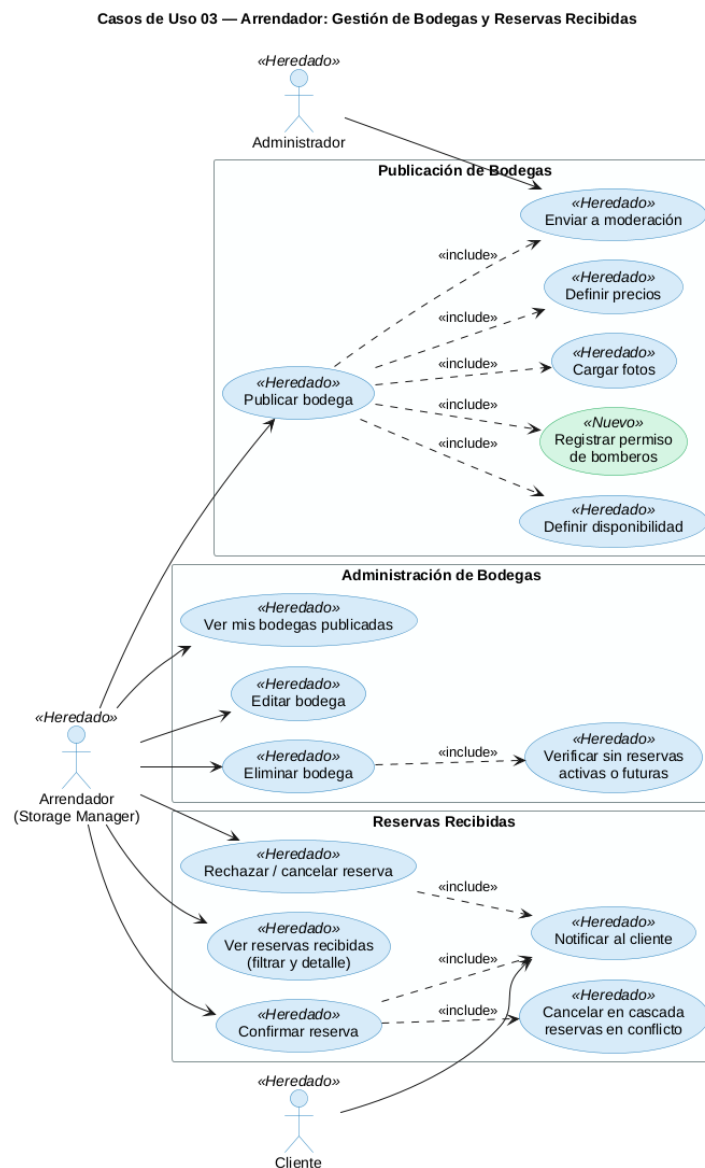


Figura 3.4: Casos de uso del Arrendador (storage manager)

Descripción: Publicar bodega «include» cargar fotos, definir precios, definir disponibilidad y enviar a moderación; incorpora además, como paso Nuevo, Registrar permiso de bomberos (ERS HUG-04/HUA-01), aún ausente en `StoreRoomsController::store`. Eliminar bodega «include» Verificar sin reservas activas o futuras (HUG-07). En reservas recibidas, Confirmar reserva «include» Cancelar en cascada las reservas en conflicto y Notificar al cliente (`ReservationService::confirm()`, extraído de `ReservationsController::updateStatus()`).

Herramienta de modelado utilizada: *PlantUML v1.2026.6*. Fuente: [recursos/](#)

diagramas_uml/casos_uso_03.puml.

Tabla de trazabilidad

Caso de uso	Fuente (ruta / clase / método)	Estado
Publicar / Editar / Eliminar bodega, Ver mis bodegas	<code>StoreRoomsController::store/update/destroy/getByLandlord</code>	Heredado
Cargar fotos, Definir precios, Definir disponibilidad	<code>StorePhotoController, StorePricesController, StoreDisponibilityController</code>	Heredado
Registrar permiso de bomberos	ERS HUG-04 (registro) / HUA-01 (validación), sin código	Nuevo (planificado)
Ver reservas recibidas, Confirmar, Rechazar/cancelar	<code>ReservationsController::landlordIndex/updateStatus; ReservationService</code>	Heredado

3.1.6. Casos de Uso 04 – Administrador: Moderación, Reportes y Cuentas

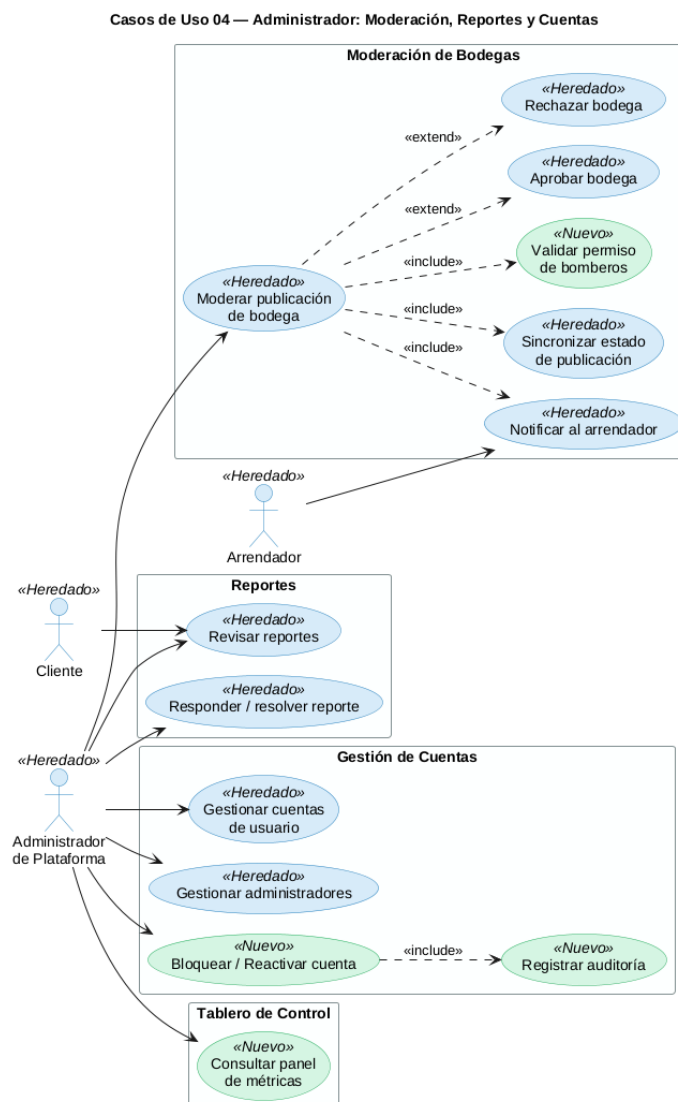


Figura 3.5: Casos de uso del Administrador de Plataforma

Descripción: Moderar publicación de bodega «extend» aprobar/rechazar y «include» Sincronizar estado de publicación y Notificar al arrendador (StoreModerationService); incorpora el paso Nuevo Validar permiso de bomberos (HUA-01). La gestión de cuentas incluye dos casos Nuevos: Bloquear/Reactivar cuenta (que «include» Registrar auditoría, HUA-03) y Consultar panel de métricas (HUA-02). Hoy solo existe el campo `User.state` como base parcial del bloqueo.

Herramienta de modelado utilizada: *PlantUML v1.2026.6*. Fuente: [recursos/diagramas_uml/casos_uso_04.puml](#).

Tabla de trazabilidad

Caso de uso	Fuente (ruta / clase / método)	Estado
Moderar publicación / Sincronizar estado / Notificar	<code>StoreModerationController;</code> <code>backend/app/Services/</code> <code>StoreModerationService.php</code>	Heredado
Validar permiso de bomberos	ERS HUA-01, sin código	Nuevo (planificado)
Revisar / Responder / Resolver reportes	<code>ReportsController::</code> <code>index/update/updateStatus</code>	Heredado
Gestionar cuentas de usuario / administradores	<code>UserController::update/destroy,</code> <code>AdminController</code>	Heredado
Bloquear/Reactivar cuenta / Registrar auditoría	ERS HUA-03; campo <code>User.state</code> existe, flujo con auditoría sin código	Nuevo (planificado)
Consultar panel de métricas	ERS HUA-02, sin código	Nuevo (planificado)

3.1.7. Casos de Uso 05 – Módulos Nuevos: Organizaciones e Inventario Externo

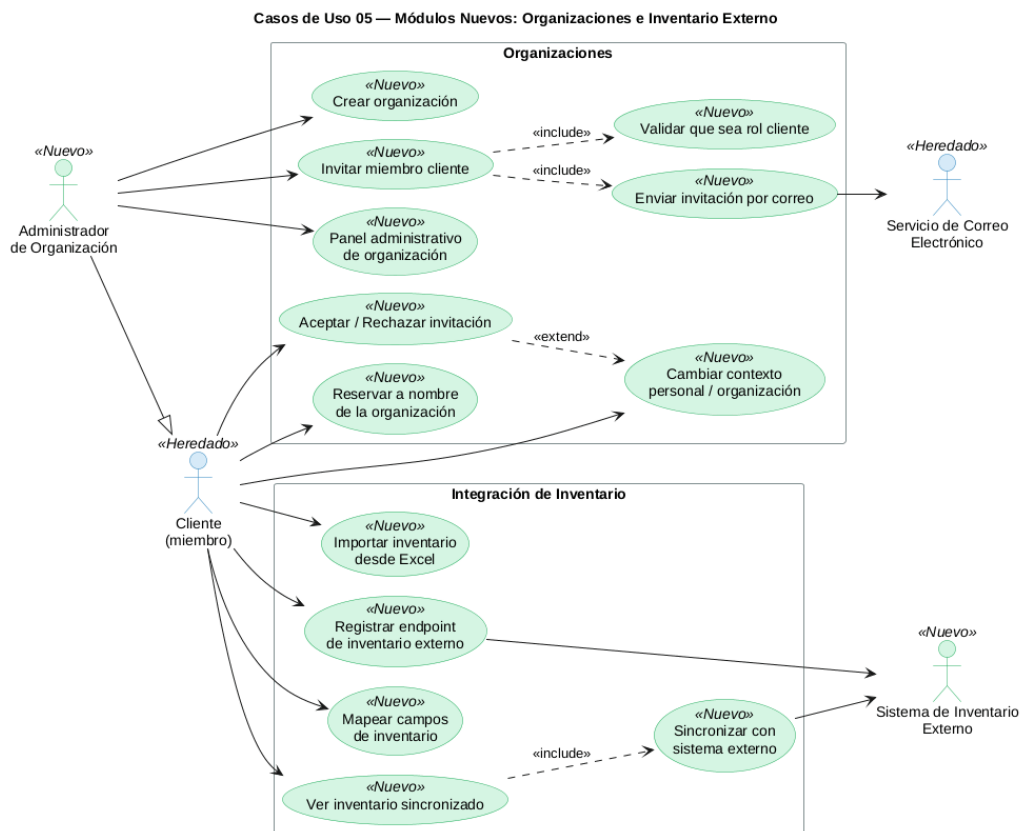


Figura 3.6: Casos de uso de Organizaciones e Integración de Inventario (planificados)

Descripción: Ambos paquetes son íntegramente Nuevos/planificados (ERS, sin código todavía). El Administrador de Organización crea la organización e invita miembros; Invitar miembro cliente «include» Validar que sea rol cliente (un arrendador no puede unirse a una organización, regla del ERS HUE-01) y Enviar invitación por correo hacia el actor externo. El cliente miembro acepta/rechaza la invitación («extend» cambiar contexto), reserva a nombre de la organización, y opera la integración de inventario: Ver inventario sincronizado «include» Sincronizar con sistema externo (actor secundario).

Herramienta de modelado utilizada: *PlantUML v1.2026.6*. Fuente: [recursos/diagramas_uml/casos_uso_05.puml](#).

Tabla de trazabilidad

Caso de uso	Fuente (ERS)	Estado
Crear organización, Panel administrativo de organización	ERS HUE-04, HUE-03/HUE-06	Nuevo (planificado)
Invitar miembro / Validar rol / Enviar invitación	ERS HUE-01; correo por actor externo	Nuevo (planificado)
Aceptar/Rechazar invitación, Cambiar contexto, Reservar por organización	ERS HUC-14, HUE-02, HUE-05	Nuevo (planificado)
Registrar endpoint, Mapear campos, Ver inventario sincronizado, Importar Excel	ERS HUC-10/HUC-11/HUC-12/HUC-13	Nuevo (planificado)

3.1.8. Catálogo de Casos de Uso

La Tabla 3.7 consolida todos los casos de uso del sistema con su actor principal, relaciones «include»/«extend», fuente y estado. Las fichas detalladas (formato [21]) se presentan a continuación para los casos transaccionales medulares; el resto comparte la misma plantilla y puede expandirse a partir de esta tabla.

Tabla 3.7: Catálogo consolidado de casos de uso

ID	Caso de uso	Actor	Relaciones / Fuente	Estado
UC-01	Registrarse	Visitante	POST /user, /register	Her.
UC-02	Iniciar sesión	Visitante	incl. Validar cuenta no bloqueada	Her.
UC-03	Recuperar contraseña	Visitante	incl. Enviar correo → Correo	Her.
UC-04	Cerrar sesión	Usuario Reg.	/logout	Her.
UC-05	Gestionar perfil	Usuario Reg.	/profile	Her.
UC-06	Cambiar contraseña	Usuario Reg.	/password	Her.
UC-07	Gestionar sesiones activas	Usuario Reg.	/sessions	Her.
UC-08	Eliminar mi cuenta	Usuario Reg.	DELETE /account	Her.
UC-09	Gestionar conversaciones y mensajes	Usuario Reg.	incl. Notificar; extend Enviar mensaje	Her.
UC-10	Consultar notificaciones	Usuario Reg.	/notifications	Her.
UC-11	Buscar y filtrar bodegas (mapa)	Visitante / Cliente	extend Ver detalle; HUC-01	Her.
UC-12	Ver detalle de bodega	Visitante / Cliente	HUC-02	Her.
UC-13	Gestionar favoritos	Cliente	/favorites	Her.
UC-14	Reservar bodega	Cliente	incl. Conflicto de fechas, Notificar; extend consulta	Her.
UC-15	Confirmar pago	Cliente	incl. Confirmar reserva; HUC-05	Her.
UC-16	Gestionar mis reservas	Cliente	extend Cancelar reserva futura; HUC-07	Her.
UC-17	Calificar arrendamiento	Cliente	incl. Validar calificación única; HUC-09	Her.
UC-18	Reportar incidente	Cliente	incl. Notificar admins/arrendador; HUC-08	Her.
UC-19	Publicar bodega	Arrendador	incl. fotos/precios/disp./moderación; HUC-04	Her.

continúa en la siguiente página

Tabla 3.7 (continuación)

ID	Caso de uso	Actor	Relaciones / Fuente	Estado
UC-20	Registrar permiso de bomberos	Arrendador	incl. de UC-19; HUG-04/HUA-01	Nuevo
UC-21	Editar bodega	Arrendador	HUG-08	Her.
UC-22	Eliminar bodega	Arrendador	incl. Verificar sin reservas; HUG-07	Her.
UC-23	Ver mis bodegas publicadas	Arrendador	/landlords/{id}/storeRooms	Her.
UC-24	Ver reservas recibidas	Arrendador	HUG-05	Her.
UC-25	Confirmar reserva	Arrendador	incl. Cascada conflictos, Notificar; HUG-02	Her.
UC-26	Rechazar / cancelar reserva	Arrendador	incl. Notificar; HUG-06	Her.
UC-27	Revisar reportes	Administrador	/reports; HUA-04	Her.
UC-28	Responder / resolver reporte	Administrador	/reports/{id}/status; HUA-04	Her.
UC-29	Moderar publicación de bodega	Administrador	incl. Sincronizar, Notificar, Validar permiso; HUA-01	Her.
UC-30	Gestionar cuentas de usuario	Administrador	PUT/DELETE /user	Her.
UC-31	Gestionar administradores	Administrador	/admin	Her.
UC-32	Bloquear / Reactivar cuenta	Administrador	incl. Registrar auditoría; HUA-03	Nuevo
UC-33	Consultar panel de métricas	Administrador	HUA-02	Nuevo
UC-34	Crear organización	Adm. Organización	HUE-04	Nuevo
UC-35	Invitar miembro cliente	Adm. Organización	incl. Validar rol, Enviar correo; HUE-01	Nuevo
UC-36	Aceptar / Rechazar invitación	Cliente	extend Cambiar contexto; HUC-14	Nuevo
UC-37	Cambiar contexto personal/organización	Cliente	HUE-02	Nuevo
UC-38	Reservar a nombre de la organización	Cliente	HUE-05	Nuevo
UC-39	Panel administrativo de organización	Adm. Organización	HUE-03/HUE-06	Nuevo
UC-40	Registrar endpoint de inventario externo	Cliente	→ Inventario Externo; HUC-10	Nuevo
UC-41	Mapear campos de inventario	Cliente	HUC-11	Nuevo
UC-42	Ver inventario sincronizado	Cliente	incl. Sincronizar → Inventario; HUC-12	Nuevo

continúa en la siguiente página

Tabla 3.7 (continuación)

ID	Caso de uso	Actor	Relaciones / Fuente	Estado
UC-43	Importar inventario desde Excel	Cliente	HUC-13	Nuevo

3.1.9. Documentación Detallada de Casos de Uso Medulares

Se documentan en formato extendido [21] los casos de uso transaccionales más representativos del sistema. Los casos restantes de la Tabla 3.7 siguen esta misma plantilla.

Nombre de caso de uso:	UC-14 – Reservar bodega		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	Permite al cliente solicitar la reserva de una bodega disponible en un rango de fechas.		
Actores:	Cliente (principal), Arrendador (secundario, notificado).		
Precondiciones:	1. El cliente está autenticado. 2. La bodega existe y está publicada.		
Postcondiciones:	1. Se crea una Reservations en estado pending . 2. Se notifica al arrendador (RESERVATION_REQUEST).		
Flujo:	1. El cliente selecciona la bodega y las fechas y confirma la solicitud. 2. El sistema (include) verifica que no exista una reserva confirmed solapada en esas fechas. 3. El sistema registra la reserva en estado pending . 4. El sistema (include) notifica al arrendador.		
Flujos Alternativos:	1. (extend) El cliente envía una consulta pre-reserva al arrendador antes de reservar.		
Excepciones:	E1: fechas en conflicto con una reserva confirmada → ReservationConflictException (HTTP 409).		

Requerimientos:	– HUC-03: reserva de bodega con detección de conflicto de fechas. – No se permite solapamiento con reservas <code>confirmed</code> (<code>ReservationService</code>). – El pago se realiza en un paso posterior (UC-15).
------------------------	--

Tabla 3.8. UC-14 – Reservar bodega.

Nombre de caso de uso:	UC-15 – Confirmar pago		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	Registra un pago simulado que, al quedar exitoso, confirma la reserva asociada.		
Actores:	Cliente (principal).		
Precondiciones:	1. El cliente está autenticado y es dueño de la reserva (<code>PaymentsPolicy</code>). 2. La reserva está en estado <code>pending</code>.		
Postcondiciones:	1. El pago queda en estado <code>paid</code>. 2. La reserva asociada pasa a estado <code>confirmed</code>. 3. El cliente es notificado.		
Flujo:	1. El cliente confirma el pago (simulado) de la reserva. 2. El sistema valida la titularidad (<code>PaymentsPolicy</code>). 3. El sistema registra el pago como <code>paid</code> dentro de una transacción. 4. El sistema (include) confirma la reserva (<code>ReservationService::confirm</code>) y notifica.		
Flujos Alternativos:	—		

Excepciones:	E1: fechas ya ocupadas por otra reserva confirmada → <code>ReservationConflictException</code> y <i>rollback</i> de la transacción.
Requerimientos:	– HUC-05: confirmación de pago simulado. – Registrar el pago y confirmar la reserva ocurren en una sola transacción (<code>PaymentService</code>). – No hay pasarela de pago real (fuera de alcance del ERS).

Tabla 3.9. UC-15 – Confirmar pago.

Nombre de caso de uso:	UC-25 – Confirmar reserva (arrendador)		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	El arrendador acepta una solicitud de reserva sobre una de sus bodegas.		
Actores:	Arrendador (principal), Cliente (secundario, notificado).		
Precondiciones:	1. El arrendador está autenticado y es dueño de la bodega (<code>ReservationsPolicy</code>). 2. La reserva está en estado <code>pending</code>.		
Postcondiciones:	1. La reserva pasa a estado <code>confirmed</code>. 2. Las reservas en conflicto se cancelan en cascada. 3. El cliente es notificado.		
Flujo:	1. El arrendador confirma la reserva. 2. El sistema verifica que no exista otra reserva <code>confirmed</code> en conflicto. 3. El sistema marca la reserva como <code>confirmed</code>. 4. El sistema (include) cancela en cascada las reservas en conflicto y notifica al cliente.		

Flujos Alternativos:	1. El arrendador rechaza o cancela la reserva (UC-26).
Excepciones:	E1: otra reserva ya confirmada en esas fechas → <code>ReservationConflictException</code> (HTTP 409).
Requerimientos:	– HUG-02: notificación de pago y activación del contrato. – Solo el dueño de la bodega puede confirmar la reserva (<code>ReservationsPolicy</code>).

Tabla 3.10. UC-25 – Confirmar reserva (arrendador).

Nombre de caso de uso:	UC-18 – Reportar incidente		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	Permite al cliente reportar un incidente sobre una bodega, opcionalmente con evidencia.		
Actores:	Cliente (principal), Administrador y Arrendador (secundarios, notificados).		
Precondiciones:	1. El cliente está autenticado.		
Postcondiciones:	1. Se crea un <code>Reports</code> en estado <code>pending</code> . 2. Se notifica a todos los administradores y al arrendador dueño.		
Flujo:	1. El cliente describe el incidente sobre la bodega y lo envía. 2. El sistema registra el reporte en estado <code>pending</code> . 3. El sistema (include) notifica en cascada a administradores y al arrendador (<code>STORE_REPORTED</code>).		

Flujos Alternativos:	1. (extend) El cliente adjunta evidencia (<code>ReportEvidences</code>).
Excepciones:	E1: datos inválidos → error de validación (HTTP 422).
Requerimientos:	– HUC-08: reporte y seguimiento de incidentes. – La notificación evita duplicar el envío si el arrendador también es administrador (<code>ReportService::create()</code>).

Tabla 3.11. UC-18 – Reportar incidente.

Nombre de caso de uso:	UC-29 – Moderar publicación de bodega		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	El administrador aprueba o rechaza la publicación de una bodega enviada a moderación.		
Actores:	Administrador (principal), Arrendador (secundario, notificado).		
Precondiciones:	1. El administrador está autenticado (<code>role:admin</code>). 2. La bodega está en moderación.		
Postcondiciones:	1. <code>StoreModeration.status</code> queda actualizado. 2. <code>StoreRooms.publication_status</code> queda sincronizado. 3. El arrendador es notificado (<code>STORE_APPROVED/STORE_REJECTED</code>).		

Flujo:	1. El administrador revisa la solicitud de publicación. 2. El sistema (include) valida el permiso de bomberos (<i>planificado, HUA-01</i>). 3. El administrador aprueba (extend) o rechaza (extend) la bodega. 4. El sistema (include) sincroniza el estado de la bodega y notifica al arrendador.
Flujos Alternativos:	1. El administrador rechaza la bodega indicando un motivo.
Excepciones:	E1: usuario sin rol <code>admin</code> → HTTP 403.
Requerimientos:	– HUA-01: validación del permiso de bomberos y aprobación/rechazo de publicaciones. – <code>StoreModerationService</code> mantiene consistentes la moderación y el estado de publicación. – La validación automática del permiso de bomberos aún no está implementada.

Tabla 3.12. UC-29 – Moderar publicación de bodega.

Nombre de caso de uso:	UC-32 – Bloquear / Reactivar cuenta (nuevo)		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	Permite al administrador bloquear o reactivar la cuenta de un usuario, dejando registro de auditoría.		
Actores:	Administrador (principal).		
Precondiciones:	1. El administrador está autenticado (<code>role:admin</code>). 2. La cuenta objetivo existe.		

Postcondiciones:	1. <code>User.state</code> pasa a <code>blocked/active</code>. 2. Se registra la acción en la auditoría. 3. El bloqueo impide el login (<code>AccountBlockedException</code>).
Flujo:	1. El administrador selecciona la cuenta y la acción (bloquear/reactivar). 2. El sistema actualiza <code>User.state</code>. 3. El sistema (include) registra la auditoría de la acción.
Flujos Alternativos:	—
Excepciones:	E1: usuario sin rol <code>admin</code> → HTTP 403.
Requerimientos:	– HUA-03: bloqueo y reactivación de cuentas con auditoría (Nuevo/planificado). – Hoy el guard de login ya rechaza <code>state = blocked</code>; falta el flujo de administración y la auditoría. – El campo <code>User.state</code> ya existe como base parcial.

Tabla 3.13. UC-32 – Bloquear / Reactivar cuenta (nuevo).

Nombre de caso de uso:	UC-35 – Invitar miembro a organización (nuevo)		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	El administrador de una organización invita a un cliente a unirse como miembro.		
Actores:	Administrador de Organización (principal), Servicio de Correo y Cliente invitado (secundarios).		
Precondiciones:	1. La organización existe. 2. El invitado debe tener rol cliente.		

Postcondiciones:	1. La invitación queda en estado pending. 2. El cliente es notificado por panel y por correo.
Flujo:	1. El administrador busca al usuario por correo. 2. El sistema (include) valida que el usuario tenga rol cliente. 3. El sistema registra la invitación en estado pending. 4. El sistema (include) envía la invitación por correo (actor externo).
Flujos Alternativos:	1. El usuario objetivo es un arrendador → el sistema bloquea la acción con un mensaje.
Excepciones:	E1: el correo no corresponde a ningún usuario → error de validación.
Requerimientos:	– HUE-01: invitación de miembros a la organización (Nuevo/planificado, sin código). – Un arrendador (Storage Manager) no puede unirse a organizaciones.

Tabla 3.14. UC-35 – Invitar miembro a organización (nuevo).

Nombre de caso de uso:	UC-42 – Ver inventario sincronizado (nuevo)		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	Muestra al cliente su inventario sincronizado desde un sistema externo previamente configurado.		
Actores:	Cliente (principal), Sistema de Inventario Externo (secundario).		

Precondiciones:	1. El cliente está autenticado. 2. El endpoint de inventario está registrado (UC-40) y los campos mapeados (UC-41).
Postcondiciones:	1. Se muestra el inventario según el mapeo configurado.
Flujo:	1. El cliente abre la sección de inventario. 2. El sistema (include) sincroniza con el sistema externo (actor secundario). 3. El sistema presenta el inventario mapeado.
Flujos Alternativos:	1. Importación manual desde un archivo Excel (UC-43).
Excepciones:	E1: el sistema externo no responde → se informa el error de sincronización.
Requerimientos:	– HUC-10 a HUC-12: registro de endpoint, mapeo y visualización de inventario sincronizado (Nuevo/planificado). – Sin código todavía; el contrato del endpoint externo lo define el cliente.

Tabla 3.15. UC-42 – Ver inventario sincronizado (nuevo).

Nombre de caso de uso:	UC-01 – Registrarse		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	Permite a un visitante crear una cuenta seleccionando su rol (cliente o arrendador).		
Actores:	Visitante (principal).		

Precondiciones:	1. El correo no debe estar registrado previamente.
Postcondiciones:	1. Se crea el User (estado pending por defecto). 2. Se crea su perfil de rol asociado.
Flujo:	1. El visitante elige su rol e ingresa sus datos. 2. El sistema valida los datos (RegisterRequest). 3. El sistema crea la cuenta y el perfil (UserRegistrationService).
Flujos Alternativos:	—
Excepciones:	E1: intento de auto-asignarse rol admin en una petición anónima → rechazado por UserController::store .
Requerimientos:	– Alta de cuenta con selección de rol (registro público). – El alta anónima no puede crear administradores.

Tabla 3.16. UC-01 – Registrarse.

Nombre de caso de uso:	UC-02 – Iniciar sesión		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	Autentica al usuario y emite un token de acceso (Laravel Sanctum).		
Actores:	Visitante (principal).		
Precondiciones:	1. La cuenta existe.		

Postcondiciones:	1. La sesión queda iniciada con un token Bearer.
Flujo:	1. El visitante ingresa correo y contraseña. 2. El sistema valida las credenciales (<code>Auth::attempt</code>). 3. El sistema (include) verifica que la cuenta no esté bloqueada. 4. El sistema emite el token de acceso.
Flujos Alternativos:	—
Excepciones:	E1: credenciales inválidas → HTTP 401. E2: cuenta blocked → <code>AccountBlockedException</code> (HTTP 403).
Requerimientos:	– Autenticación con token (Laravel Sanctum). – Una cuenta con <code>state = blocked</code> no puede iniciar sesión (guard agregado en la refactorización).

Tabla 3.17. UC-02 – Iniciar sesión.

Nombre de caso de uso:	UC-19 – Publicar bodega		
Creado por:	Equipo Leodega	Actualizado por última vez por:	Equipo Leodega
Fecha de creación:	14/08/2026	Última fecha de revisión:	14/08/2026
Descripción:	El arrendador registra una nueva bodega con sus fotos, precios y disponibilidad, y la envía a moderación.		
Actores:	Arrendador (principal), Administrador (secundario, modera).		
Precondiciones:	1. El arrendador está autenticado.		
Postcondiciones:	1. La bodega queda creada y enviada a moderación. 2. Queda pendiente de aprobación del administrador.		

Flujo:	1. El arrendador ingresa dimensiones, ubicación y tarifa. 2. El sistema (include) carga fotos, define precios y disponibilidad. 3. El sistema (include) envía la bodega a moderación.
Flujos Alternativos:	1. (include, planificado) Registrar el permiso de bomberos (UC-20, HUG-04).
Excepciones:	E1: datos inválidos → HTTP 422.
Requerimientos:	– HUG-04: registro de bodega con permiso de bomberos. – La bodega no es visible en el catálogo hasta ser aprobada por un administrador (UC-29). – La captura del permiso de bomberos aún no está implementada en <code>StoreRoomsController::store</code>.

Tabla 3.18. UC-19 – Publicar bodega.

3.2. Diagrama de Clases

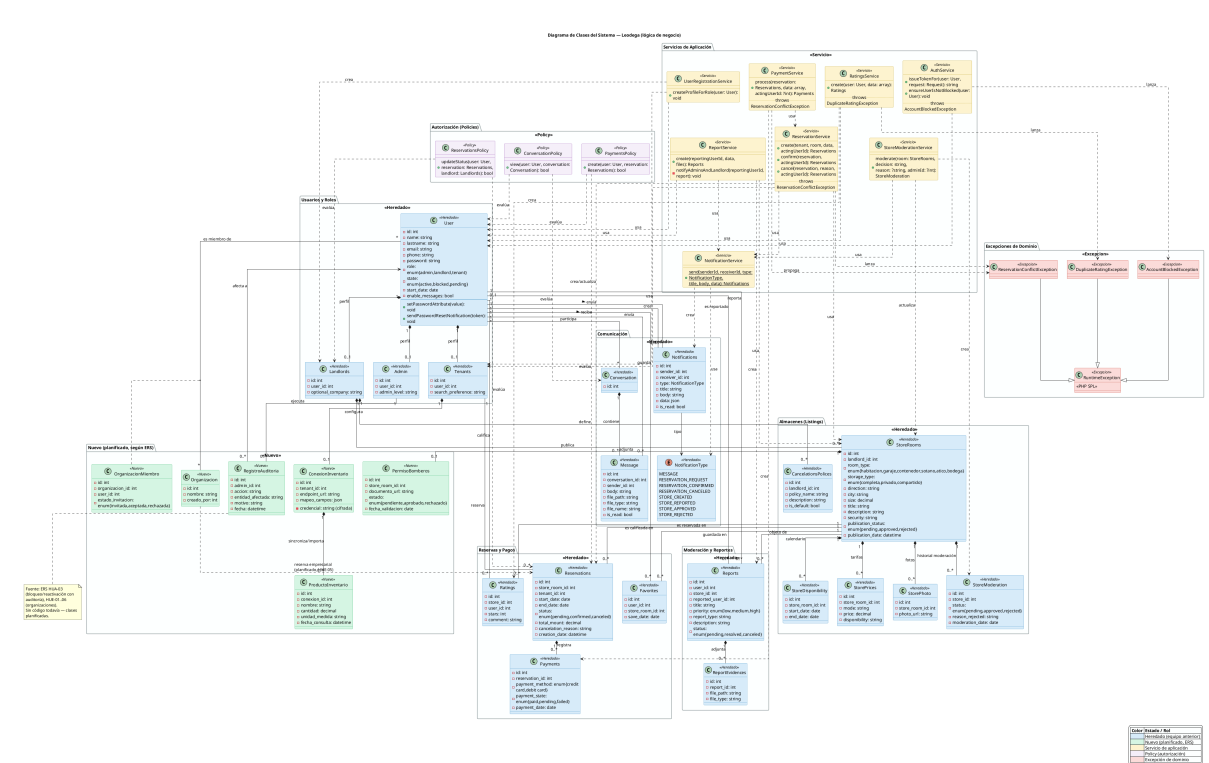


Figura 3.7: Diagrama de clases del sistema (lógica de negocio)

Descripción: El diagrama modela la capa de **lógica de negocio** completa: 19 modelos Eloquent **heredados** (agrupados en los paquetes Usuarios y Roles, Almacenes, Reservas y Pagos, Comunicación, y Moderación y Reportes), los 8 **servicios de aplicación** que concentran las reglas de negocio reales (p. ej. **ReservationService** resuelve conflictos de fechas y cancela en cascada; **ReportService** notifica en cascada a administradores y al arrendador; **StoreModerationService** sincroniza **StoreRooms** con su historial de moderación y notifica al arrendador; **PaymentService** confirma la reserva asociada cuando un pago se registra como exitoso), las 3 **policias** de autorización (incluida **PaymentsPolicy**, que valida que quien paga sea el *tenant* dueño de la reserva), el enum **NotificationType** (incluye **STORE_APPROVED/STORE_REJECTED**) y las 3 **excepciones de dominio** (**DuplicateRatingException**, **ReservationConflictException**, **AccountBlockedException**, todas extienden **RuntimeException**). El patrón **User 1 – (0..1) Landlords/Tenants/Admin** refleja una decisión real de diseño del código: los roles se modelan como **perfiles compuestos** (relación **hasOne**, no herencia de clases), lo que permite añadir nuevos roles sin modificar **User** (ver Open/Closed en la Tabla 3.20). **StoreModerationService**, **PaymentService**, **PaymentsPolicy** y **AccountBlockedException** se clasifican como **heredado** (no **nuevo**): pertenecen a módulos que el ERS ya declara completos (moderación de bodegas, registro de pagos, autenticación), no a requerimientos

nuevos del alcance. Se añade además el paquete **Nuevo (planificado)** con 6 clases derivadas directamente del ERS (§1.3 Alcance) para los módulos aún sin código: **Organizacion**, **OrganizacionMiembro**, **ConexionInventario**, **ProductoInventario**, **PermisoBomberos** y **RegistroAuditoria**.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (renderizado localmente con `plantuml.jar` sobre OpenJDK, motor de layout Graphviz/dot). Fuente: [recursos/diagramas_uml/diagrama_clases.puml](#).

Tabla de trazabilidad

Elemento	Fuente (archivo / clase)	Estado
User, Landlords, Tenants, Admin	<code>backend/app/Models/User.php</code> , <code>Landlords.php</code> , <code>Tenants.php</code> , <code>Admin.php</code>	Heredado
StoreRooms, StorePrices, StorePhoto, StoreDisponibility, StoreModeration, CancelationsPolices	<code>backend/app/Models/StoreRooms.php</code> y modelos relacionados	Heredado
Reservations, Payments, Ratings, Favorites	<code>backend/app/Models/Reservations.php</code> , <code>Payments.php</code> , <code>Ratings.php</code> , <code>Favorites.php</code>	Heredado
Conversation, Message, Notifications, NotificationType	<code>backend/app/Models/Conversation.php</code> , <code>Message.php</code> , <code>Notifications.php</code> ; <code>backend/app/Enums/NotificationType.php</code>	Heredado
Reports, ReportEvidences	<code>backend/app/Models/Reports.php</code> , <code>ReportEvidences.php</code>	Heredado
AuthService, NotificationService, RatingsService, ReportService, ReservationService, UserRegistrationService, StoreModerationService, PaymentService	<code>backend/app/Services/*.php</code>	Heredado
ConversationPolicy, ReservationsPolicy, PaymentsPolicy	<code>backend/app/Policies/*.php</code>	Heredado

Elemento	Fuente (archivo / clase)	Estado
DuplicateRatingException, ReservationConflictException, AccountBlockedException	<code>backend/app/Exceptions/*.php</code>	Heredado
Organizacion, OrganizacionMiembro	ERS §1.3, HUE-01/HUE-04 (sin código)	Nuevo (planifica- do)
ConexionInventario, ProductoInventario	ERS §1.3, HUC-10/HUC-11/HUC-12/HUC-13 (sin código)	Nuevo (planifica- do)
PermisoBomberos	ERS HUG-04 (registro), HUA-01 (validación) (sin código)	Nuevo (planifica- do)
RegistroAuditoria	ERS HUA-03 (bloqueo/reactivación con auditoría) (sin código)	Nuevo (planifica- do)

3.2.1. Cumplimiento de Principios SOLID

Tabla 3.20: Aplicación de principios SOLID en el diseño de clases

Principio	Justificación / Clases involucradas
S – Single Responsibility	Los controladores quedan delgados (validan y delegan) mientras cada regla de negocio no trivial vive en un servicio dedicado con una sola razón para cambiar: ReservationService solo resuelve conflictos de fechas y transiciones de estado de Reservations; RatingsService solo valida la regla “una calificación por usuario/bodega”; ReportService solo orquesta creación de reportes y su notificación en cascada. Antes esta lógica vivía inline en los controladores (ver comentarios de <code>app/Services/*.php</code>); extraerla es la aplicación más clara de SRP en el código real.
O – Open/Closed	NotificationType es un <i>enum</i> PHP: agregar un nuevo tipo de notificación (p. ej. para los módulos nuevos) no requiere modificar <code>NotificationService::send()</code>. De igual forma, el patrón de perfil compuesto (<code>User 1–(0..1)</code> Landlords/Tenants/Admin, en vez de subclases de User) permite agregar un futuro rol (p.ej. OrganizacionAdmin) sin tocar la clase User ni el esquema de autenticación (Sanctum) que depende de ella.
L – Liskov Substitution	Todas las excepciones de dominio (DuplicateRatingException, ReservationConflictException) extienden RuntimeException sin alterar su contrato (no fuerzan <i>catch</i> obligatorio ni cambian la semántica de “excepción no verificada” de PHP), por lo que son sustituibles en cualquier bloque que ya capture RuntimeException o Throwable. Los controladores (ReservationsController, RatingsController) las capturan por su tipo concreto sin depender de detalles internos, respetando el contrato de la clase base.

continúa en la siguiente página

Tabla 3.20 (continuación)

Principio	Justificación / Clases involucradas
I – Interface Segregation	No hay interfaces PHP explícitas en el código actual (todas las clases de servicio y policy son concretas) — ver §3.2.3, “Ausencia de contratos/interfaces”, como oportunidad de mejora identificada. En su forma actual, cada <i>policy</i> (<code>ConversationPolicy</code> , <code>ReservationsPolicy</code>) expone un único método de autorización específico a su caso de uso, en vez de un contrato genérico “autorizable” con métodos que la clase implementante no necesitaría, lo cual sí respeta el <i>espíritu</i> de ISP aunque no exista una interfaz formal.
D – Dependency Inversion	Laravel resuelve los servicios (<code>ReservationService</code> , <code>RatingsService</code> , etc.) por <i>method injection</i> en la firma de las acciones del controlador (p.ej. <code>ReservationsController::store(StoreReservationRequest \$request, ReservationService \$reservationService)</code>), en vez de instanciarlos con <code>new</code> dentro del método. Esto desacopla el controlador de la construcción concreta del servicio y permite sustituirlo (p.ej. en tests) sin modificar el controlador.

3.2.2. Patrones de Diseño Aplicados

Tabla 3.21: Patrones de diseño utilizados en el sistema [22]

Patrón	Categoría	Dónde se aplica / Justificación
Service Layer	Arquitectónico	<code>AuthService</code> , <code>NotificationService</code> , <code>RatingsService</code> , <code>ReportService</code> , <code>ReservationService</code> , <code>UserRegistrationService</code> , <code>StoreModerationService</code> , <code>PaymentService</code> (<code>backend/app/Services/</code>). Separa la lógica de negocio de los controladores HTTP, que quedan como adaptadores delgados de entrada/salida.

continúa en la siguiente página

Tabla 3.21 (continuación)

Patrón	Categoría	Dónde se aplica / Justificación
Template Method	Comportamiento	ApiController (<code>backend/app/Http/Controllers/ApiController.php</code>) define el esqueleto genérico de operaciones CRUD (<code>indexModel</code> , <code>showModel</code> , <code>storeModel</code> , <code>updateModel</code> , <code>destroyModel</code>) parametrizado por <code>\$modelClass</code> ; 13 controladores concretos (<code>AdminController</code> , <code>PaymentsController</code> , <code>TenantsController</code> , <code>RatingsController</code> , <code>ReportsController</code> , etc.) heredan ese esqueleto y solo indican a qué modelo aplicarlo.
Static Factory Method	Creacional	<code>NotificationService::send(...)</code> (método estático) centraliza y abstrae la construcción de instancias de <code>Notifications</code> a partir de un <code>NotificationType</code> , usado por <code>ReservationService</code> y <code>ReportService</code> en vez de que cada uno llame directamente a <code>Notifications::create(...)</code> .
Strategy (autorización)	Comportamiento	Las <i>Policies</i> de Laravel (<code>ConversationPolicy::view</code> , <code>ReservationsPolicy::updateStatus</code> , <code>PaymentsPolicy::create</code> , invocadas vía <code>Gate::authorize</code>) encapsulan cada regla de autorización como una unidad intercambiable y testeable por separado, en vez de condicionales <code>if</code> inline en el controlador (así vivía antes en <code>MessageController</code> y <code>ReservationsController</code> , según sus propios comentarios).

continúa en la siguiente página

Tabla 3.21 (continuación)

Patrón	Categoría	Dónde se aplica / Justificación
Fail-Fast con excepciones de dominio	Comportamiento	<code>DuplicateRatingException</code>, <code>ReservationConflictException</code> y <code>AccountBlockedException</code> (<code>backend/app/Exceptions/</code>) reemplazan códigos de error genéricos por tipos de excepción específicos del dominio, capturados explícitamente en el controlador para traducirlos a la respuesta HTTP correcta (409 Conflict / 403 Forbidden). Coincide con las prácticas recomendadas en [17, 19, 20].

3.2.3. Malos Olores Identificados (Oportunidades de Mejora)

Tabla 3.22: Code smells detectados en la revisión del código real

Code smell	Evidencia / Recomendación
Obsesión por primitivos (<i>Primitive Obsession</i>) en estados	Seis atributos de estado distintos (<code>Reservations.status</code>, <code>StoreRooms.publication_status</code>, <code>StoreModeration.status</code>, <code>Payments.payment_state</code>, <code>Reports.status</code>, <code>User.state</code>) se validan como <i>strings</i> de un <code>enum</code> de base de datos, sin un tipo PHP dedicado que centralice las transiciones válidas — a diferencia de <code>NotificationType</code>, que sí usa un <i>backed enum</i> de PHP. Se recomienda migrar estos campos a <i>enums</i> de PHP 8.1+ para que las transiciones inválidas (ver Diagrama de Estado, Sección C) se detecten en tiempo de análisis estático, no solo en tiempo de ejecución.
Ausencia de contratos/interfaces	Ningún servicio o <i>policy</i> implementa una interfaz PHP (una búsqueda de la palabra clave <code>interface</code> en <code>app/</code> no arroja resultados); todo el desacople depende del contenedor de Laravel resolviendo clases concretas. Funciona hoy porque cada servicio tiene un único caso de uso, pero limita la sustituibilidad explícita (ISP/DIP formales) si el sistema crece con los módulos nuevos del ERS.

continúa en la siguiente página

Tabla 3.22 (continuación)

Code smell	Evidencia / Recomendación
Ausencia de máquina de estados explícita	Las transiciones de <code>Reservations.status</code> (<code>pending/confirmed/canceled</code>) se implementan como actualizaciones de <i>string</i> dispersas en <code>ReservationService</code> (ver Diagrama de Estado, Sección C) en vez de un objeto <code>State</code> explícito (patrón <i>State</i> de GoF). Es una decisión razonable dado el tamaño actual (solo 3 estados), pero sería una refactorización útil si se añade el estado “Finalizada” planificado.

3.3. Diagramas de Objetos

3.3.1. Diagrama de Objetos 01 – Reserva confirmada con pago

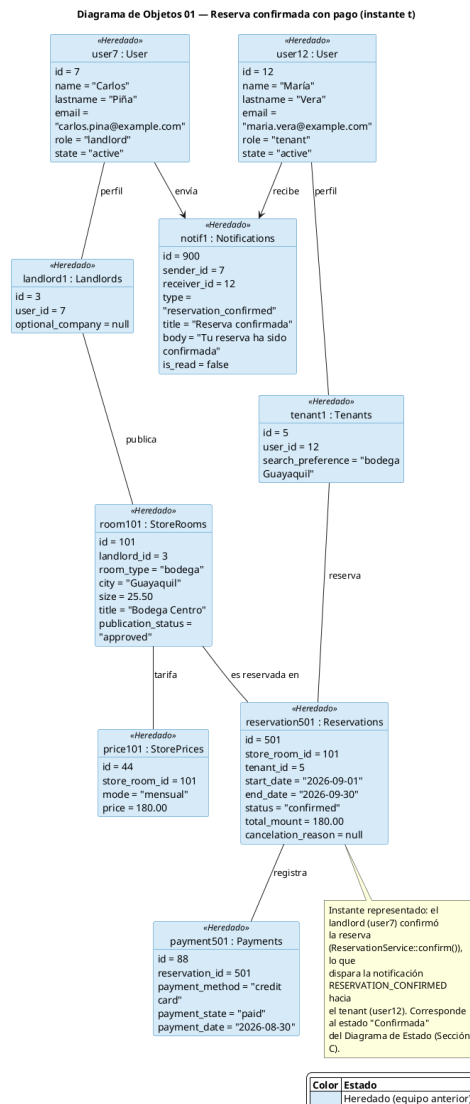


Figura 3.8: Diagrama de Objetos 01 – Reserva confirmada con pago

Descripción: Instancia concreta del núcleo transaccional del sistema en el instante en que el arrendador (user7, landlord) confirma una solicitud de reserva del arrendatario user12 sobre la bodega room101 (ReservationService::confirm()). Se muestran valores realistas y coherentes entre sí (mismos IDs enlazados entre reservation501, tenant1, room101 y payment501) y la notificación RESERVATION_CONFIRMED generada como efecto de la transición. El estado status = confirmed de reservation501 corresponde exactamente al estado *Confirmed* del Diagrama de Estado (Sección C, Figura 4.39).

Herramienta de modelado utilizada: PlantUML v1.2026.6 (motor Graphviz/dot).

Fuente: [recursos/diagramas_uml/objetos_01.puml](#).

Trazabilidad: Todos los tipos de objetos instanciados (`User`, `Landlords`, `Tenants`, `StoreRooms`, `StorePrices`, `Reservations`, `Payments`, `Notifications`) corresponden a clases Heredadas documentadas en la tabla de trazabilidad del Diagrama de Clases (§3.2).

3.3.2. Diagrama de Objetos 02 – Reporte de incidente sobre una bodega

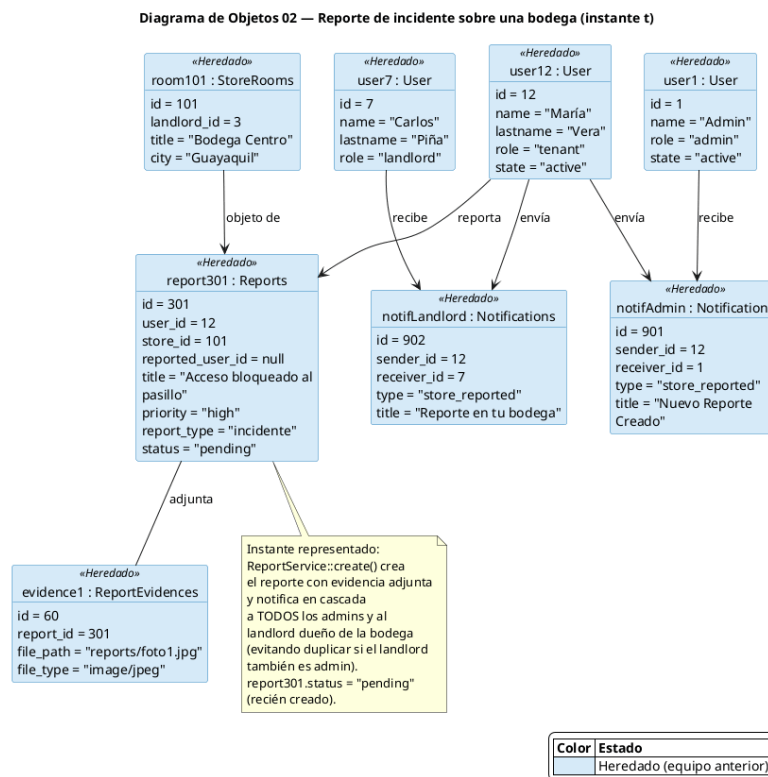


Figura 3.9: Diagrama de Objetos 02 – Reporte de incidente sobre una bodega

Descripción: Instancia concreta del flujo de moderación/reportes: el tenant `user12` reporta un incidente (`report301`) sobre la bodega `room101`, adjuntando evidencia (`evidence1`). Refleja fielmente la orquestación real de `ReportService::create()` ([backend/app/Services/ReportService.php](#)) que notifica en cascada a *todos* los administradores (`user1`) y al arrendador dueño de la bodega (`user7`), evitando duplicar la notificación si el arrendador también fuera administrador. `report301.status = pending` refleja el valor por defecto asignado en la creación.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (motor Graphviz/dot).
 Fuente: [recursos/diagramas_uml/objetos_02.puml](#).

Trazabilidad: Todos los tipos de objetos instanciados (`User`, `StoreRooms`, `Reports`, `ReportEvidences`, `Notifications`) corresponden a clases Heredadas documentadas en la tabla de trazabilidad del Diagrama de Clases (§3.2).

Nota: Agregue tantos diagramas de objetos como aspectos medulares del sistema se deseen ilustrar, duplicando alguno de los dos bloques anteriores.

3.4. Diagrama de Componentes

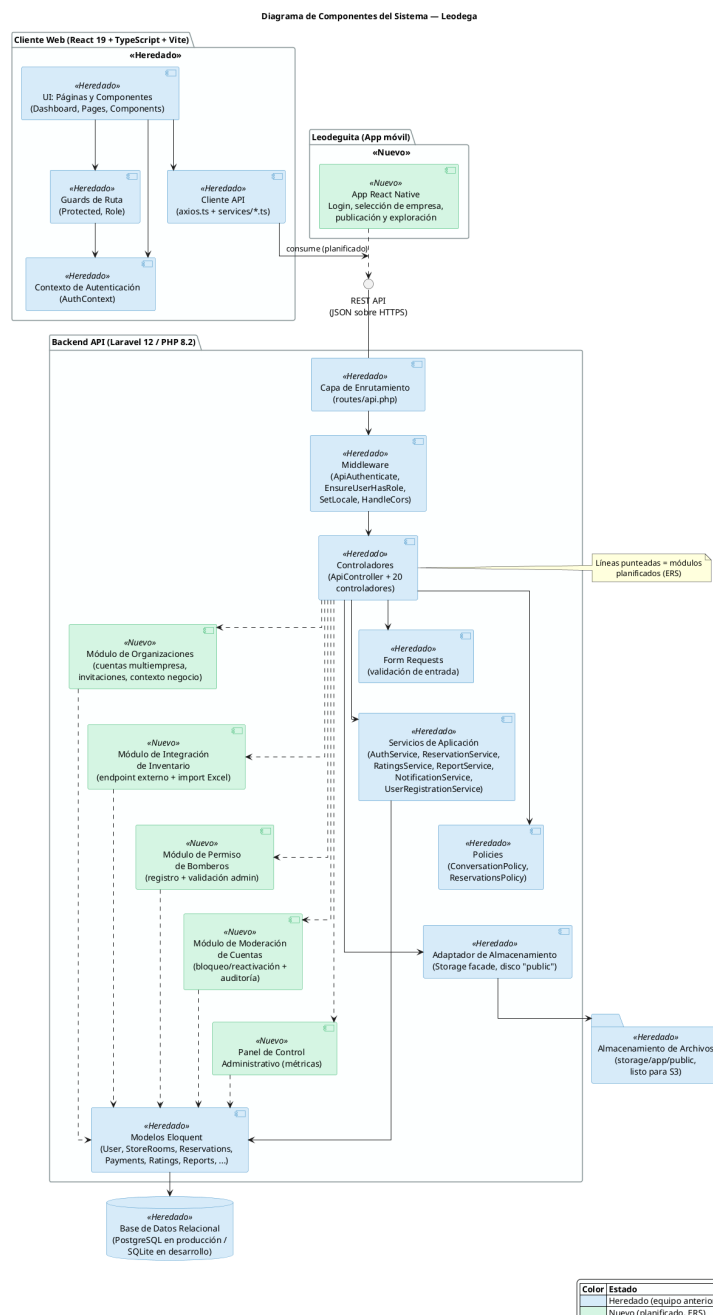


Figura 3.10: Diagrama de Componentes del Sistema

Descripción: El diagrama modela los componentes de software del sistema completo, organizados en tres niveles. El **cliente web** (React 19 + TypeScript + Vite, heredado) se compone de la capa de UI (páginas y componentes, incluidos los dashboards por rol), el contexto de autenticación (AuthContext), los guardias de ruta (Protected, Role) y el cliente API (axios.ts + services/*.ts), que consume la interfaz REST API (JSON sobre HTTPS) expuesta por el backend. El **backend** (Laravel 12 / PHP 8.2, heredado)

sigue una arquitectura en capas: enrutamiento ([routes/api.php](#)) → middleware de autenticación/rol/CORS/locale → controladores delgados (que heredan de `ApiController`, con operaciones CRUD genéricas) → servicios de aplicación que concentran la lógica de negocio (p. ej. `ReservationService` resuelve conflictos de fechas y notifica al arrendador) → modelos Eloquent → base de datos relacional. Un adaptador de almacenamiento (`Storage facade`) gestiona los archivos (fotos de bodegas) en disco local, con soporte de configuración ya listo para S3. Sobre esa base heredada se incorporan cinco módulos **nuevos**, aún no implementados en el código y derivados directamente del ERS (§1.3 Alcance): organizaciones (cuentas multiempresa), integración de inventario externo, permiso de bomberos, moderación de cuentas (bloqueo/reactivación con auditoría) y el panel de control administrativo con métricas; además de un cliente adicional, la app móvil **Leodeguita** (React Native), que según el ERS comparte el mismo backend y modelo de datos que la web.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (renderizado localmente con `plantuml.jar` sobre OpenJDK, motor de layout Graphviz/dot). Fuente: [recursos/diagramas_uml/diagrama_componentes.puml](#).

Tabla de trazabilidad

Elemento	Fuente (archivo / clase)	Estado
UI: Páginas y Componentes	frontend/src/Components/* , Dashboard/* , Pages/*	Heredado
Contexto de Autenticación	frontend/src/context/AuthContext.tsx	Heredado
Guards de Ruta	frontend/src/Routes/Protected.tsx , Role.tsx	Heredado
Cliente API	frontend/src/api/axios.ts , frontend/src/services/*.ts	Heredado
App Leodeguita	ERS §1.3 “Mobile companion”, HUL-01 a HUL-06 (sin código)	Nuevo (planificado)
Capa de Enrutamiento	backend/routes/api.php	Heredado
Middleware (Auth/Rol/Locale/CORS)	backend/app/Http/Middleware/ApiAuthenticate.php , EnsureUserHasRole.php , SetLocale.php ; bootstrap/app.php	Heredado
Controladores	backend/app/Http/Controllers/*.php (extienden <code>ApiController</code>)	Heredado

Elemento	Fuente (archivo / clase)	Estado
Form Requests	<code>backend/app/Http/Requests/*.php</code>	Heredado
Servicios de Aplicación	<code>backend/app/Services/*.php</code> (Auth, Reservation, Ratings, Report, Notification, UserRegistration)	Heredado
Policies	<code>backend/app/Policies/ConversationPolicy.php</code> , <code>ReservationsPolicy.php</code>	Heredado
Modelos Eloquent	<code>backend/app/Models/*.php</code>	Heredado
Adaptador de Almacenamiento	<code>StorePhotoController.php</code> (<code>Storage::disk('public')</code>), <code>config/filesystems.php</code>	Heredado
Base de Datos Relacional	<code>backend/config/database.php</code> (pgsql prod / sqlite dev); ERS §1.2	Heredado
Almacenamiento de Archivos	<code>backend/config/filesystems.php</code> (disco public)	Heredado
Módulo de Organizaciones	ERS §1.3, HUE-01 a HUE-06 (sin código)	Nuevo (planificado)
Módulo de Integración de Inventario	ERS §1.3 (HUC-13/HUC-14, integración externa + import Excel), sin código	Nuevo (planificado)
Módulo de Permiso de Bomberos	ERS §1.3 (registro en HUG, validación en HUA-01), sin código	Nuevo (planificado)
Módulo de Moderación de Cuentas	Campo <code>User.state</code> ya existe (<code>Models/User.php</code>); flujo de bloqueo/reactivación + auditoría: ERS HUA-02, sin código	Heredado (campo) / Nuevo (flujo)
Panel de Control Administrativo	ERS §1.3, HUA-03 (métricas), sin código	Nuevo (planificado)

3.5. Diagrama de Despliegue

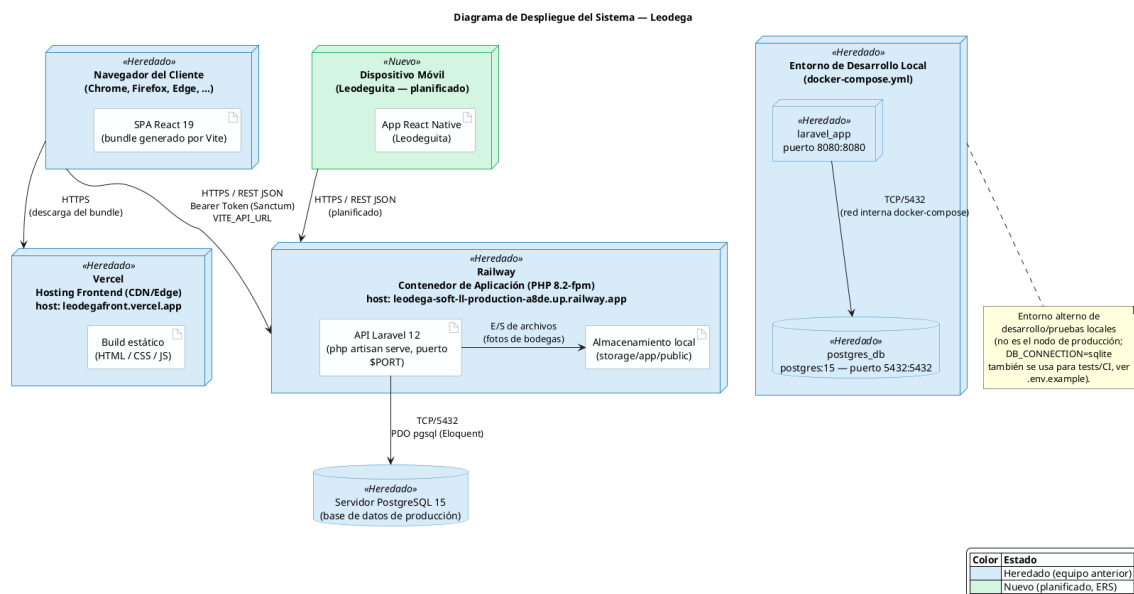


Figura 3.11: Diagrama de Despliegue del Sistema

Descripción: El diagrama refleja la topología de despliegue real declarada en el propio repositorio de código. El **navegador del cliente** descarga el bundle estático de la SPA React 19 (generado por Vite) desde **Vercel**, el nodo de hosting/CDN del frontend (dominio `leodegafont.vercel.app`; ver `frontend/vercel.json` y el origen habilitado en `backend/config/cors.php`). Desde ahí, el navegador consume la API vía HTTPS/REST JSON, autenticado con token Bearer (Laravel Sanctum), apuntando a la URL configurada en `VITE_API_URL`. El backend corre como contenedor de aplicación en **Railway** (`backend/railway.json`, `backend/Procfile`), que arranca el servidor embebido de Laravel con el comando `php artisan serve -host 0.0.0.0 -port $PORT`. Su dominio, `leodega-soft-ll-production-a8de.up.railway.app`, junto con el del frontend, están confirmados como `trustHosts` en `backend/bootstrap/app.php`. Ese contenedor se comunica por TCP/5432 con un servidor **PostgreSQL 15** (motor de base de datos en producción según el ERS §1.2 y `config/database.php`) y escribe las fotos de bodegas en almacenamiento local del propio contenedor (`storage/app/public`). Se documenta además, como evidencia real del repositorio, el **entorno de desarrollo local** definido en `backend/docker-compose.yml` (contenedor `laravel_app` en el puerto 8080 y `postgres_db` en el 5432), que no es el nodo de producción. Como elemento **nuevo** (planificado, sin infraestructura desplegada aún) se incorpora el **dispositivo móvil** que ejecutará la app Leodeguita (React Native), la cual según el ERS compartirá la misma API y modelo de datos que la web.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (renderizado localmente)

con `plantuml.jar` sobre OpenJDK, motor de layout Graphviz/dot). Fuente: [recursos/diagramas_uml/diagrama_despliegue.puml](#).

Tabla de trazabilidad

Elemento	Fuente (archivo / configuración)	Estado
Navegador del Cliente (SPA React)	<code>frontend/index.html</code> , <code>frontend/vite.config.ts</code>	Heredado
Vercel (hosting frontend)	<code>frontend/vercel.json</code> ; <code>backend/config/cors.php</code> (origen <code>https://leodegafront.vercel.app</code>); <code>backend/bootstrap/app.php</code> (<code>trustHosts</code>)	Heredado
Railway (contenedor de aplicación)	<code>backend/railway.json</code> , <code>backend/Procfile</code> , <code>backend/DockerFile</code> ; <code>bootstrap/app.php</code> (<code>trustHosts: leodega-soft-11-production-a8de.up.railway.app</code>)	Heredado
API Laravel (php artisan serve)	<code>backend/railway.json</code> (<code>startCommand</code>), <code>backend/Procfile</code>	Heredado
Almacenamiento local	<code>backend/config/filesystems.php</code> (<code>storage_path('app/public')</code>)	Heredado
Servidor PostgreSQL	ERS §1.2 (“PostgreSQL database in production”); <code>backend/config/database.php</code> (conexión <code>pgsql</code>)	Heredado
Entorno de desarrollo local	<code>backend/docker-compose.yml</code> (servicios <code>app</code> + <code>db</code>)	Heredado
Dispositivo Móvil (Leodeguita)	ERS §1.3, HUL-01 a HUL-06; explícitamente fuera de alcance como app general (“Out of scope”), sin infraestructura de despliegue aún	Nuevo (planificado)

Capítulo 4

Sección C: Modelamiento del Comportamiento del Sistema

4.1. Diagramas de Actividad

Se documenta un diagrama de actividad por cada proceso relevante del sistema.

4.1.1. Diagrama de Actividad 01 – [Nombre del proceso]

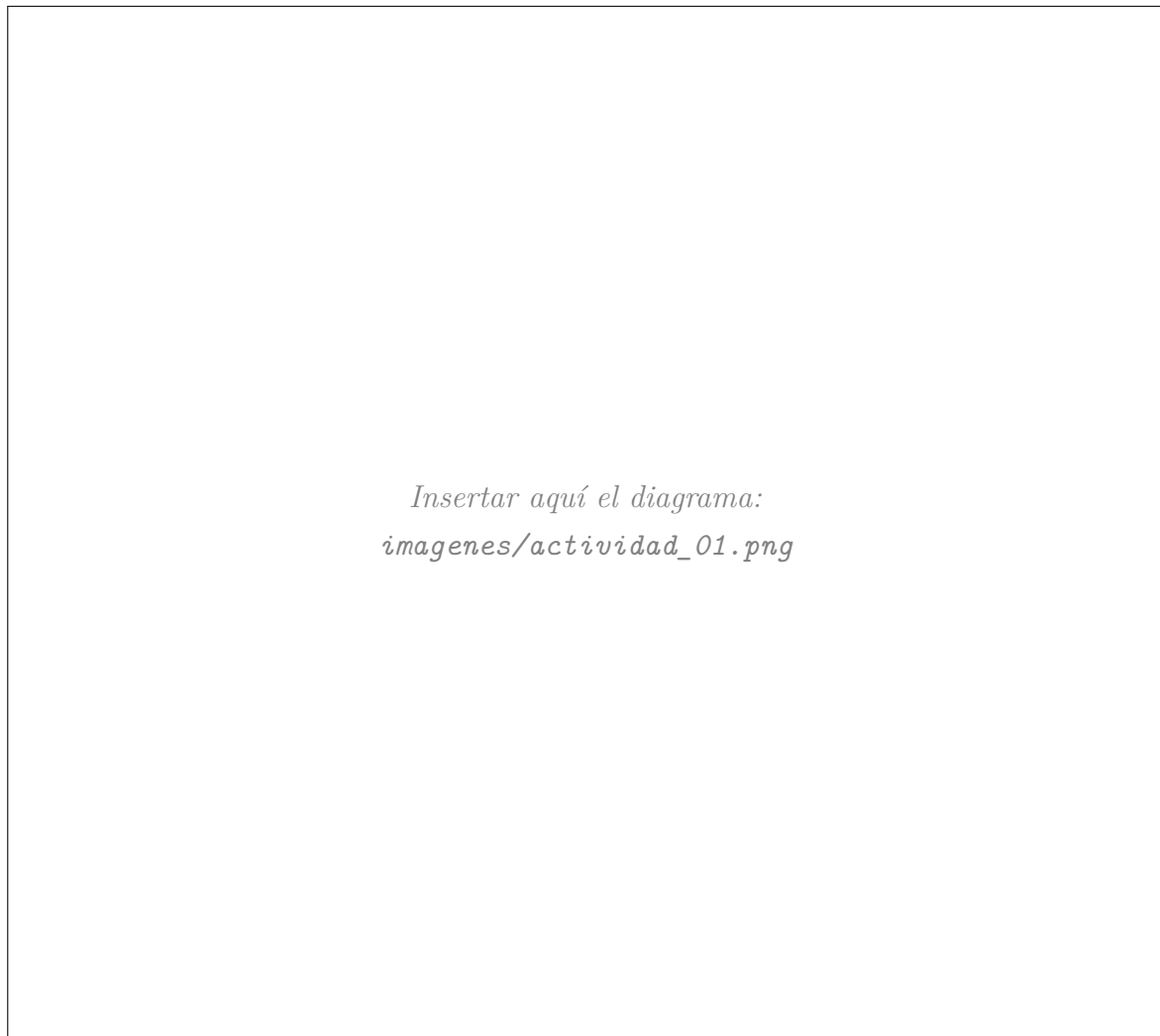


Figura 4.1: Diagrama de Actividad 01 – [Nombre del proceso]

Descripción: [Descripción del proceso de negocio modelado: actores/carriles (swimlanes) involucrados, puntos de decisión y flujos paralelos si aplica.]

Herramienta de modelado utilizada: [Ej. *Visual Paradigm*, *StarUML*, *Draw.io*]

4.1.2. Diagrama de Actividad 02 – [Nombre del proceso]

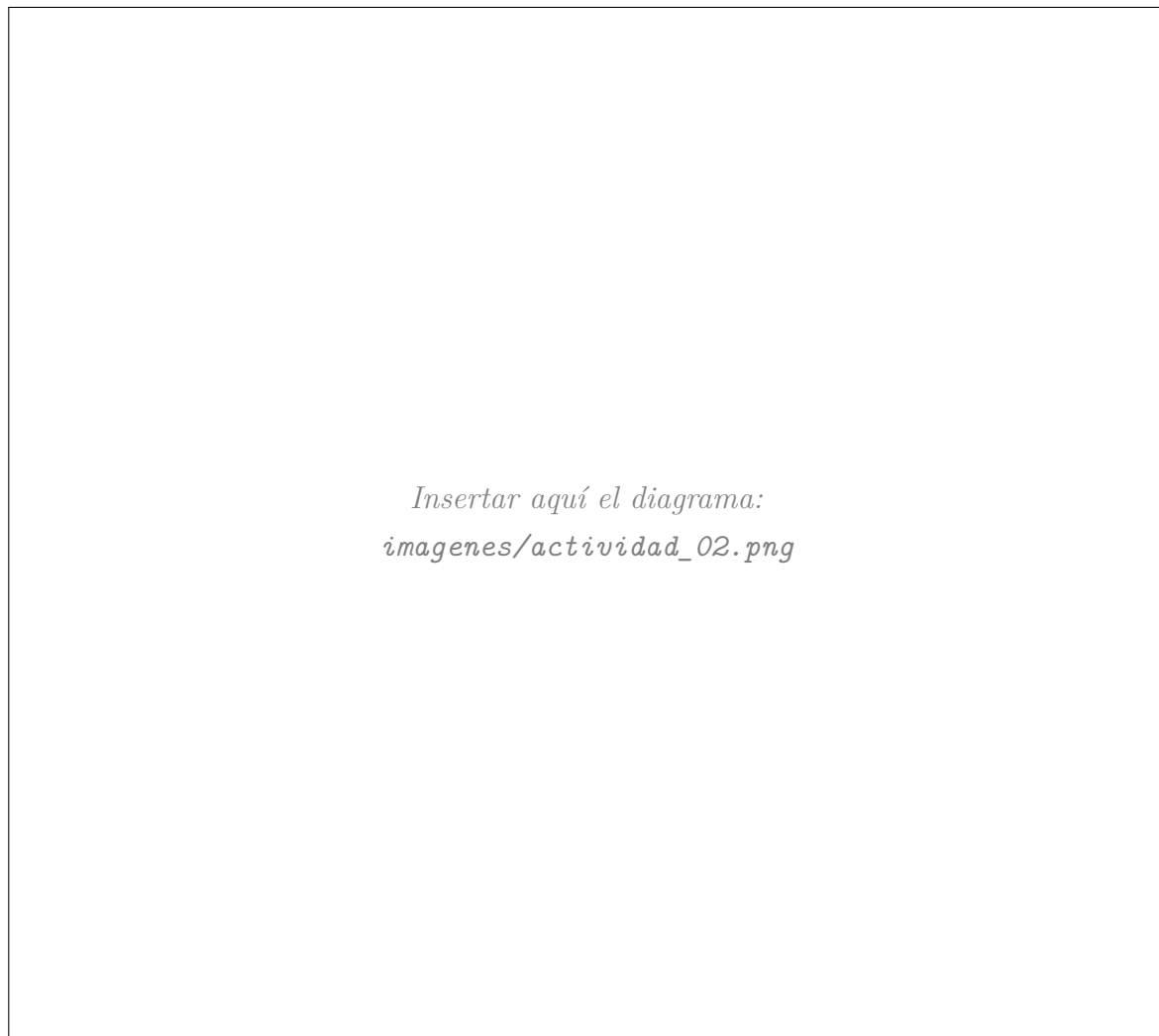


Figura 4.2: Diagrama de Actividad 02 – [Nombre del proceso]

Descripción: [Descripción del proceso.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.1.3. Diagrama de Actividad 03 – [Nombre del proceso]

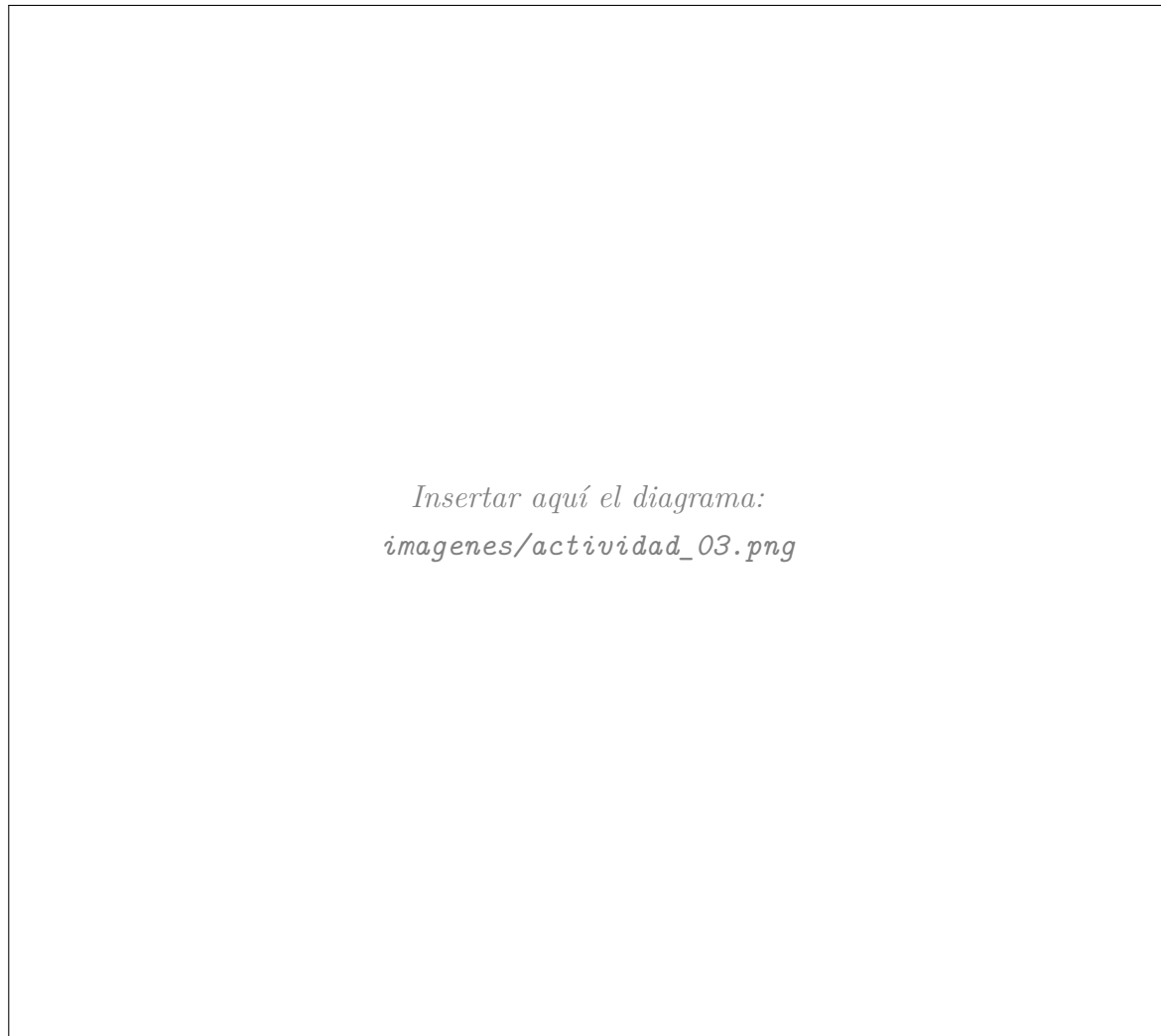


Figura 4.3: Diagrama de Actividad 03 – [Nombre del proceso]

Descripción: [Descripción del proceso.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

Nota: Duplique el llamado a \DiagramaPagina anterior hasta cubrir TODOS los procesos del sistema, según lo requiere la rúbrica.

4.2. Diagramas de Secuencia

Se documentan a continuación los diagramas de secuencia de los algoritmos transaccionales relevantes del sistema. La rúbrica exige un mínimo de **32** diagramas de secuencia; las siguientes páginas se generan automáticamente mediante un bucle de LaTeX (`pgffor`) para facilitar su llenado.

Instrucciones: para cada uno de los 32 bloques generados, reemplace “[Nombre del proceso transaccional]” por el nombre real del escenario (p. ej. “Registrar usuario”, “Procesar pago”), y coloque la imagen exportada en `imagenes/secuencia_NN.png` ($NN = 01..32$). Si su sistema requiere más de 32, agregue llamados adicionales de `\DiagramaPagina` después del bucle.

4.2.1. Diagrama de Secuencia 1 – [Nombre del proceso transaccional 1]

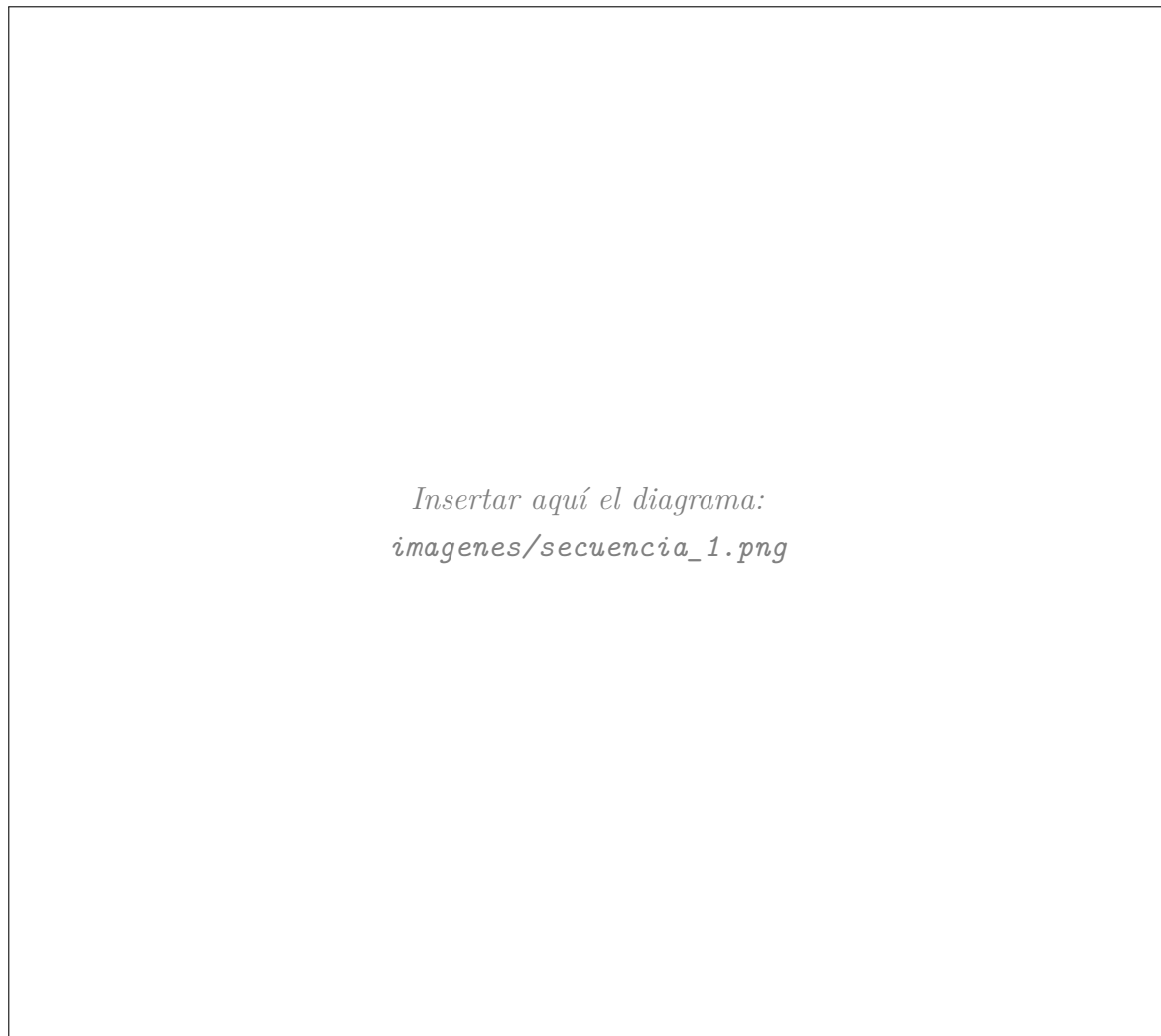


Figura 4.4: Diagrama de Secuencia 1 – [Nombre del proceso transaccional 1]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.2. Diagrama de Secuencia 2 – [Nombre del proceso transaccional 2]



Figura 4.5: Diagrama de Secuencia 2 – [Nombre del proceso transaccional 2]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.3. Diagrama de Secuencia 3 – [Nombre del proceso transaccional 3]



Figura 4.6: Diagrama de Secuencia 3 – [Nombre del proceso transaccional 3]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.4. Diagrama de Secuencia 4 – [Nombre del proceso transaccional 4]



Figura 4.7: Diagrama de Secuencia 4 – [Nombre del proceso transaccional 4]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.5. Diagrama de Secuencia 5 – [Nombre del proceso transaccional 5]

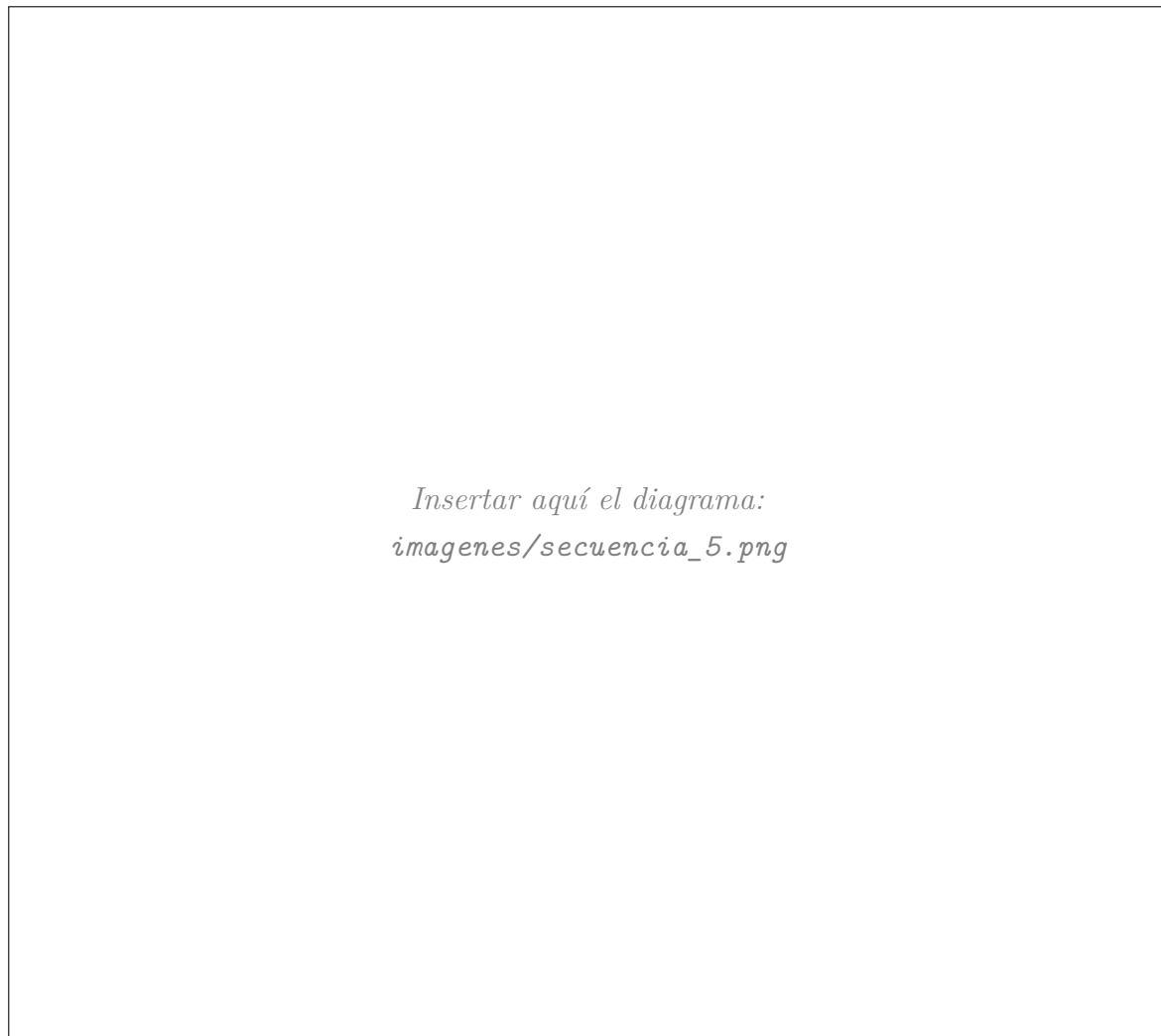


Figura 4.8: Diagrama de Secuencia 5 – [Nombre del proceso transaccional 5]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.6. Diagrama de Secuencia 6 – [Nombre del proceso transaccional 6]



Figura 4.9: Diagrama de Secuencia 6 – [Nombre del proceso transaccional 6]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.7. Diagrama de Secuencia 7 – [Nombre del proceso transaccional 7]

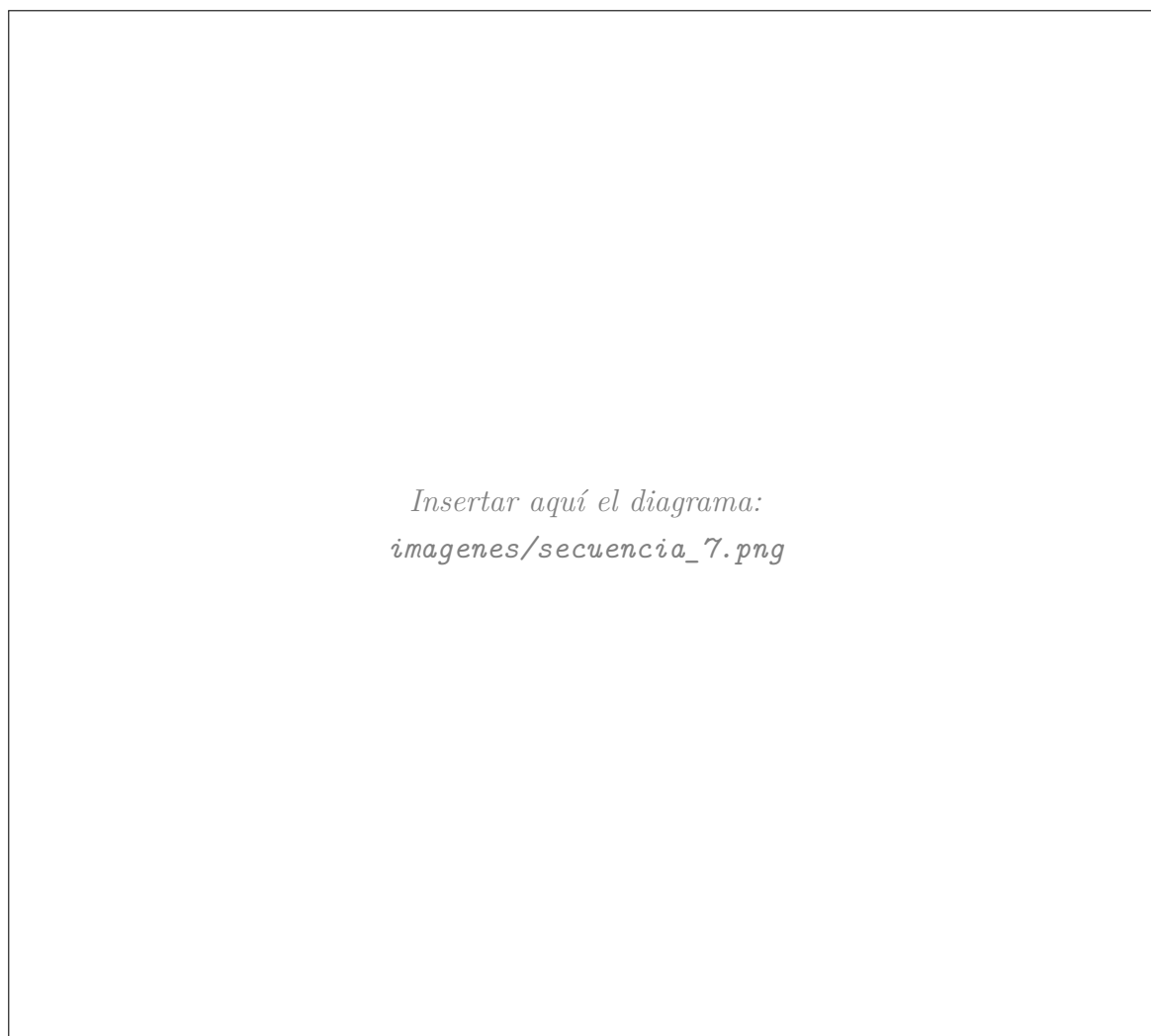


Figura 4.10: Diagrama de Secuencia 7 – [Nombre del proceso transaccional 7]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.8. Diagrama de Secuencia 8 – [Nombre del proceso transaccional 8]



Figura 4.11: Diagrama de Secuencia 8 – [Nombre del proceso transaccional 8]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.9. Diagrama de Secuencia 9 – [Nombre del proceso transaccional 9]



Figura 4.12: Diagrama de Secuencia 9 – [Nombre del proceso transaccional 9]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.10. Diagrama de Secuencia 10 – [Nombre del proceso transaccional 10]

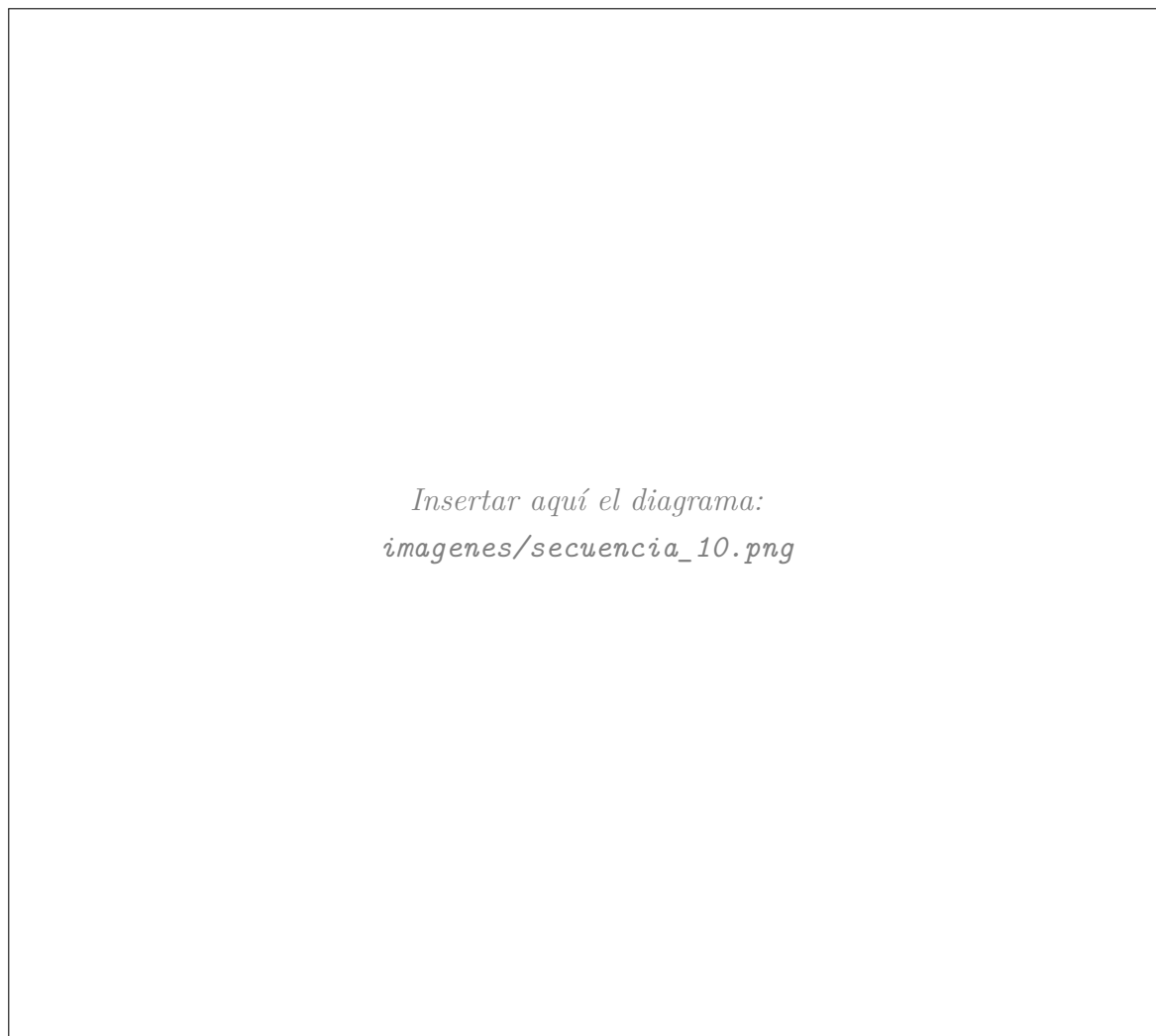


Figura 4.13: Diagrama de Secuencia 10 – [Nombre del proceso transaccional 10]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.11. Diagrama de Secuencia 11 – [Nombre del proceso transaccional 11]



Figura 4.14: Diagrama de Secuencia 11 – [Nombre del proceso transaccional 11]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.12. Diagrama de Secuencia 12 – [Nombre del proceso transaccional 12]

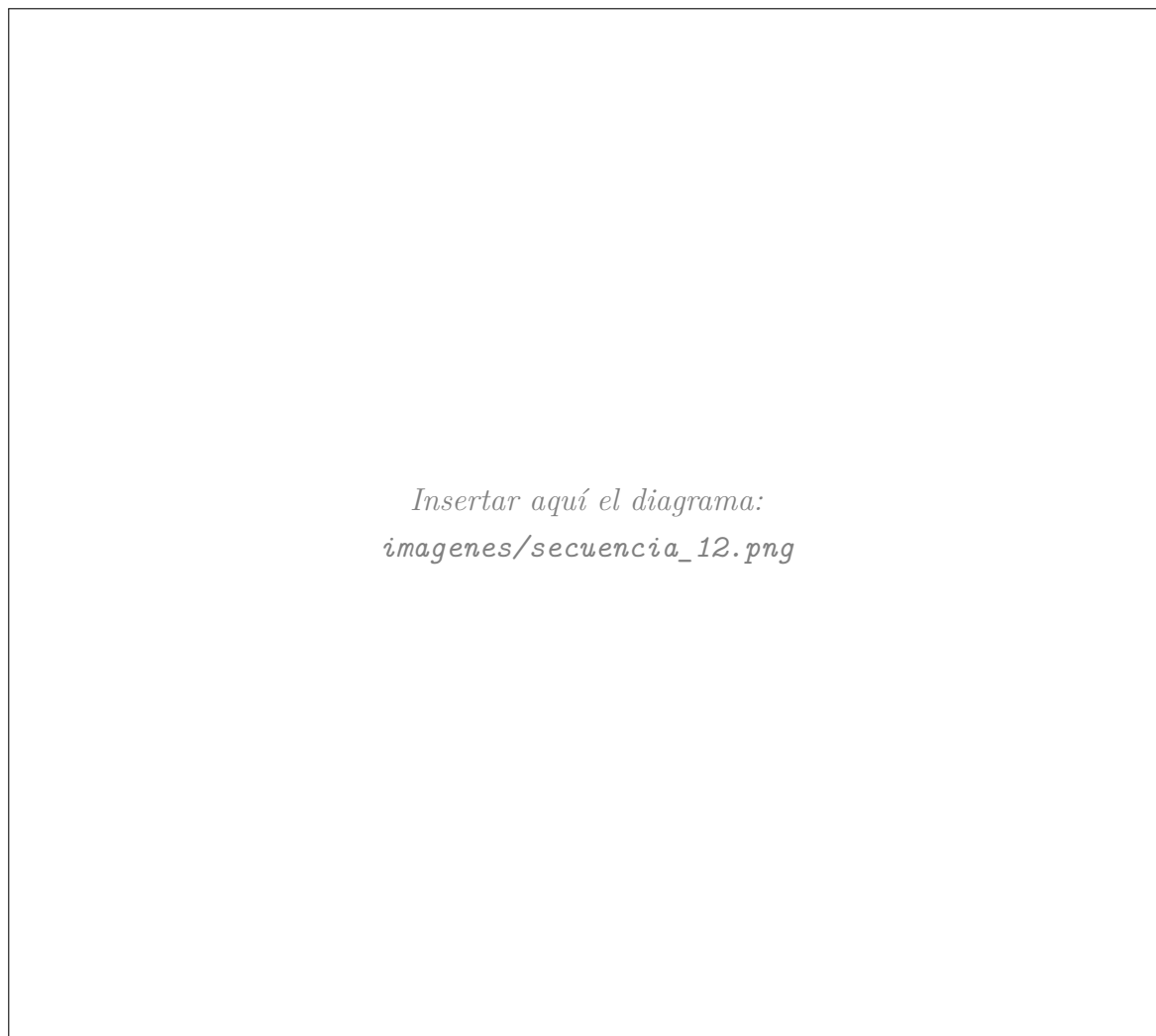


Figura 4.15: Diagrama de Secuencia 12 – [Nombre del proceso transaccional 12]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.13. Diagrama de Secuencia 13 – [Nombre del proceso transaccional 13]

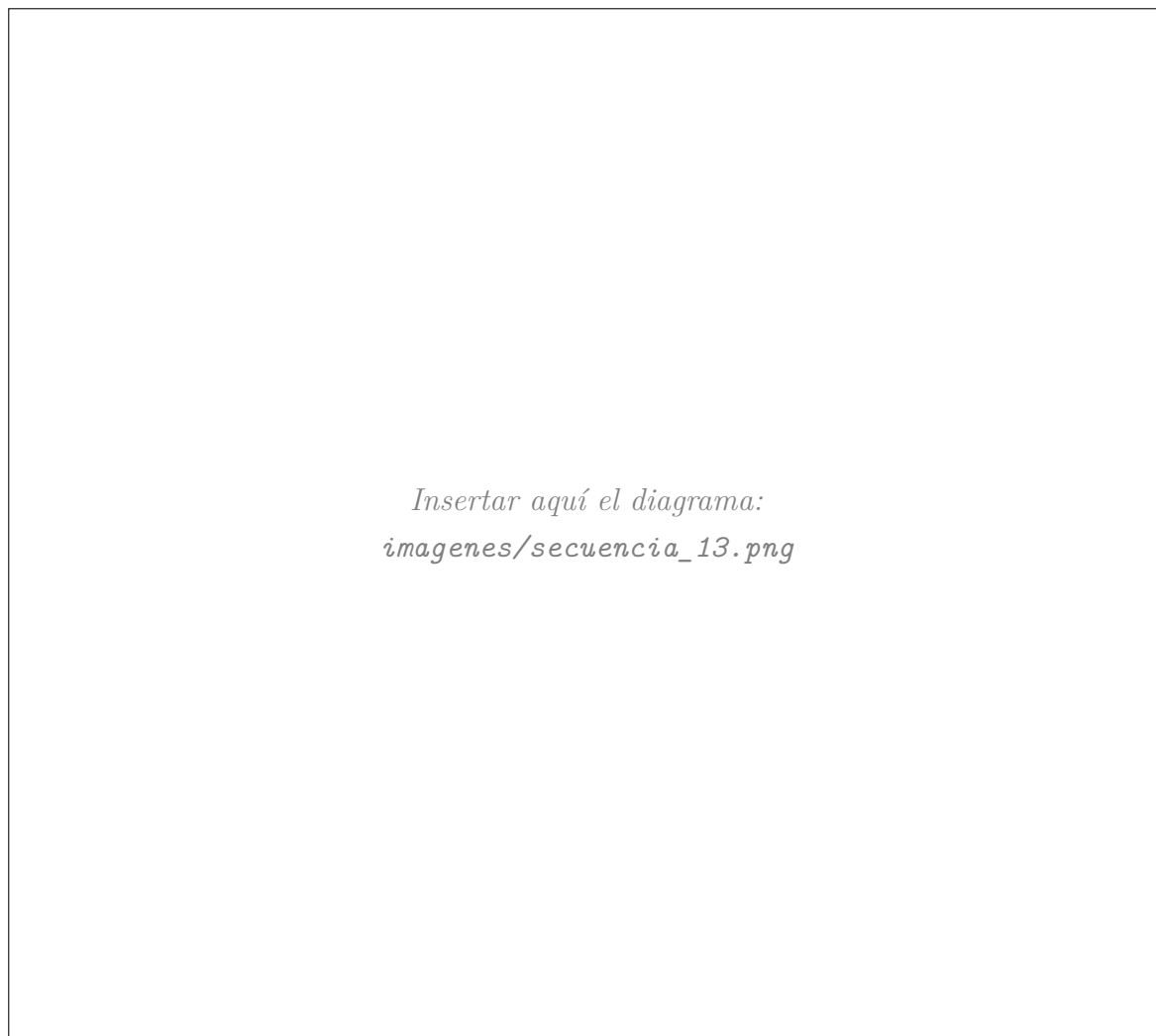


Figura 4.16: Diagrama de Secuencia 13 – [Nombre del proceso transaccional 13]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.14. Diagrama de Secuencia 14 – [Nombre del proceso transaccional 14]



Figura 4.17: Diagrama de Secuencia 14 – [Nombre del proceso transaccional 14]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.15. Diagrama de Secuencia 15 – [Nombre del proceso transaccional 15]

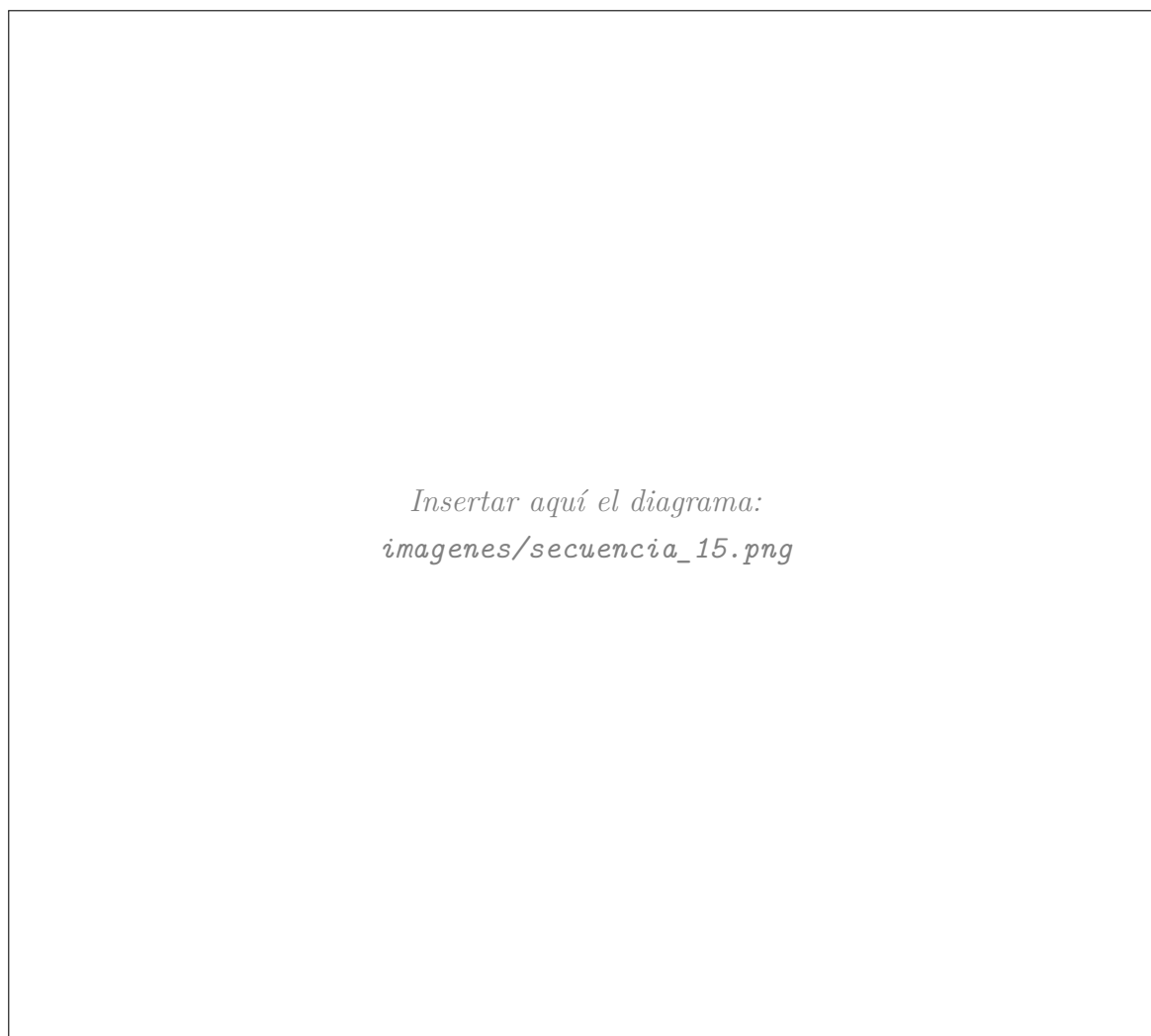


Figura 4.18: Diagrama de Secuencia 15 – [Nombre del proceso transaccional 15]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.16. Diagrama de Secuencia 16 – [Nombre del proceso transaccional 16]

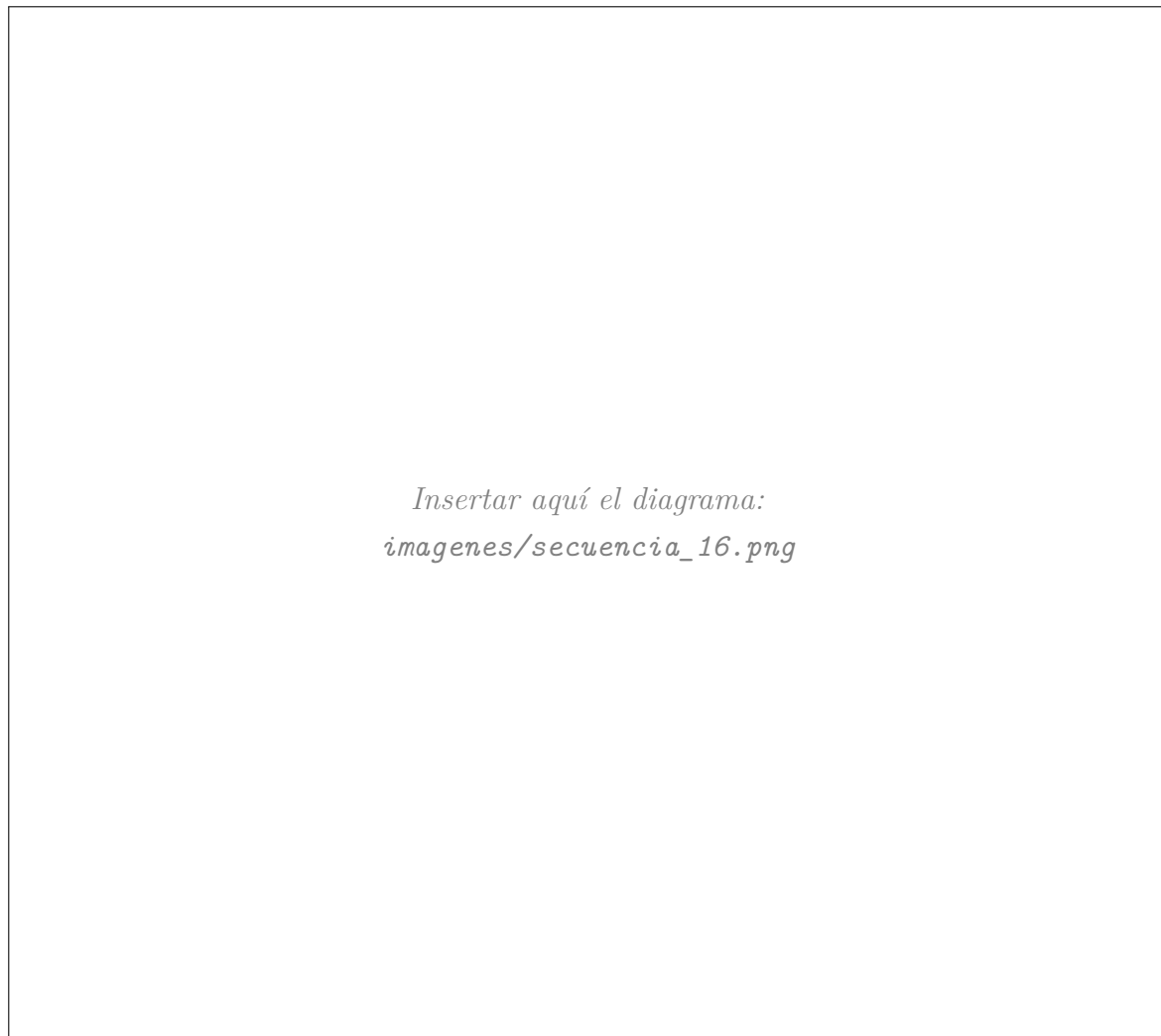


Figura 4.19: Diagrama de Secuencia 16 – [Nombre del proceso transaccional 16]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.17. Diagrama de Secuencia 17 – [Nombre del proceso transaccional 17]

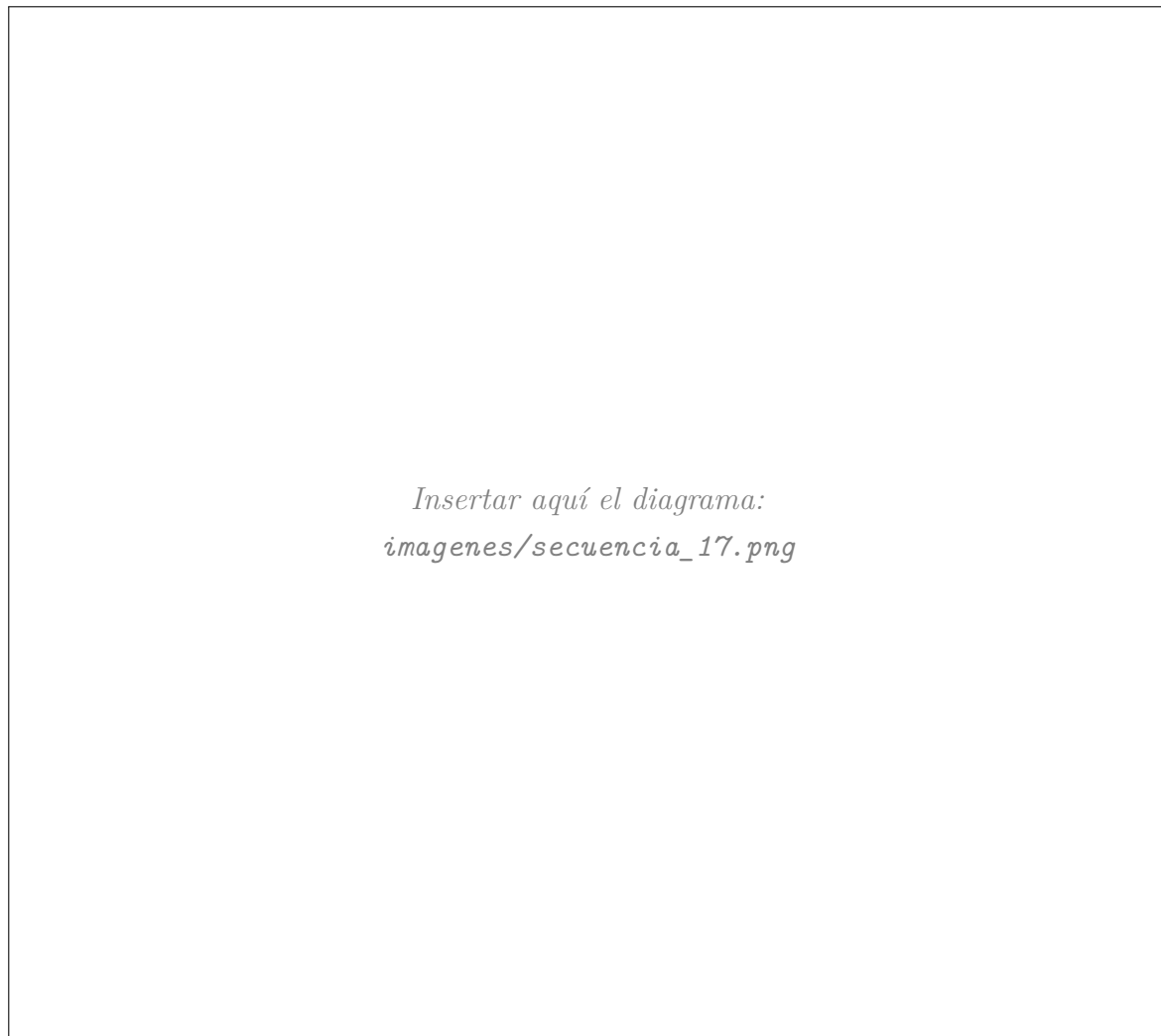


Figura 4.20: Diagrama de Secuencia 17 – [Nombre del proceso transaccional 17]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.18. Diagrama de Secuencia 18 – [Nombre del proceso transaccional 18]

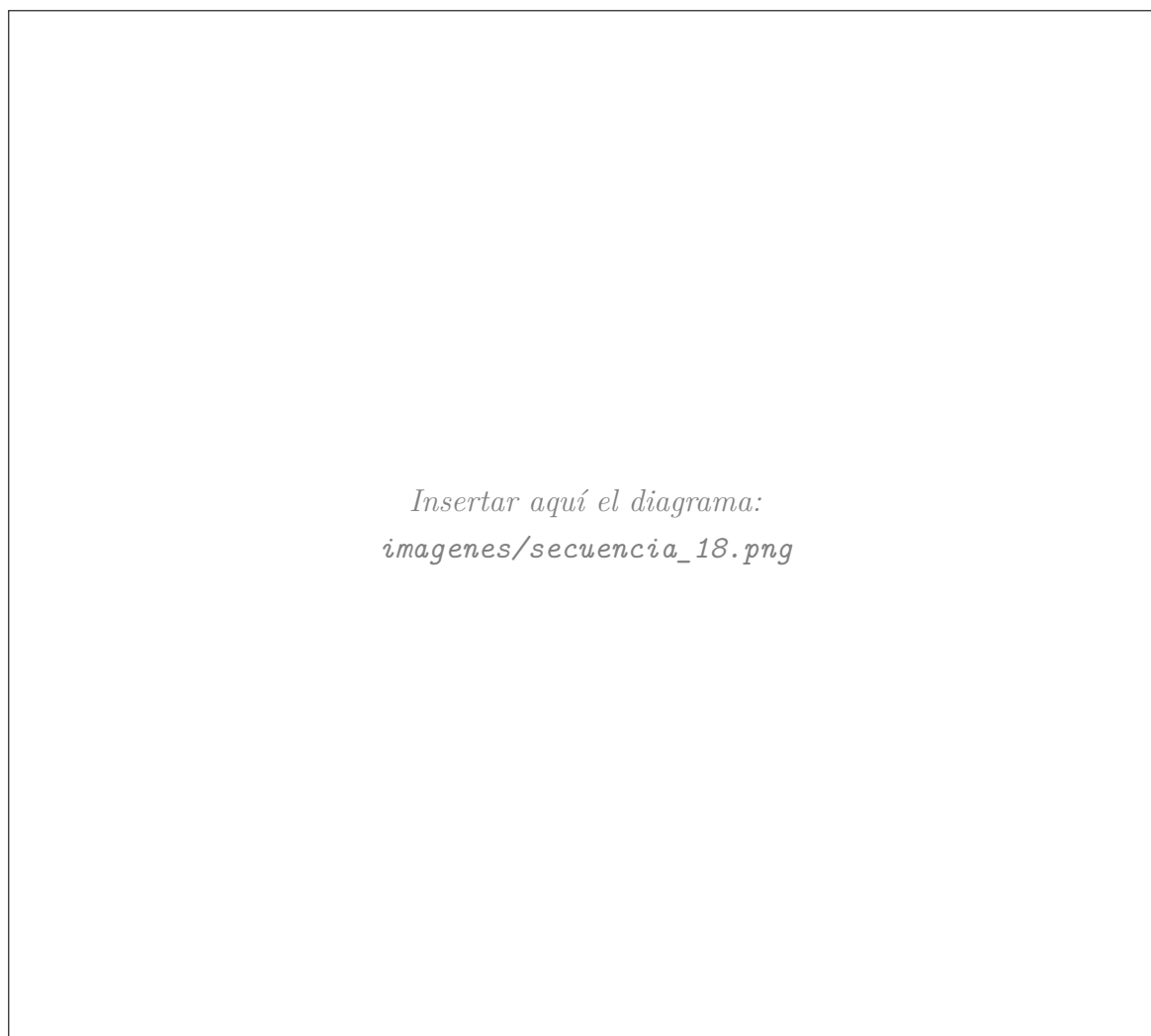


Figura 4.21: Diagrama de Secuencia 18 – [Nombre del proceso transaccional 18]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.19. Diagrama de Secuencia 19 – [Nombre del proceso transaccional 19]

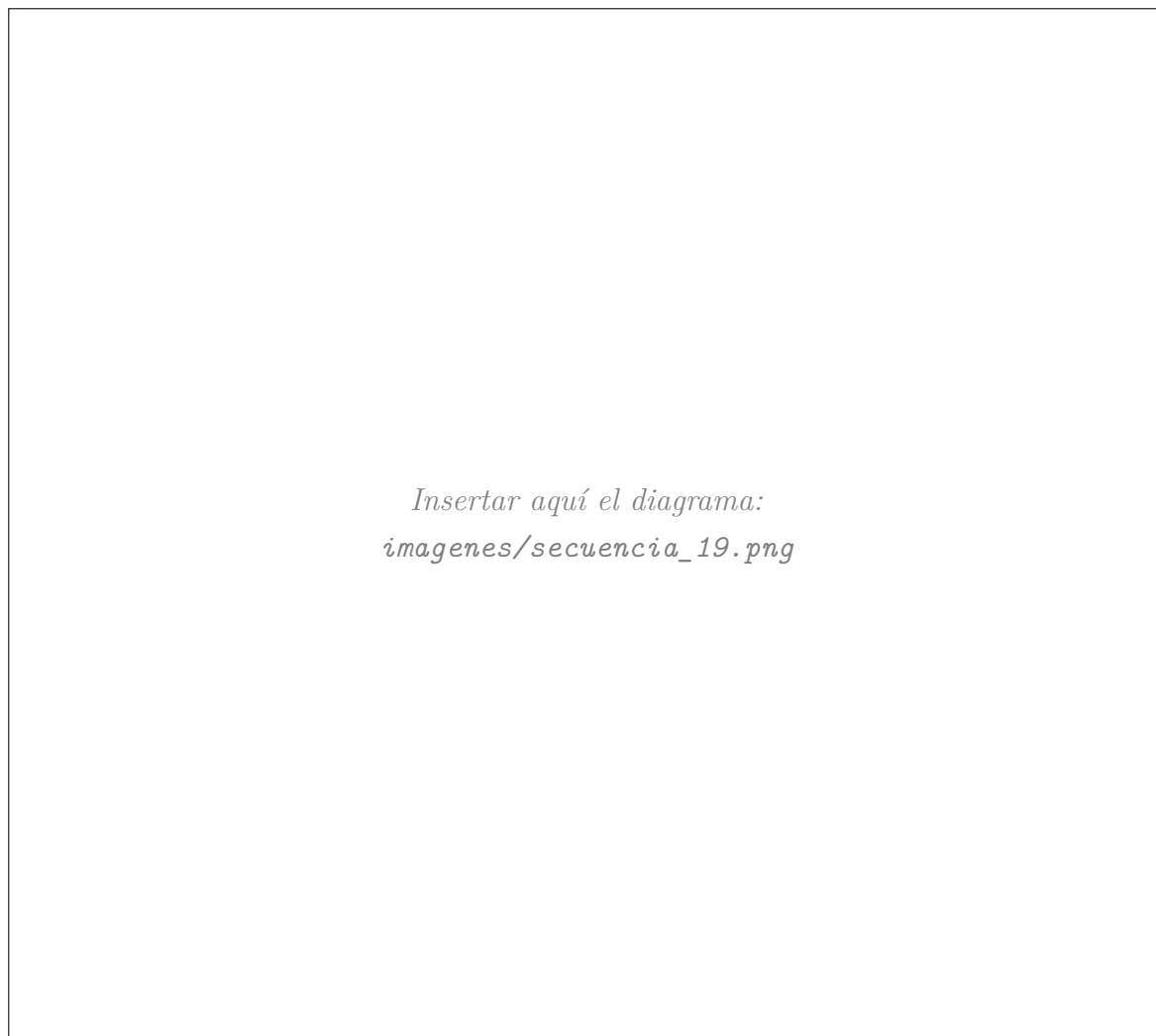


Figura 4.22: Diagrama de Secuencia 19 – [Nombre del proceso transaccional 19]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.20. Diagrama de Secuencia 20 – [Nombre del proceso transaccional 20]

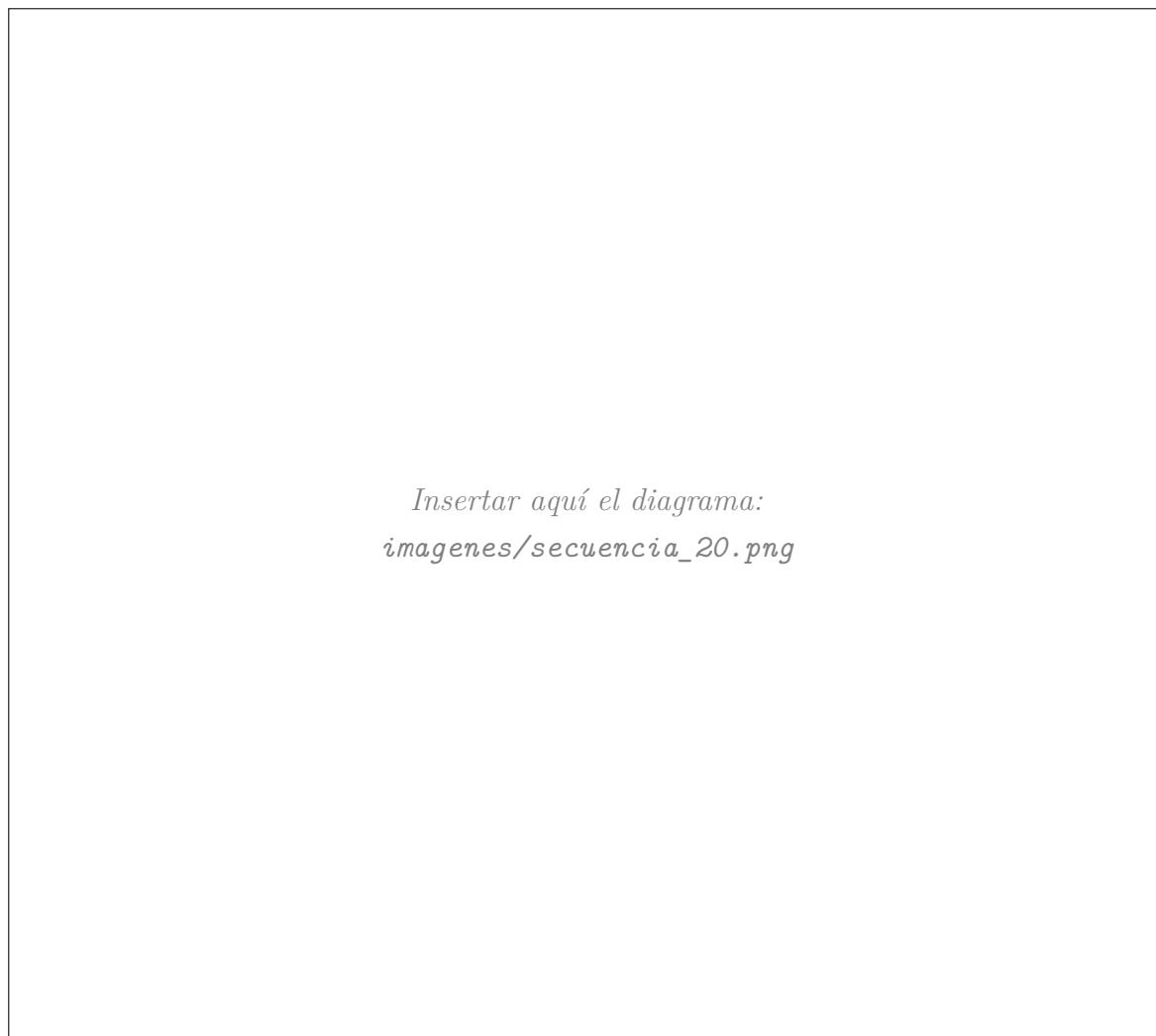


Figura 4.23: Diagrama de Secuencia 20 – [Nombre del proceso transaccional 20]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.21. Diagrama de Secuencia 21 – [Nombre del proceso transaccional 21]

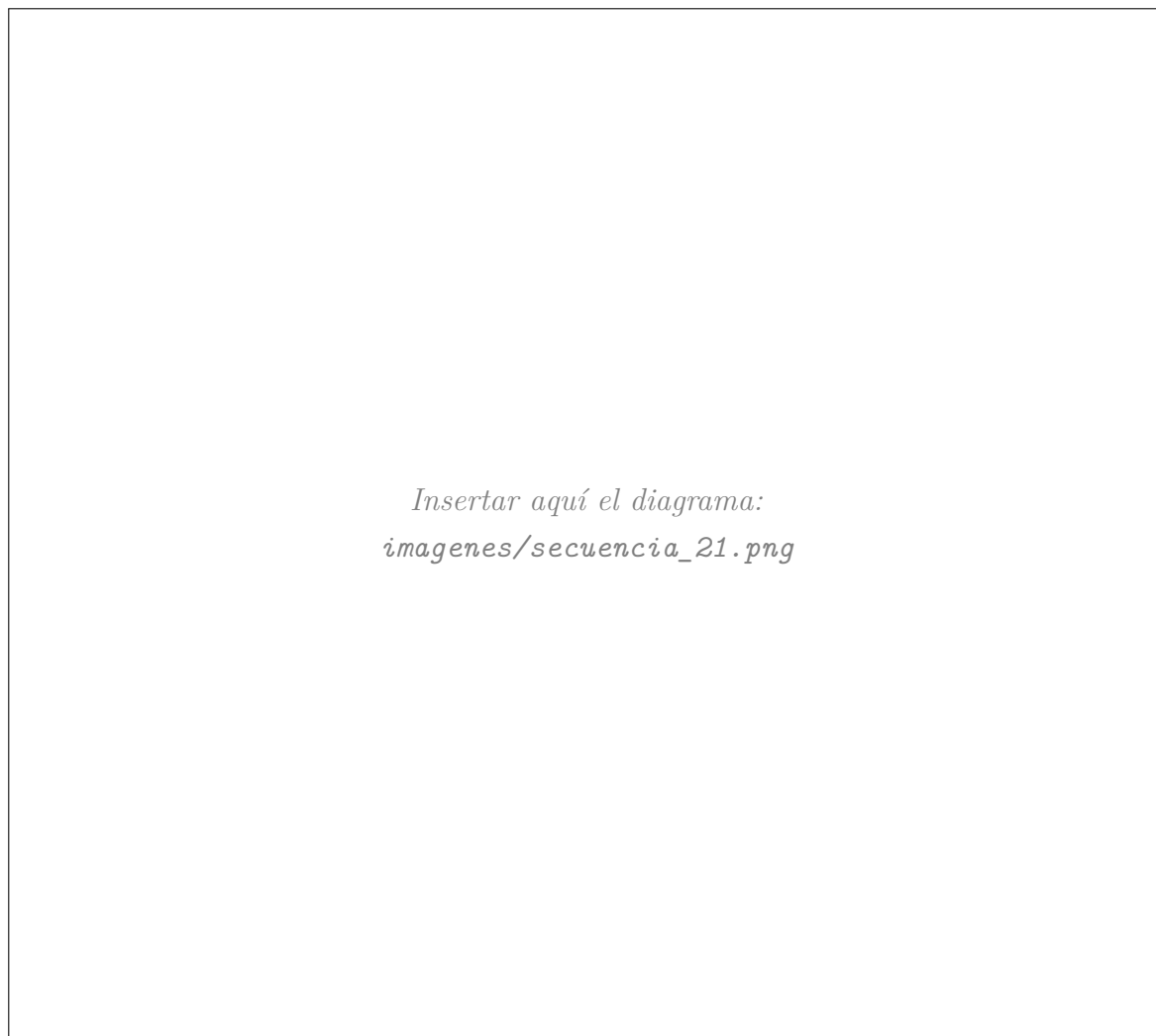


Figura 4.24: Diagrama de Secuencia 21 – [Nombre del proceso transaccional 21]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.22. Diagrama de Secuencia 22 – [Nombre del proceso transaccional 22]

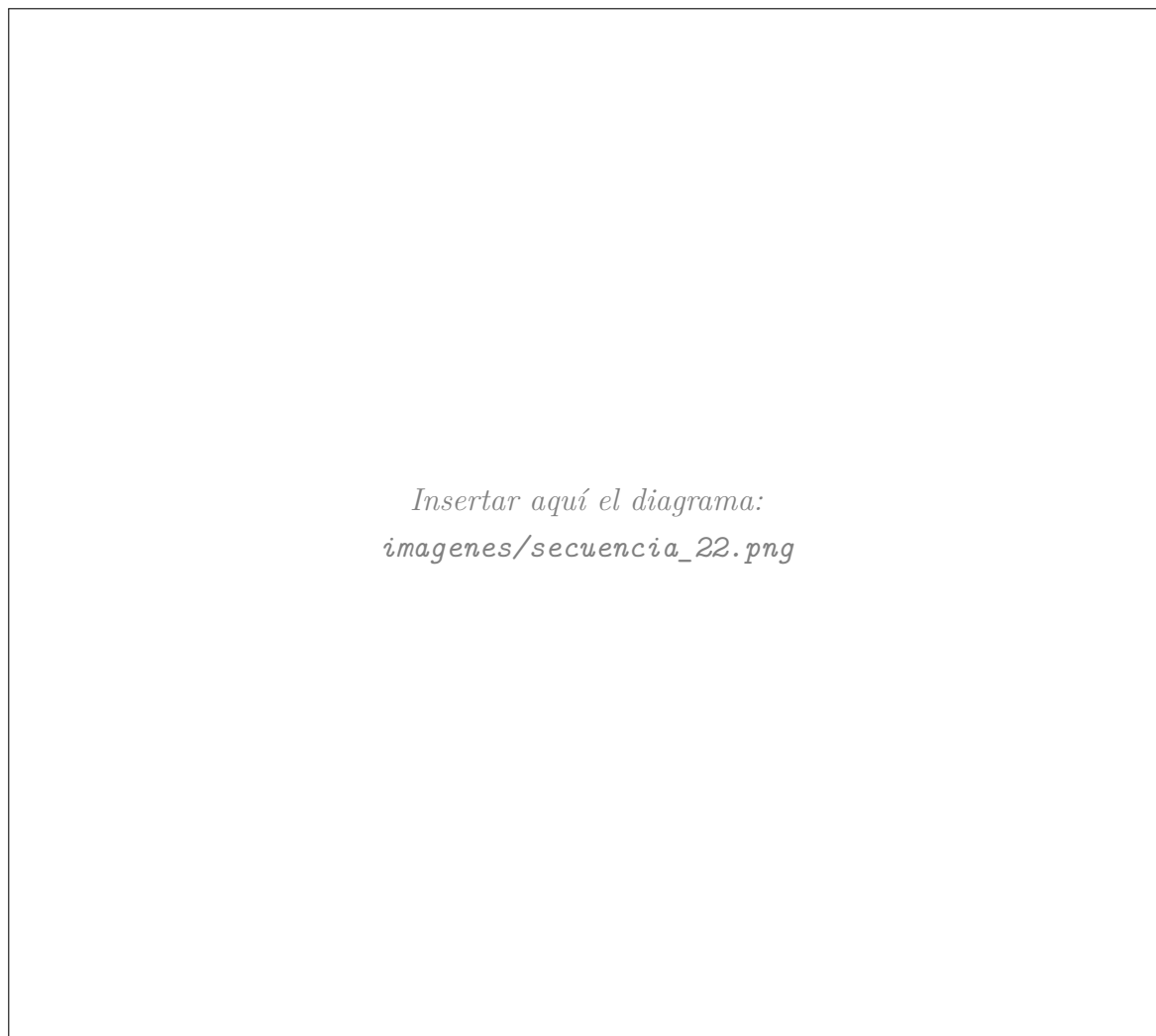


Figura 4.25: Diagrama de Secuencia 22 – [Nombre del proceso transaccional 22]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.23. Diagrama de Secuencia 23 – [Nombre del proceso transaccional 23]

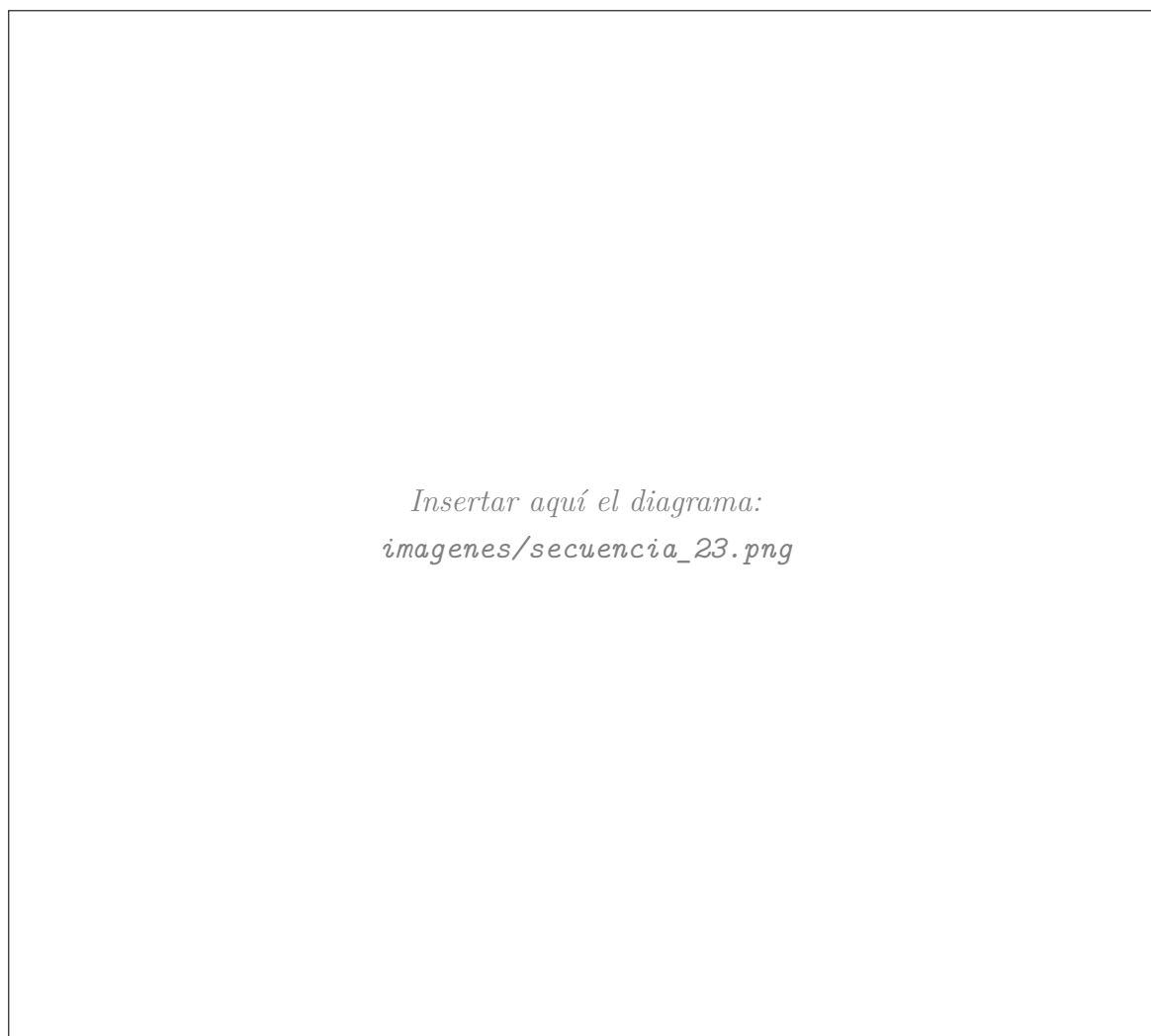


Figura 4.26: Diagrama de Secuencia 23 – [Nombre del proceso transaccional 23]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.24. Diagrama de Secuencia 24 – [Nombre del proceso transaccional 24]

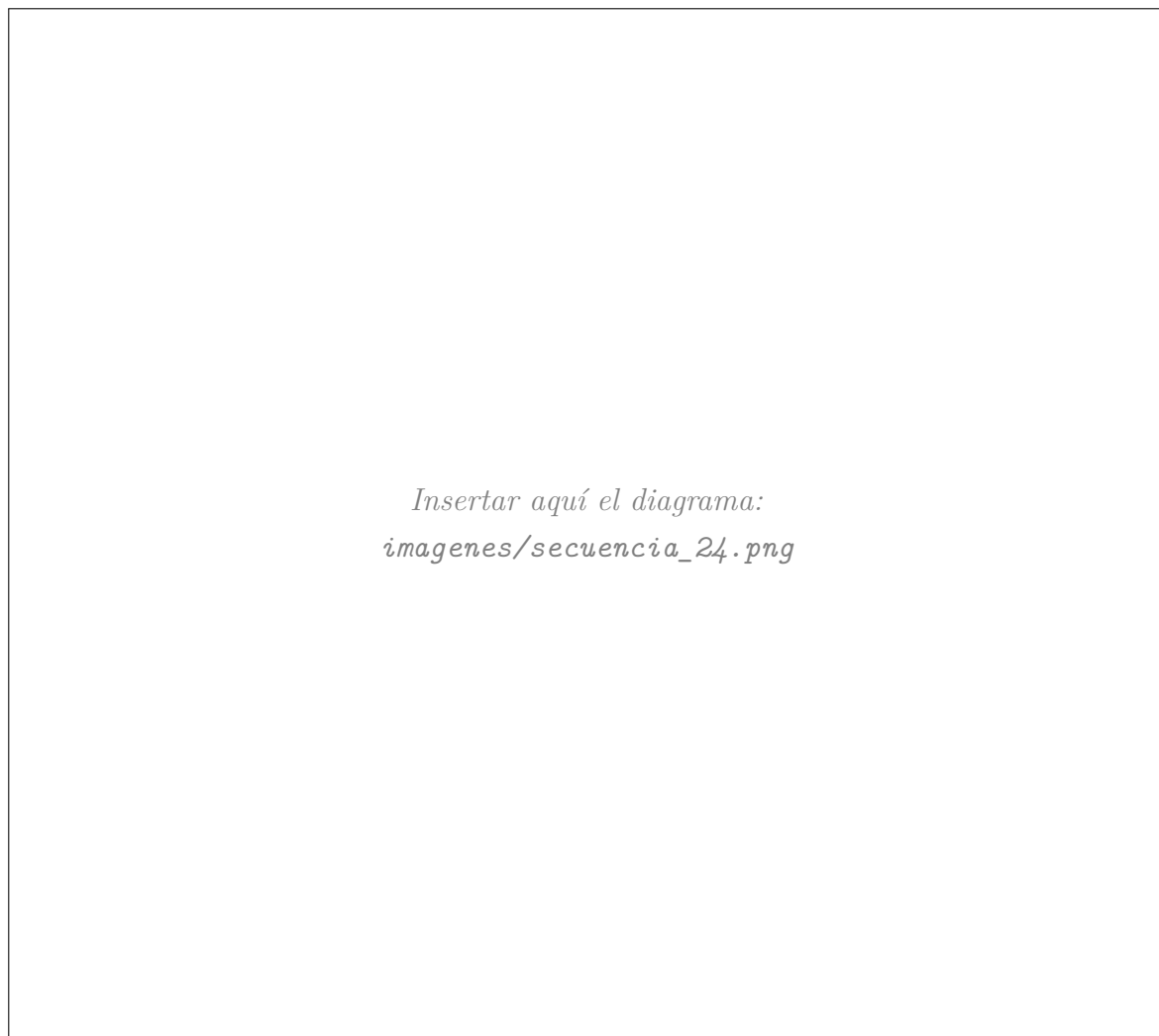


Figura 4.27: Diagrama de Secuencia 24 – [Nombre del proceso transaccional 24]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.25. Diagrama de Secuencia 25 – [Nombre del proceso transaccional 25]

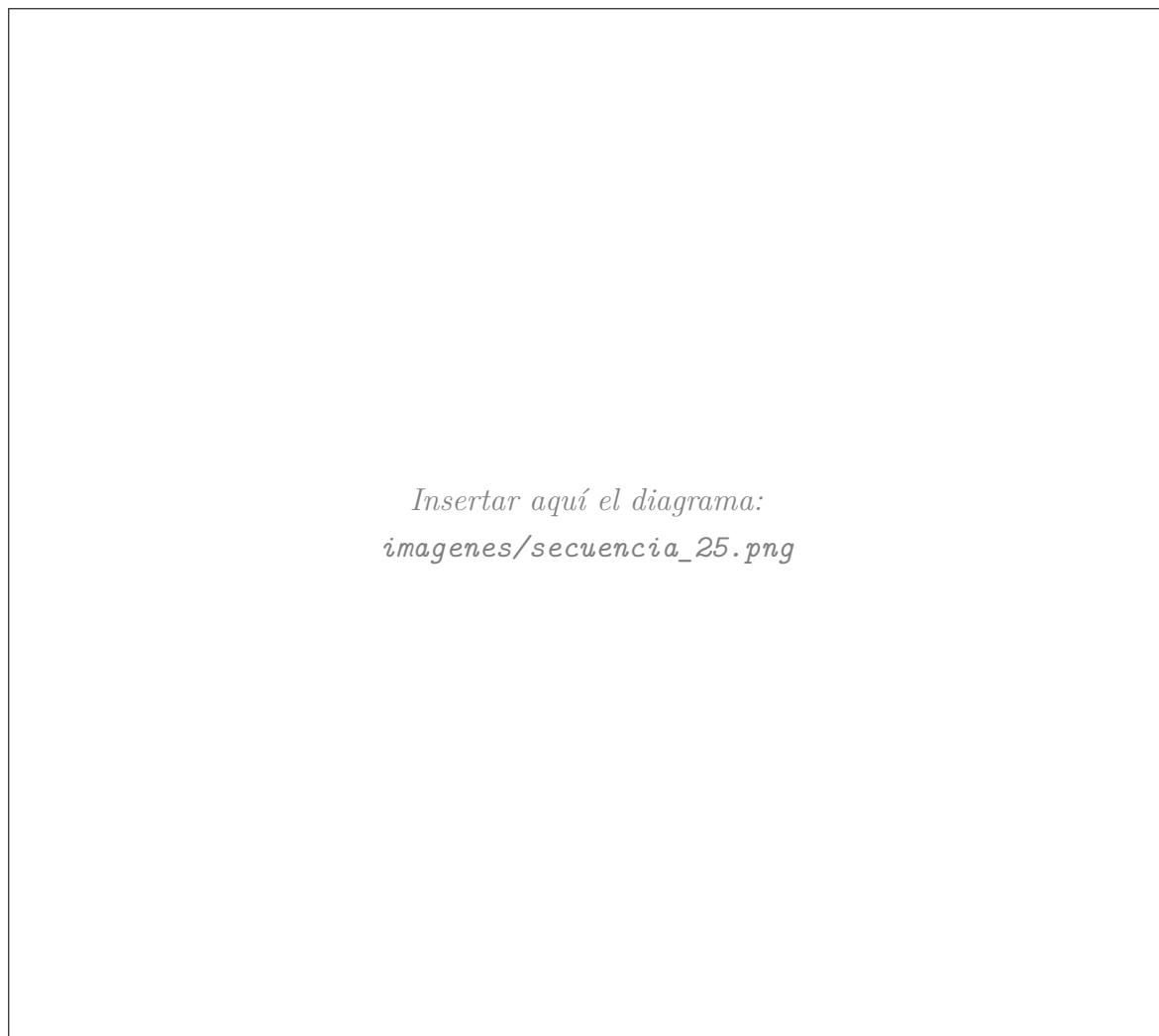


Figura 4.28: Diagrama de Secuencia 25 – [Nombre del proceso transaccional 25]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.26. Diagrama de Secuencia 26 – [Nombre del proceso transaccional 26]

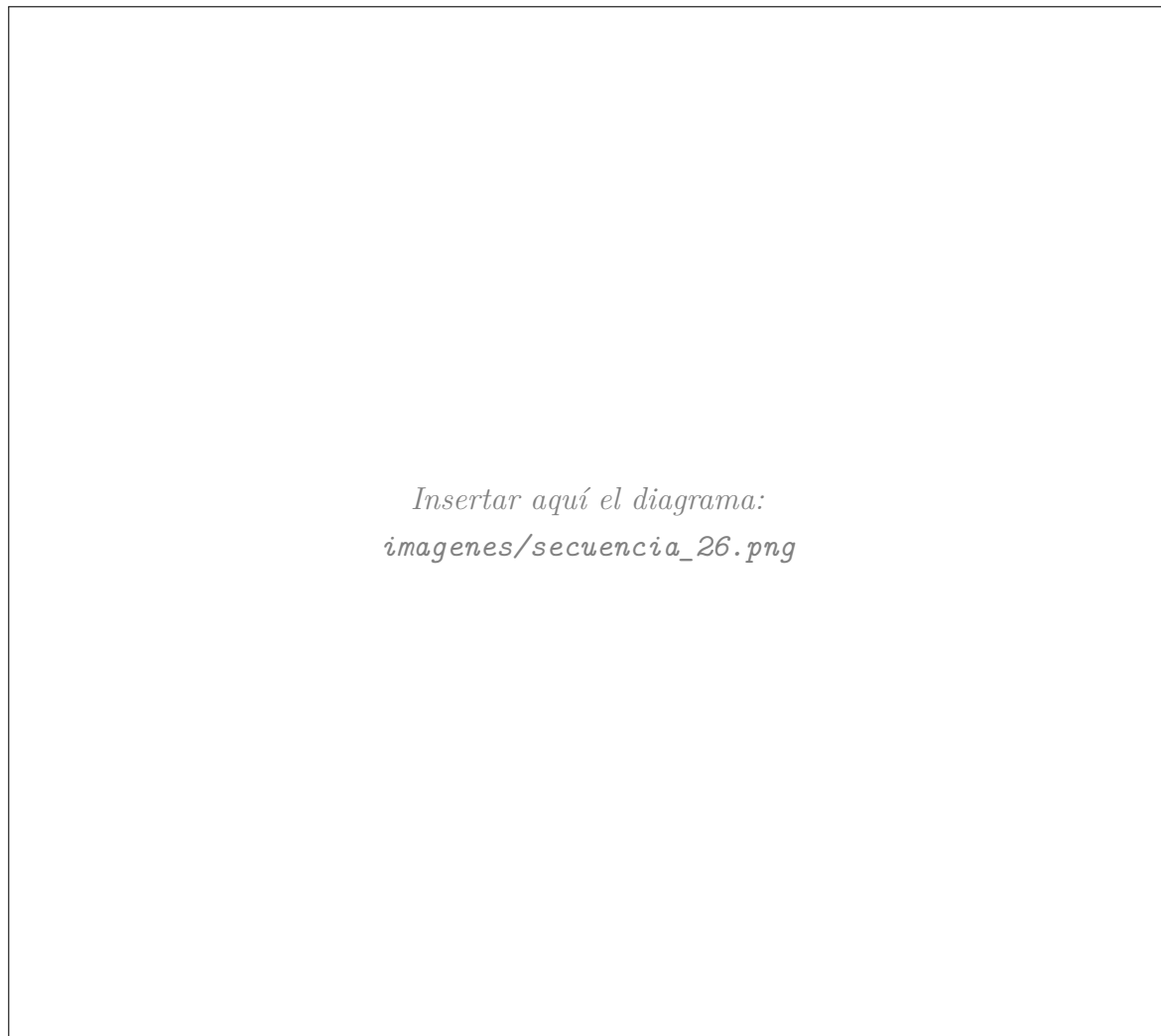


Figura 4.29: Diagrama de Secuencia 26 – [Nombre del proceso transaccional 26]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.27. Diagrama de Secuencia 27 – [Nombre del proceso transaccional 27]

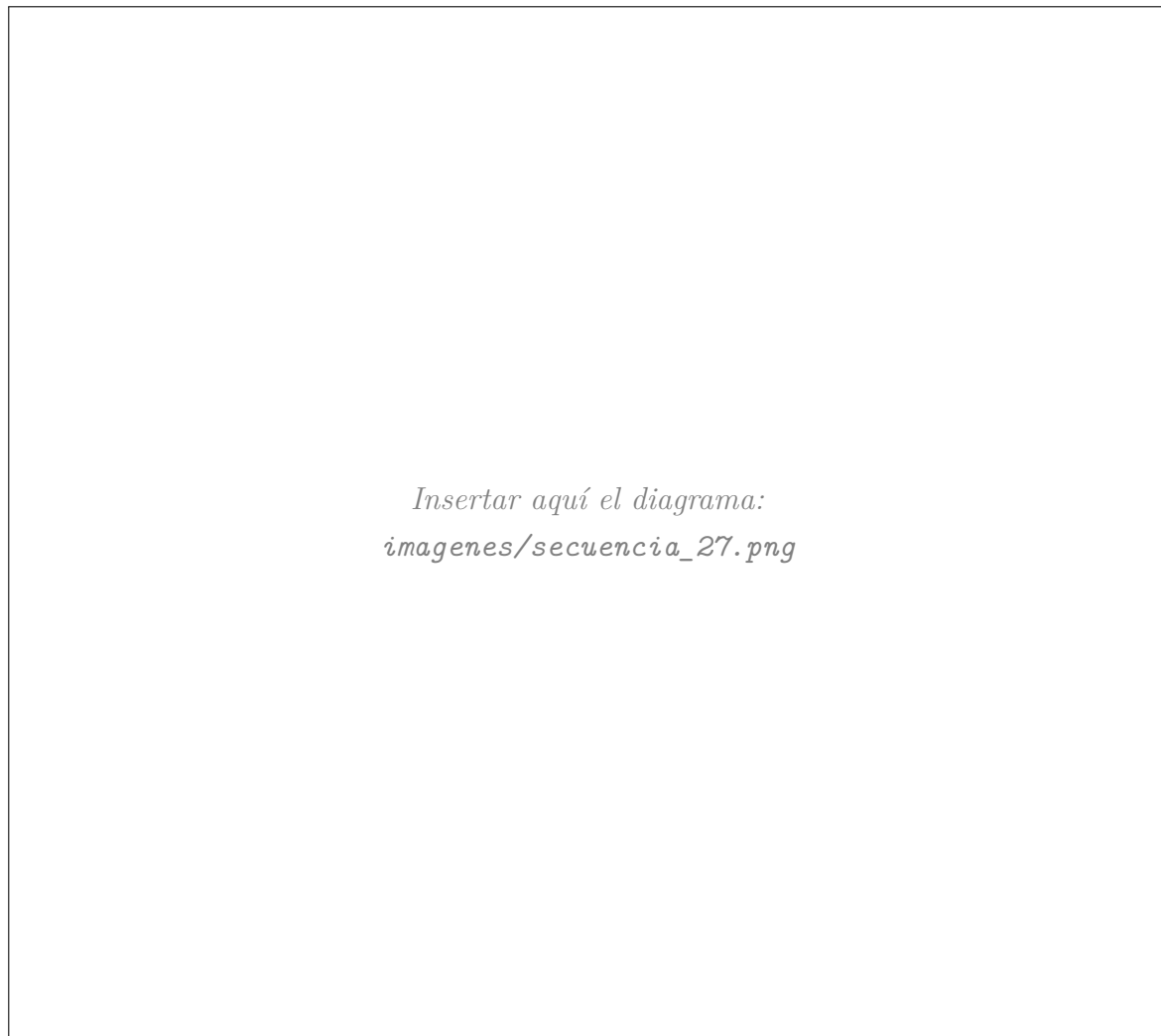


Figura 4.30: Diagrama de Secuencia 27 – [Nombre del proceso transaccional 27]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.28. Diagrama de Secuencia 28 – [Nombre del proceso transaccional 28]

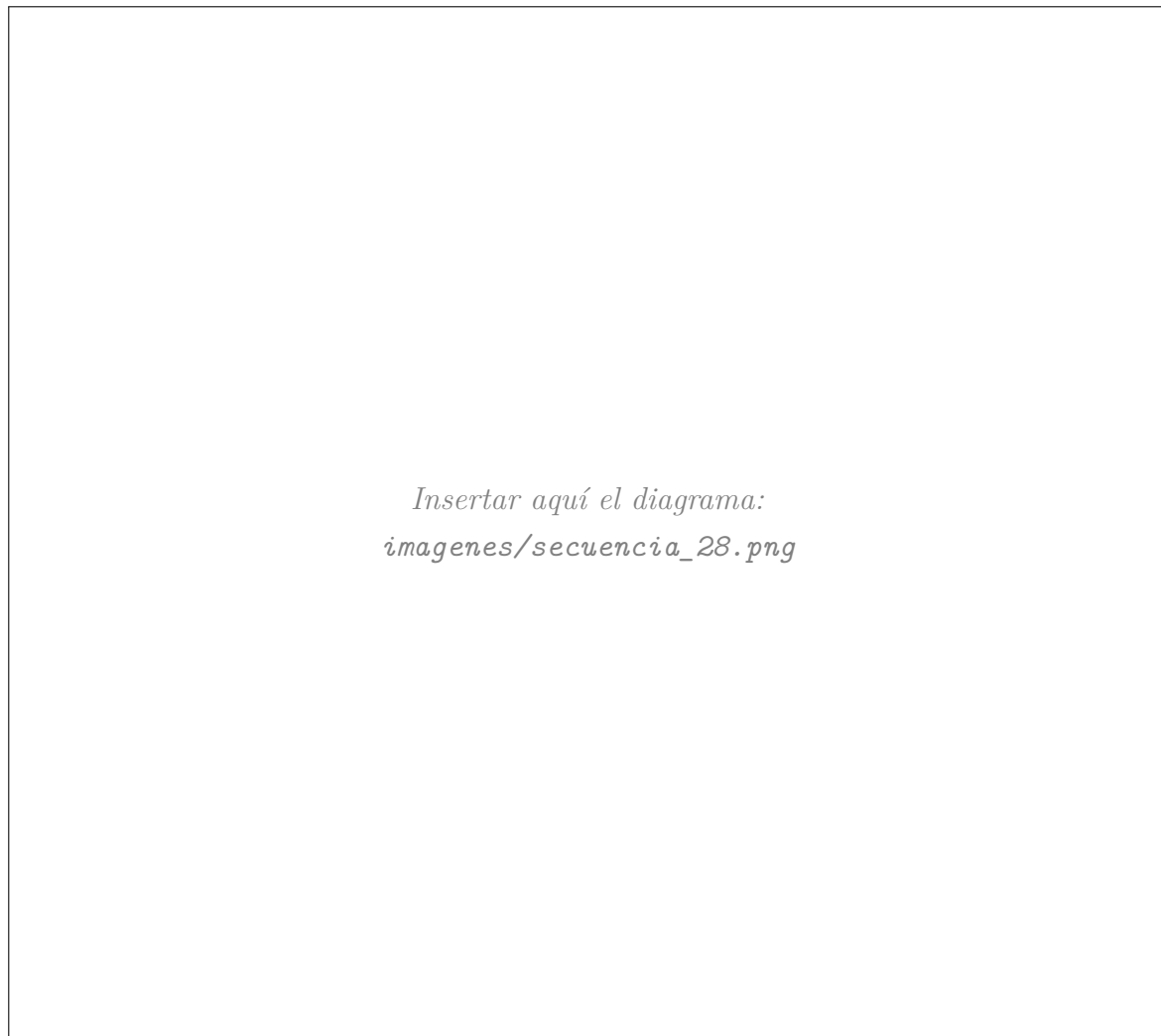


Figura 4.31: Diagrama de Secuencia 28 – [Nombre del proceso transaccional 28]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.29. Diagrama de Secuencia 29 – [Nombre del proceso transaccional 29]

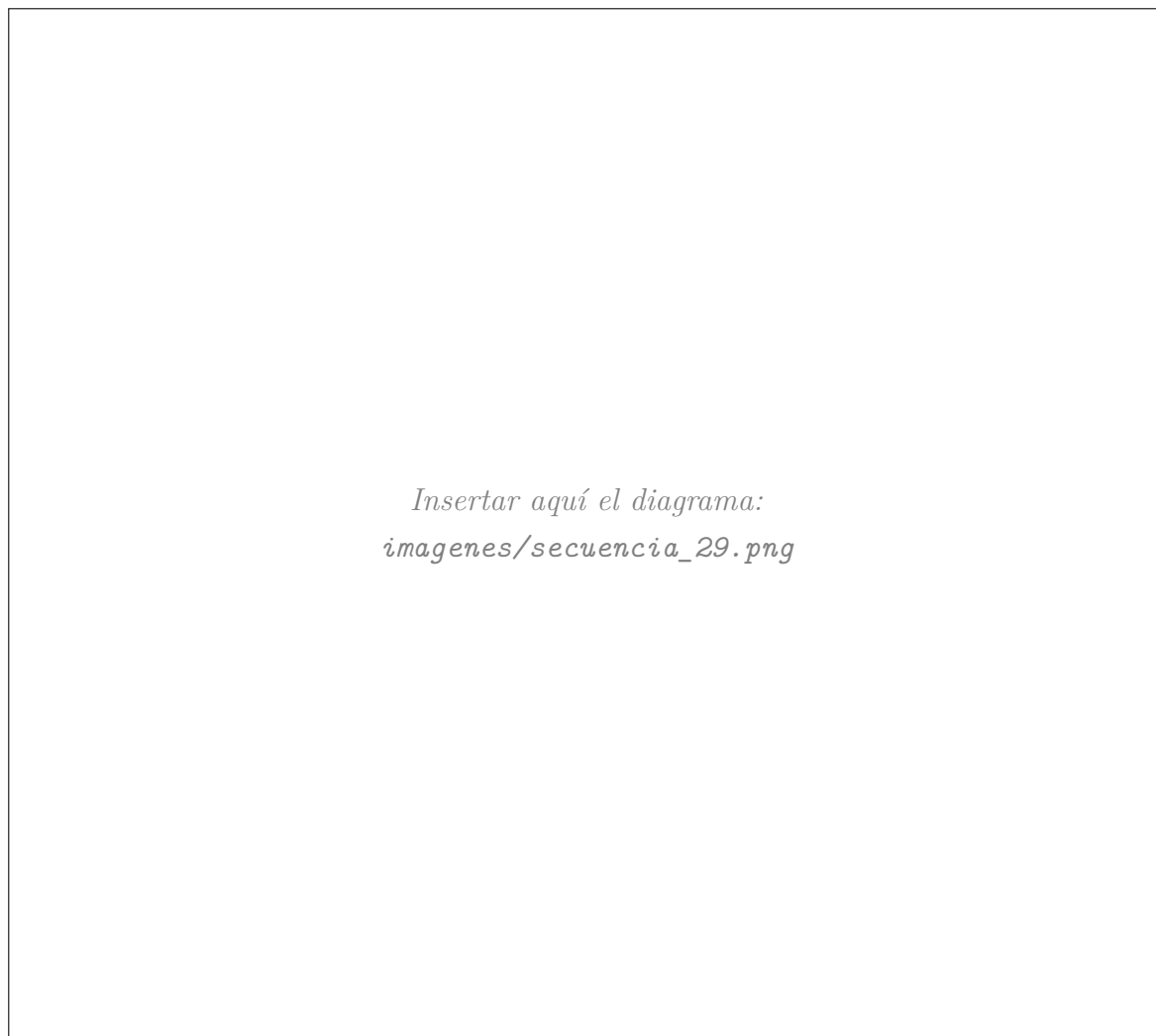


Figura 4.32: Diagrama de Secuencia 29 – [Nombre del proceso transaccional 29]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.30. Diagrama de Secuencia 30 – [Nombre del proceso transaccional 30]

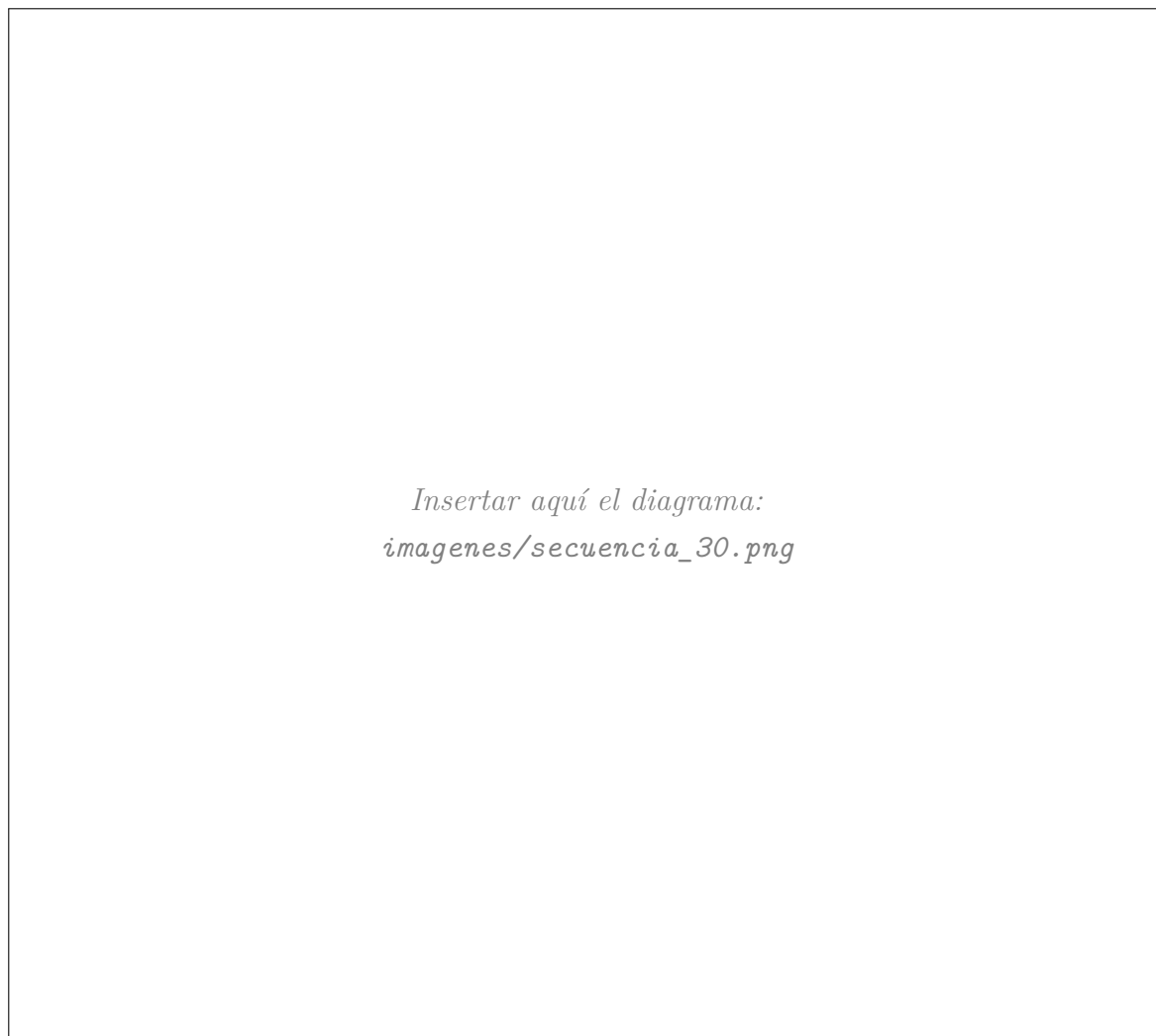


Figura 4.33: Diagrama de Secuencia 30 – [Nombre del proceso transaccional 30]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.31. Diagrama de Secuencia 31 – [Nombre del proceso transaccional 31]

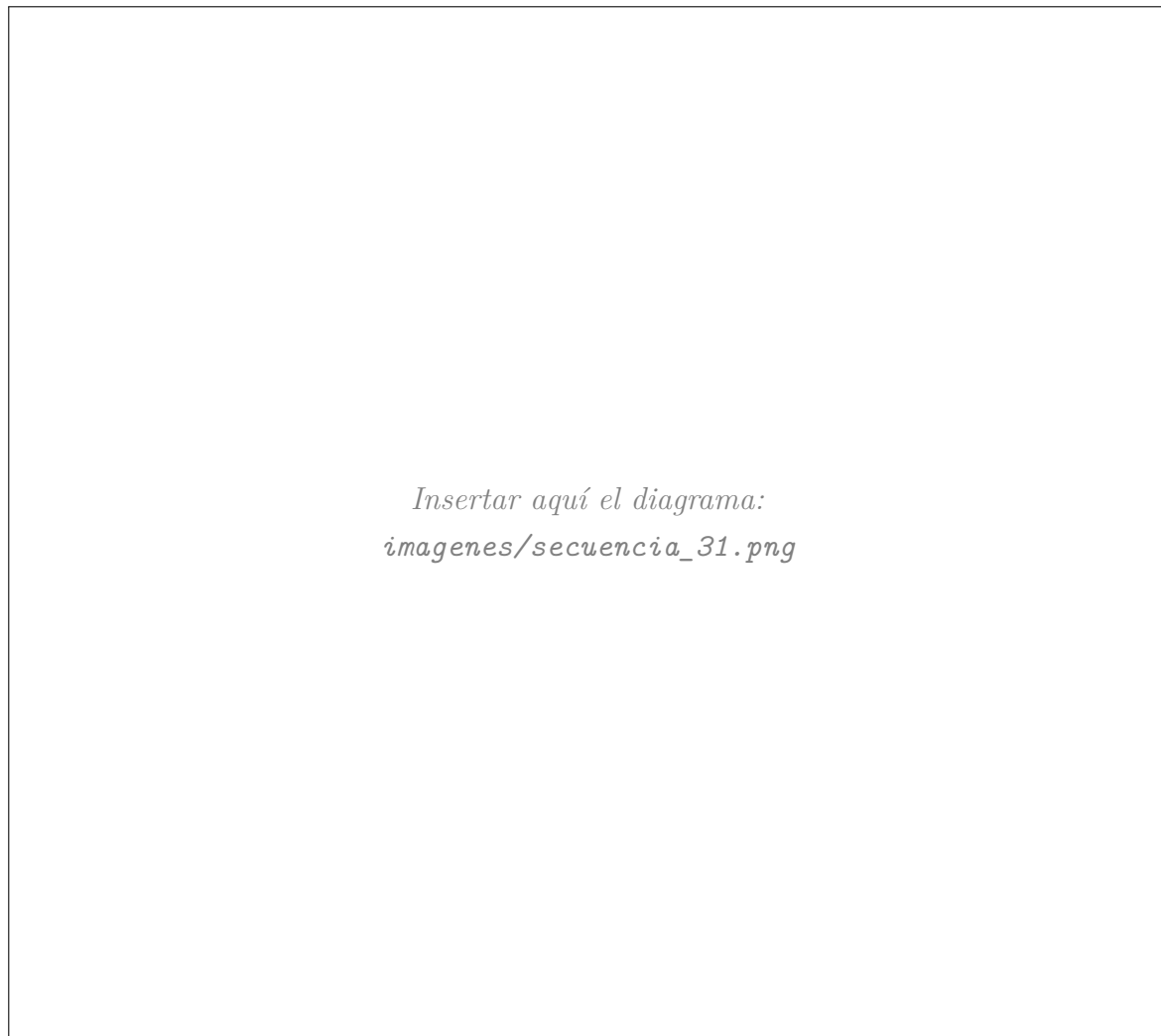


Figura 4.34: Diagrama de Secuencia 31 – [Nombre del proceso transaccional 31]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.2.32. Diagrama de Secuencia 32 – [Nombre del proceso transaccional 32]

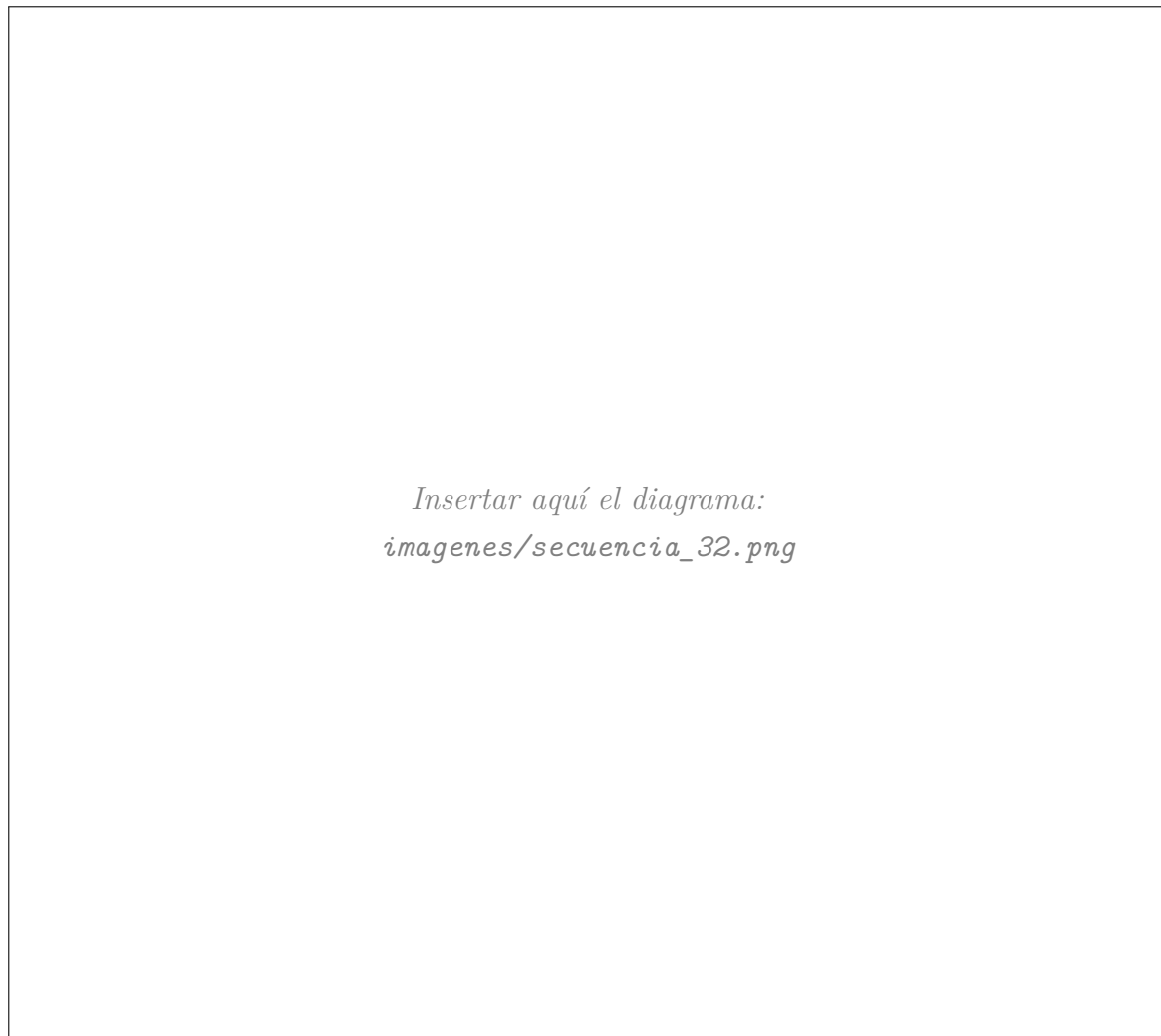


Figura 4.35: Diagrama de Secuencia 32 – [Nombre del proceso transaccional 32]

Descripción: [Descripción del escenario: actores/objetos participantes, mensajes intercambiados en orden temporal, y condiciones o bucles relevantes (alt/opt/loop) del algoritmo transaccional.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.3. Diagramas de Colaboración / Comunicación

Con fines ilustrativos, se presentan versiones en diagrama de comunicación de algunos de los escenarios de secuencia más representativos.

4.3.1. Diagrama de Comunicación 01 – [Escenario ilustrado, ej. correspondiente a Secuencia #1]



Figura 4.36: Diagrama de Comunicación 01 – [Escenario ilustrado, ej. correspondiente a Secuencia #1]

Descripción: [Descripción del escenario y correspondencia con el diagrama de secuencia equivalente.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.3.2. Diagrama de Comunicación 02 – [Escenario ilustrado]

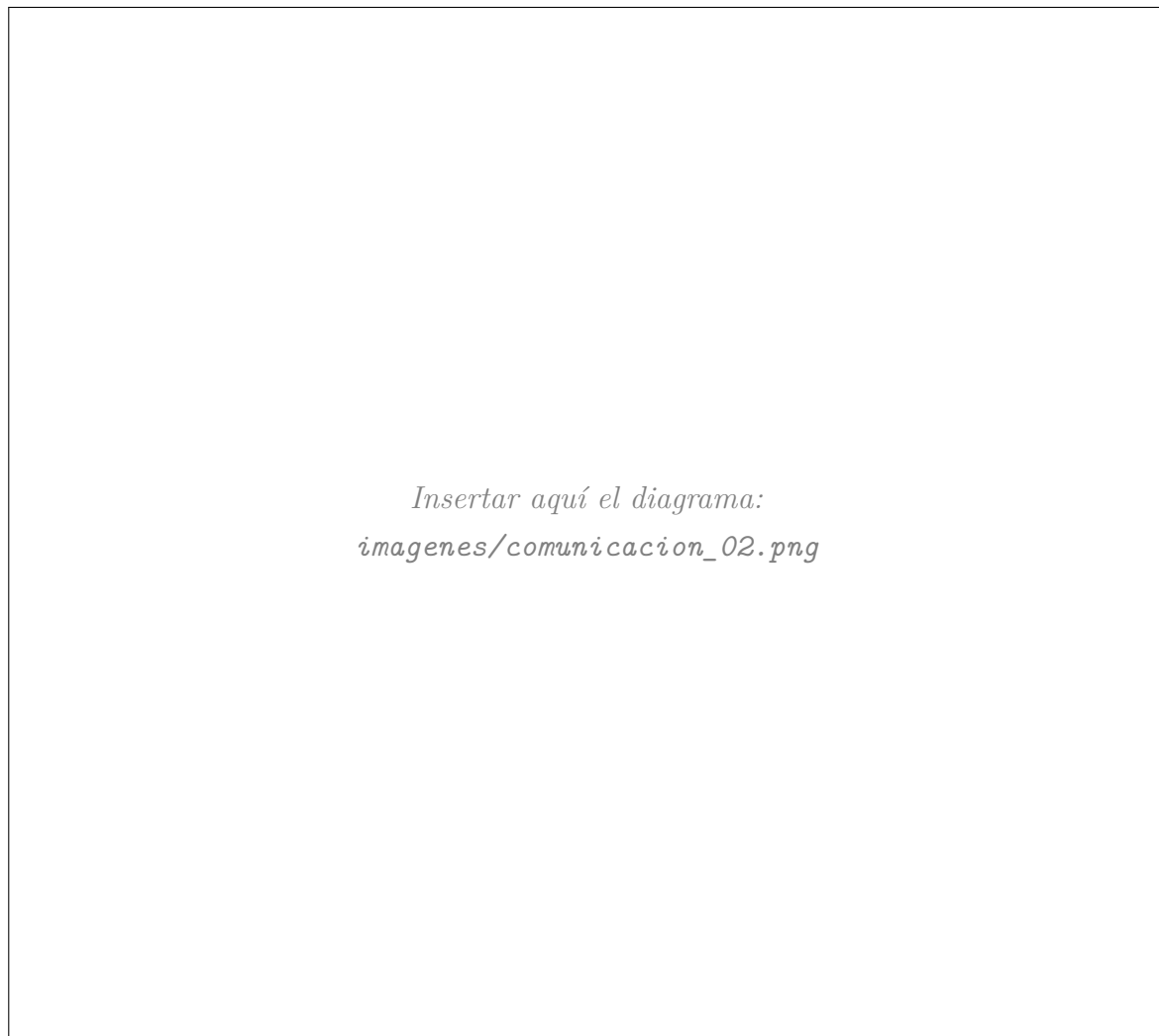


Figura 4.37: Diagrama de Comunicación 02 – [Escenario ilustrado]

Descripción: [Descripción del escenario.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

4.3.3. Diagrama de Comunicación 03 – [Escenario ilustrado]

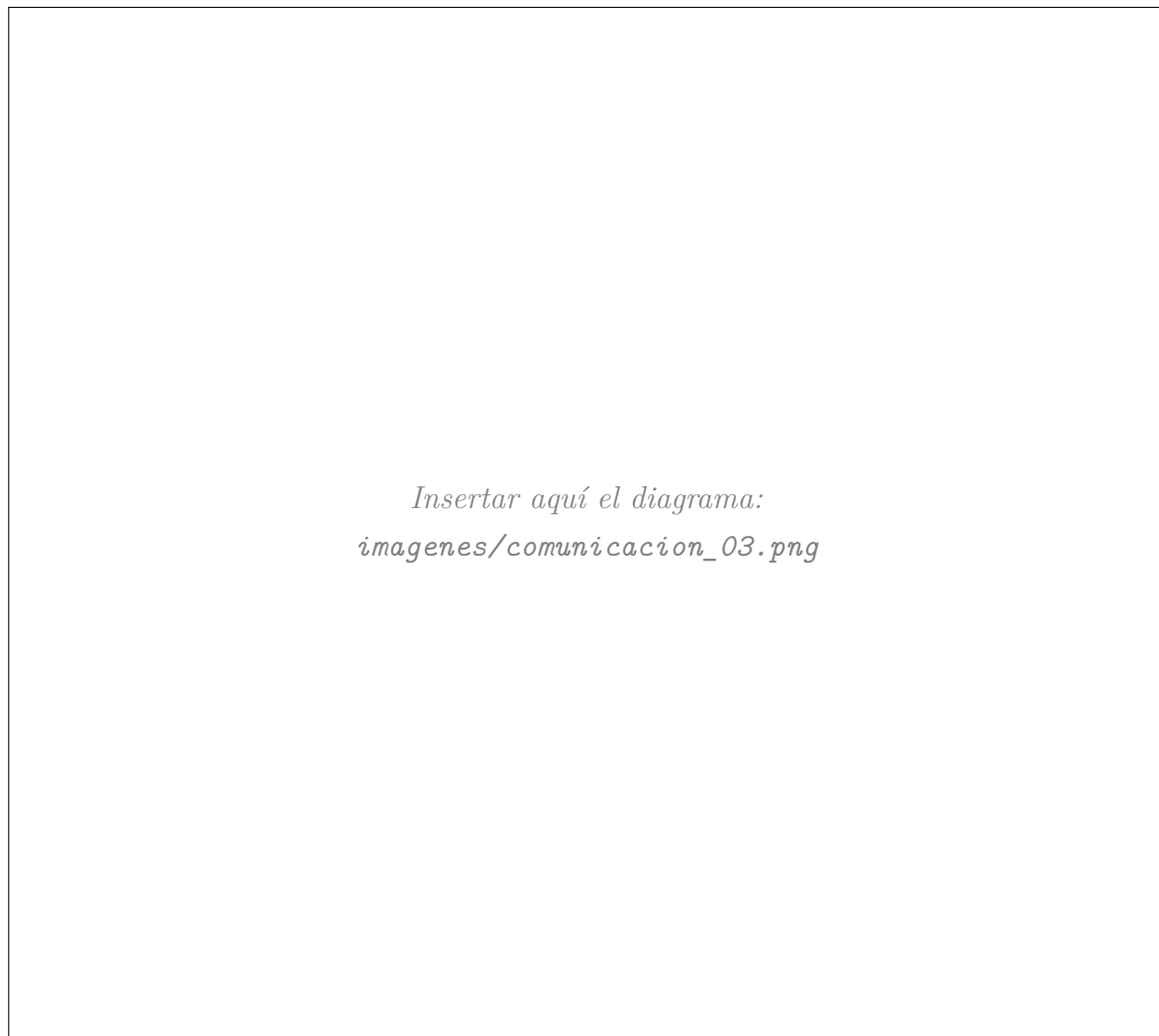


Figura 4.38: Diagrama de Comunicación 03 – [Escenario ilustrado]

Descripción: [Descripción del escenario.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

Nota: Agregue más bloques si desea ilustrar escenarios adicionales.

4.4. Diagramas de Estado

Se documenta el ciclo de vida de los objetos/clases más relevantes del sistema cuyo comportamiento depende significativamente de su estado interno.

4.4.1. Diagrama de Estado – Clase Reservations

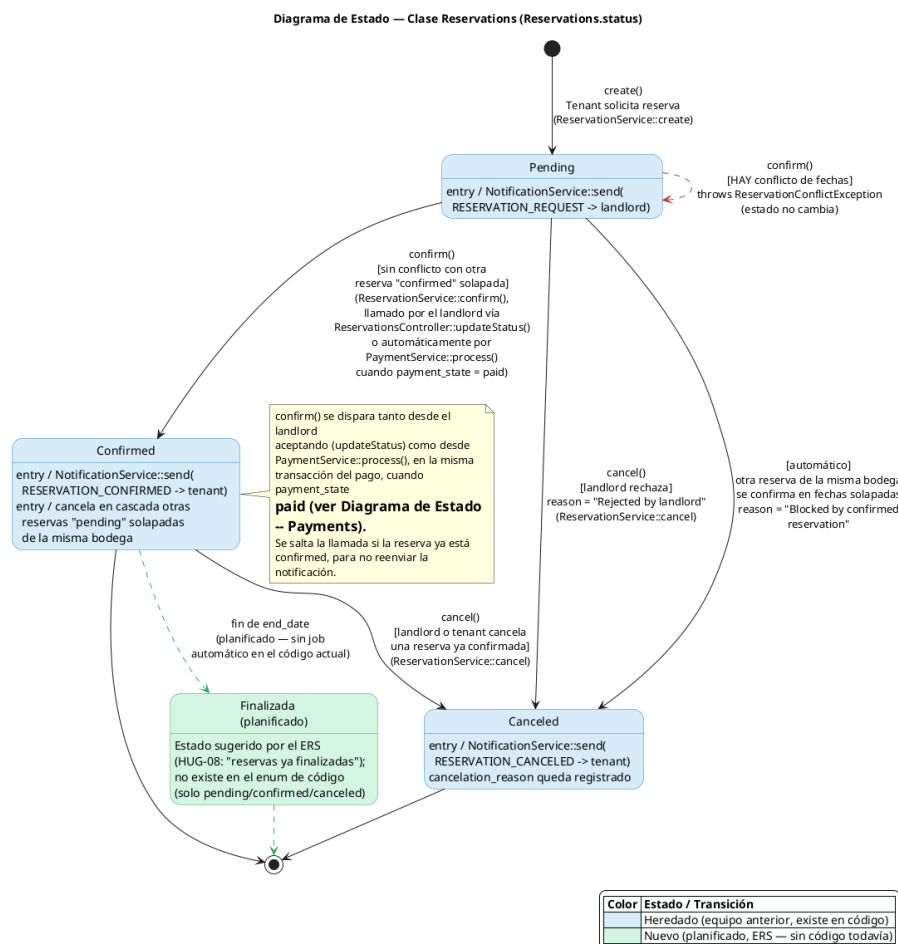


Figura 4.39: Diagrama de Estado – Clase Reservations

Descripción: Ciclo de vida de `Reservations.status`, la entidad transaccional central del *marketplace* de almacenamiento. Extraído directamente de `ReservationService` (`backend/app/Services/ReservationService.php`) y de la restricción `enum` de tres valores (`pending`, `confirmed`, `canceled`) declarada en la migración `database/migrations/2025_10_15_012859_create_reservations_table.php`: toda reserva nace en *Pending* al ejecutar `create()` (el tenant solicita); pasa a *Confirmed* vía `confirm()` solo si no hay otra reserva *confirmed* solapada en fechas para la misma bodega (guarda `ReservationConflictException` en caso contrario, sin cambiar de estado); y pasa a *Canceled* vía `cancel()`, ya sea por rechazo directo del arrendador o **automáticamente en cascada** cuando otra reserva

de la misma bodega se confirma en fechas solapadas (`reason = Blocked by confirmed reservation`). La transición a *Confirmed* tiene dos disparadores: el arrendador aceptando vía `ReservationsController::updateStatus()`, o `PaymentService::process()` (`backend/app/Services/PaymentService.php`) invocando el mismo `ReservationService::confirm()` dentro de su propia transacción cuando un pago se registra con `payment_state = paid` (ver Diagrama de Estado – Payments, Figura 4.42); esta segunda vía omite la llamada si la reserva ya está *confirmed*, para no reenviar la notificación. Cada transición dispara una notificación real (`RESERVATION_REQUEST`, `RESERVATION_CONFIRMED` o `RESERVATION_CANCELED` vía `NotificationService::send()`). Se documenta además, marcado como **nuevo (planificado)**, un estado *Finalizada* sugerido por el ERS (HUG-08 menciona reservas “ya finalizadas” para efectos de auditoría al eliminar una bodega) que **no existe todavía** en el código: el `enum` de la base de datos solo contempla los tres estados heredados, sin ningún job automático que cierre una reserva al vencer su `end_date`.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (renderizado localmente con `plantuml.jar` sobre OpenJDK, motor de layout `Graphviz/dot`). Fuente: [recursos/diagramas_uml/estado_01.puml](#).

Tabla de trazabilidad

Elemento	Fuente (archivo / método)	Estado
Pending (estado inicial)	<code>ReservationService::create()</code>	Heredado
Pending → Confirmed (arrendador acepta)	<code>ReservationService::confirm()</code> vía <code>ReservationsController</code>	Heredado
Pending → Confirmed (pago exitoso)	<code>ReservationService::confirm()</code> vía <code>PaymentService::process()</code>	Heredado
Pending → Canceled (rechazo)	<code>ReservationService::cancel()</code>	Heredado
Pending → Canceled (cascada automática)	<code>ReservationService::confirm()</code> , bloque de cancelación en cascada	Heredado
Confirmed → Canceled	<code>ReservationService::cancel()</code>	Heredado
Guarda de conflicto (auto-transición)	<code>ReservationConflictException</code> (<code>backend/app/Exceptions/ReservationConflictException.php</code>)	Heredado
Finalizada (estado planificado)	ERS HUG-08 (reservas <i>ya finalizadas</i>), sin código todavía	Nuevo (planificado)

4.4.2. Diagrama de Estado – Clase StoreRooms

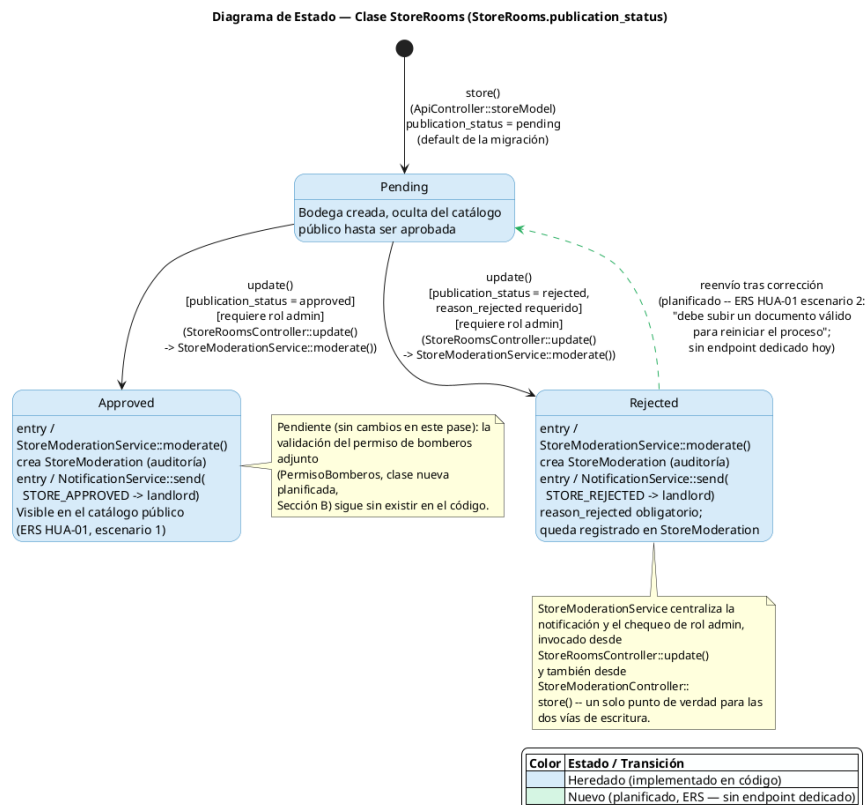


Figura 4.40: Diagrama de Estado – Clase StoreRooms

Descripción: Ciclo de vida de `StoreRooms.publication_status` (`pending`, `approved`, `rejected`), el campo que determina si una bodega es visible en el catálogo público (ERS HUA-01, “Fire department permit validation”). El cambio de `publication_status` a `approved/rejected` se enruta por `StoreModerationService::moderate()` ([backend/app/Services/StoreModerationService.php](#)), que en una sola transacción actualiza `StoreRooms`, crea el registro de auditoría en `store_moderation` y notifica al arrendador (`NotificationType::STORE_APPROVED/STORE_REJECTED`). El servicio se invoca desde el mismo endpoint que ya usan el frontend y los tests (`PUT /storeRooms/{id}`): `StoreRoomsController::update()` detecta cuando el payload trae ese cambio específico y lo enruta por el servicio, con un chequeo de rol `admin` exigido específicamente en esa rama (cualquier otro autenticado, incluido el propio arrendador dueño de la bodega, recibe 403 si intenta cambiar `publication_status`). `StoreModerationController::store()` (`POST /store_moderation`) también se enruta por el mismo servicio, de modo que ambas vías de escritura queden sincronizadas.

Pendiente: la transición `Rejected` → `Pending` (reenvío tras corrección) sigue sin un endpoint dedicado, y la validación del permiso de bomberos adjunto (`PermisoBomberos`, clase nueva planificada, Sección B) sigue sin existir en el código.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (motor Graphviz/dot).

Fuente: [recursos/diagramas_uml/estado_02.puml](#).

Tabla de trazabilidad

Elemento	Fuente (archivo / método)	Estado
Pending (estado inicial)	StoreRoomsController::store() (default de la migración)	Heredado
Pending → Approved / Rejected	StoreRoomsController::update() → StoreModerationService::moderate()	Heredado
Registro de auditoría + notificación	StoreModerationService.php (StoreModeration::create(), NotificationService::send())	Heredado
Chequeo de rol admin en la transición	StoreRoomsController::update()	Heredado
Sincronización con StoreModeration	StoreModerationController::store() → mismo servicio	Heredado
Rejected → Pending (reenvío)	ERS HUA-01, escenario 2; sin endpoint dedicado hoy	Nuevo (planificado)
Validación de PermisoBomberos	ERS HUG-04 / HUA-01; sin código todavía	Nuevo (planificado)

4.4.3. Diagrama de Estado – Clase Reports

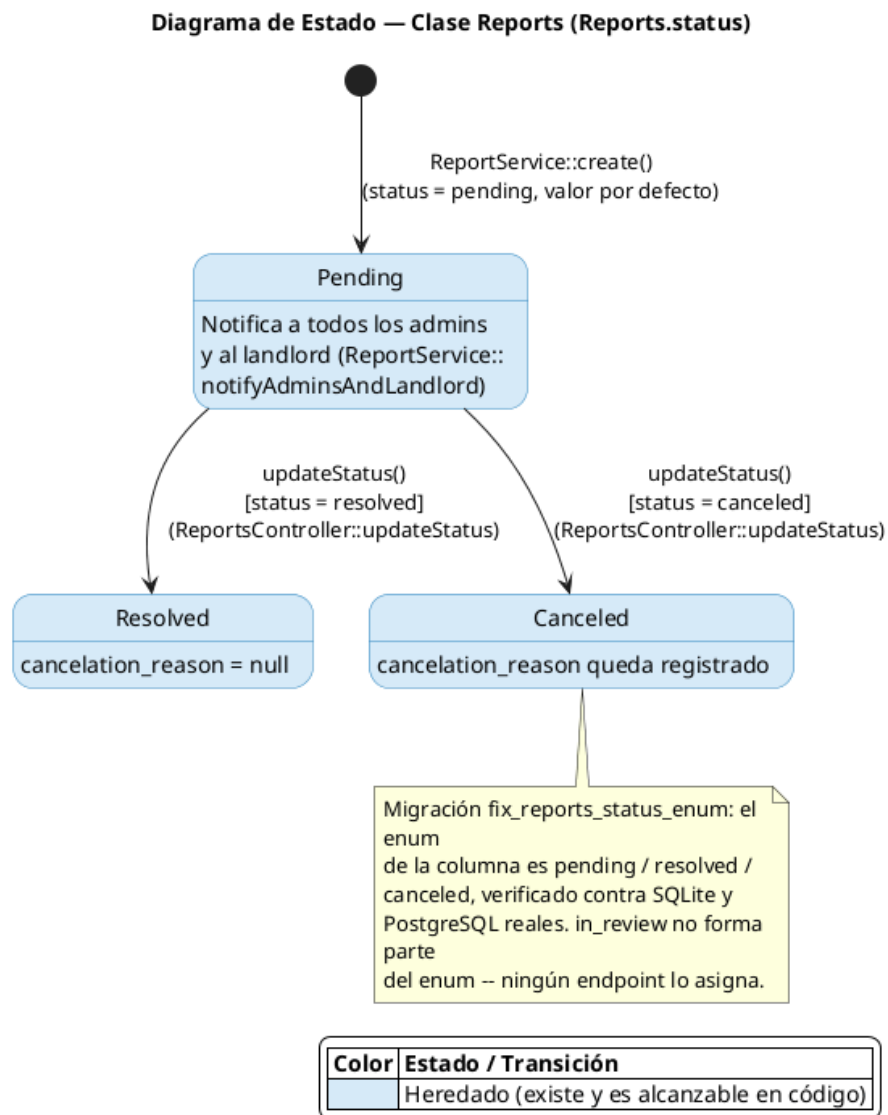


Figura 4.41: Diagrama de Estado – Clase Reports

Descripción: Ciclo de vida de `Reports.status`, creado por `ReportService::create()` (`backend/app/Services/ReportService.php`) en estado *Pending* y notificando en cascada a administradores y al arrendador. Desde *Pending*, `ReportsController::updateStatus()` transiciona a *Resolved* o *Canceled* según el valor de `status` recibido, validado por `UpdateReportStatusR` contra `in:resolved,canceled`. El enum de la columna, declarado en la migración `database/migrations/2026_08_14_000001_fix_reports_status_enum.php`, contempla exactamente esos tres valores (`pending/resolved/canceled`) — consistente con la validación del *request* y con lo que persiste el controlador. Esa migración reconstruye el `CHECK constraint` con SQL específico por driver (`Schema::getConnection()->getDriverName()`), porque el `$table->enum(...)->change()` nativo de Laravel 12 genera SQL inválido para Post-

greSQL aunque funciona en SQLite; se verificó empíricamente contra un PostgreSQL 16 real, no solo contra SQLite.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (motor Graphviz/dot).

Fuente: [recursos/diagramas_uml/estado_03.puml](#).

Tabla de trazabilidad

Elemento	Fuente (archivo / método)	Estado
Pending (estado inicial)	ReportService::create()	Heredado
Pending → Resolved	ReportsController::updateStatus()	Heredado
Pending → Canceled	ReportsController::updateStatus()	Heredado
Enum de la columna (pending/resolved/canceled)	backend/database/migrations/2026_08_14_000001_fix_reports_status_enum.php	Heredado

4.4.4. Diagrama de Estado – Clase Payments

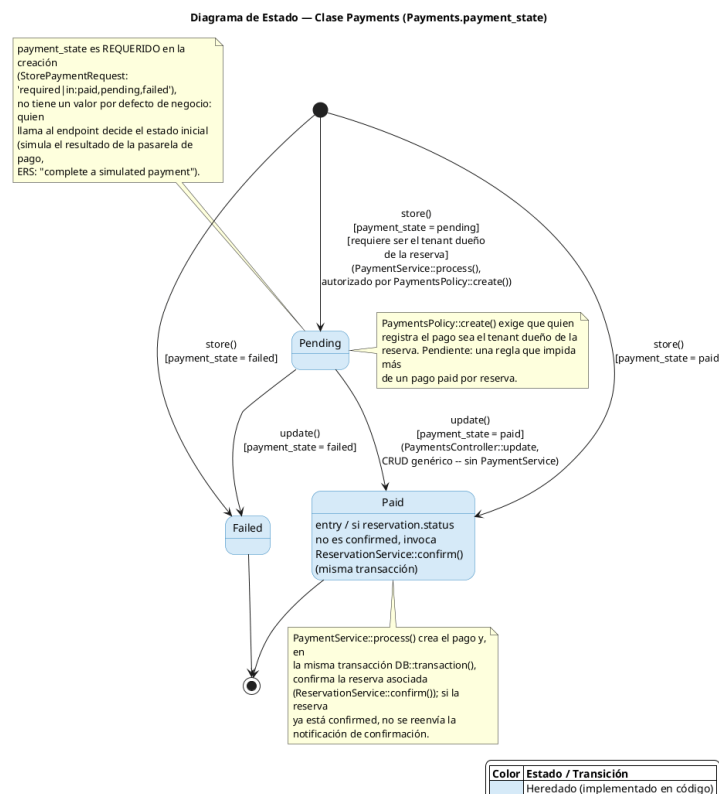


Figura 4.42: Diagrama de Estado – Clase Payments

Descripción: Ciclo de vida de Payments.payment_state (pending, paid, failed). A diferencia de las demás entidades transaccionales, payment_state es un campo **requerido** en la creación (StorePaymentRequest::rules(), backend/app/Http/Requests/StorePaymentRequest.php: 'required|in:paid,pending,failed'), sin valor por defecto de negocio: quien invoca el endpoint decide el estado inicial, simulando el resultado de una pasarela de pago (ERS: "complete a simulated payment to confirm a reservation"). PaymentService::process() (backend/app/Services/PaymentService.php) crea el Payments y, si payment_state = paid y la reserva no está ya confirmed, invoca ReservationService::confirm() dentro de la misma transacción — registrar el pago y confirmar la reserva quedan como una sola operación de servidor. PaymentsPolicy::create() exige, además, que quien registra el pago sea el tenant dueño de la reserva.

Pendiente: no existe una regla que impida más de un pago paid por reserva.

Herramienta de modelado utilizada: PlantUML v1.2026.6 (motor Graphviz/dot).

Fuente: recursos/diagramas_uml/estado_04.puml.

Tabla de trazabilidad

Elemento	Fuente (archivo / método)	Estado
Pending / Paid / Failed (creación)	PaymentsController::store() → PaymentService::process()	Heredado
Pending → Paid / Failed	PaymentsController::update(), CRUD genérico	Heredado
Paid confirma la reserva asociada	PaymentService.php → ReservationService::confirm(), misma transacción	Heredado
Autorización de ownership	PaymentsPolicy::create() (backend/app/Policies/PaymentsPolicy.php)	Heredado

4.4.5. Diagrama de Estado – Clase User

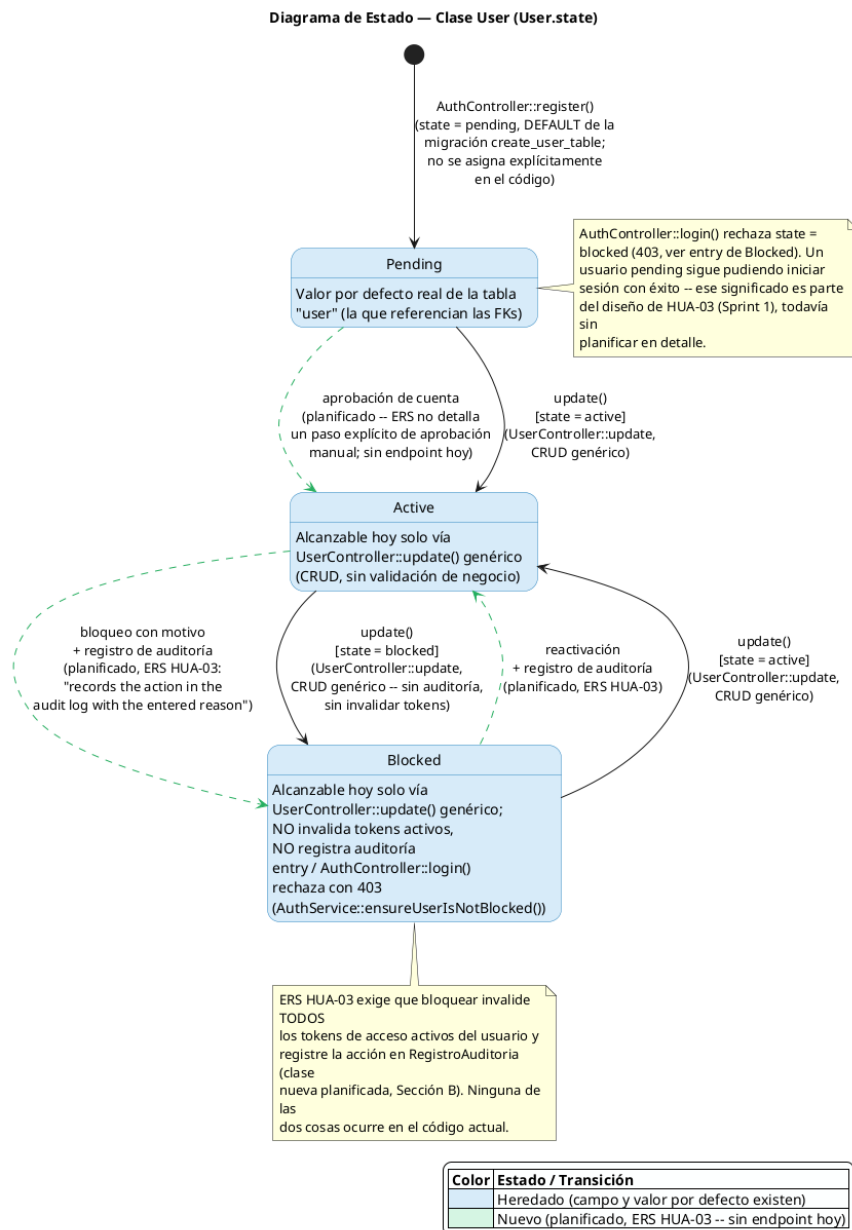


Figura 4.43: Diagrama de Estado – Clase User

Descripción: Ciclo de vida de `User.state` (pending, active, blocked), el campo base para la moderación de cuentas (ERS HUA-03, “User account blocking and reactivation”). `AuthController::register()` (`backend/app/Http/Controllers/AuthController.php`) no asigna `state` explícitamente, por lo que toda cuenta nueva nace en **pending** (valor por defecto real de la migración `database/migrations/2025_10_15_009000_create_user_table.php`, la tabla `user` en minúscula que referencian las llaves foráneas). Las transiciones **pending→active** y **active↔blocked** solo son alcanzables a través del CRUD genérico (`UserController::update()`), **sin** invalidar tokens de acceso activos y **sin** registrar auditoría.

trar auditoría — ambos requisitos explícitos de HUA-03 (mapeados a la clase planificada `RegistroAuditoria`, Sección B), historia de sprint que define ese diseño y que queda fuera del alcance de este código. `AuthController::login()` invoca `AuthService::ensureUserIsNotBlocked()` (`backend/app/Services/AuthService.php`), que rechaza con 403 (`AccountBlockedException`) cuando `state = blocked`; el guard es deliberadamente mínimo y no toca el significado de `pending` (una cuenta así sigue iniciando sesión con éxito), porque decidir eso es parte del diseño de HUA-03.

Herramienta de modelado utilizada: *PlantUML v1.2026.6* (motor Graphviz/dot).

Fuente: `recursos/diagramas_uml/estado_05.puml`.

Tabla de trazabilidad

Elemento	Fuente (archivo / método)	Estado
Pending (estado inicial)	<code>AuthController::register()</code> (default de la migración)	Heredado
Pending → Active, Active ↔ Blocked (CRUD)	<code>UserController::update()</code> , sin invalidar tokens ni auditar	Heredado
Guard de <code>state = blocked</code> en el login	<code>AuthService::ensureUserIsNotBlocked()</code>	Heredado
<code>pending</code> sin guard en el login (a propósito)	<code>backend/app/Http/Controllers/AuthController.php</code> (<code>login()</code>)	Heredado
Bloqueo/reactivación con auditoría y revocación de tokens	ERS HUA-03; <code>RegistroAuditoria</code> (Sección B), sin código todavía	Nuevo (planificado)

Nota: Los cinco diagramas anteriores cubren las clases del sistema cuyo comportamiento depende significativamente de su estado interno, identificadas en el Diagrama de Clases (Sección B) y en la tabla de Malos Olores (§3.2.3): `Reservations`, `StoreRooms`, `Reports`, `Payments` y `User`. `StoreModeration.status` no recibió un diagrama propio porque su ciclo de vida ahora está gobernado por el mismo `StoreModerationService` que `StoreRooms.publication_status` (documentado en la Figura 4.40).

Contribuciones Individuales

A continuación se detalla el aporte específico de cada integrante del equipo en la elaboración del presente documento y sus artefactos asociados.

Tabla 4.6: Contribuciones individuales de los integrantes del equipo

Integrante	Contribución específica	% Aporte
[Nombre integrante 1]	[Ej. Diagramas de clases, casos de uso UC-01 a UC-03, gestión de riesgos]	[25 %]
[Nombre integrante 2]	[Ej. Diagramas de secuencia 1-16, diagrama de componentes]	[25 %]
[Nombre integrante 3]	[Ej. Diagramas de secuencia 17-32, diagrama de despliegue]	[25 %]
[Nombre integrante 4]	[Ej. Diagramas de actividad, estado, cronograma y sprint backlogs]	[25 %]

Nota: El porcentaje de aporte debe sumar 100 % y ser consistente con la co-evaluación individual enviada por cada integrante en la tarea “Co-evaluación Proyecto”.

Referencias

- [1] Robertson S. and Robertson J, *Mastering the Requirements Process: Getting Requirements Right*.
- [2] Pressman and Maxim, *Software Engineering*.
- [3] Ian Sommerville, *Software Engineering*.
- [4] IEEE Recommended Practice for Software Requirements Specifications.
- [5] Systems and software engineering – Life cycle processes – Requirements engineering.
- [6] The IEEE Guide to the Software Engineering Body of Knowledge – SWEBOK.
- [7] MockupScreens, <http://www.mockupscreens.com>
- [8] Perdita Stevens, *Using UML*.
- [9] Jaye Hannah, “What’s the Difference Between a Wireframe, a Prototype, and a Mockup?”, <https://careerfoundry.com/en/blog/ux-design/difference-between-wireframes-prototypes-mockups/>
- [10] ETS, “Writing guidelines and templates”, <https://www.etsmtl.ca/en/studies/Guidelines-and-templates>
- [11] Medvidovic, N., & Taylor, R. N. (2000). *A classification and comparison framework for software architecture description languages*. IEEE Transactions on Software Engineering, 26(1), 70-93.
- [12] Visual Paradigm, “Tutorials”, <https://online.visual-paradigm.com/diagrams/tutorials/>
- [13] Visual Paradigm, “Learning Guides”, <https://www.visual-paradigm.com/guide/>
- [14] Visual Paradigm, “Software Design Handbook”, <https://www.visual-paradigm.com/learning/handbooks/software-design-handbook/>
- [15] Kirill Fakhroutdinov, “The Unified Modeling Language”, <https://www.uml-diagrams.org>
- [16] Bizzdesign, “UML modeling”, <https://support.bizzdesign.com/display/knowledge/UML+modeling>
- [17] Jakob Jenkov, “Exception Handling Strategy – Overview”, <https://jenkov.com/tutorials/exception-handling-strategies/overview.html>

- [18] Common Weakness Enumeration, “Bad Coding Practices”, <https://cwe.mitre.org/data/definitions/1006.html>
- [19] Dzone, “9 Best Practices to Handle Exceptions in Java”, <https://dzone.com/articles/9-best-practices-to-handle-exceptions-in-java>
- [20] Baeldung, “Exception Handling in Java”, <https://www.baeldung.com/java-exceptions>
- [21] Project Management Docs, “Use Case Document Template”, <https://www.projectmanagementdocs.com/template/project-documents/use-case-document/>
- [22] Refactoring Guru, “The Catalog of Design Patterns”, <https://refactoring.guru/design-patterns/catalog>
- [23] Volere, Volere Requirements Specification Template, <https://www.volere.org/templates/volere-requirements-specification-template/>

Anexo A

Especificación de Requerimientos de Software (Versión Mejorada)

Este anexo corresponde a la versión mejorada del documento de especificación de requerimientos (ERS) del sistema, elaborado con base en la plantilla Volere [23] y aprobado en el parcial anterior. Debe ser consistente con el diseño presentado en las Secciones A, B y C de este documento.

Opción 1 – Adjuntar como PDF independiente ya diagramado:

Si el ERS mejorado ya está en un PDF aparte, descomente la siguiente línea y coloque el archivo en `imagenes/ers_mejorado.pdf`:

Opción 2 – Incluir el contenido directamente en este documento:

[Pegar aquí, o mediante `\input{}`, el contenido completo del ERS mejorado: propósito, alcance, requerimientos funcionales y no funcionales, restricciones, atributos de calidad, etc., siguiendo la estructura Volere.]

A.1. Prototipo de Alta Fidelidad

Se presentan las capturas de pantalla del prototipo de alta fidelidad aprobado por el cliente, junto con el flujo de ventanas (navegación) correspondiente.

A.1.1. Flujo de Ventanas del Prototipo



Figura A.1: Diagrama de flujo de ventanas del prototipo

Herramienta utilizada: [Ej. Figma, Adobe XD, MockupScreens [7]]

A.1.2. Capturas de Pantalla

A.1.3. Pantalla 01 – [Nombre de la pantalla]

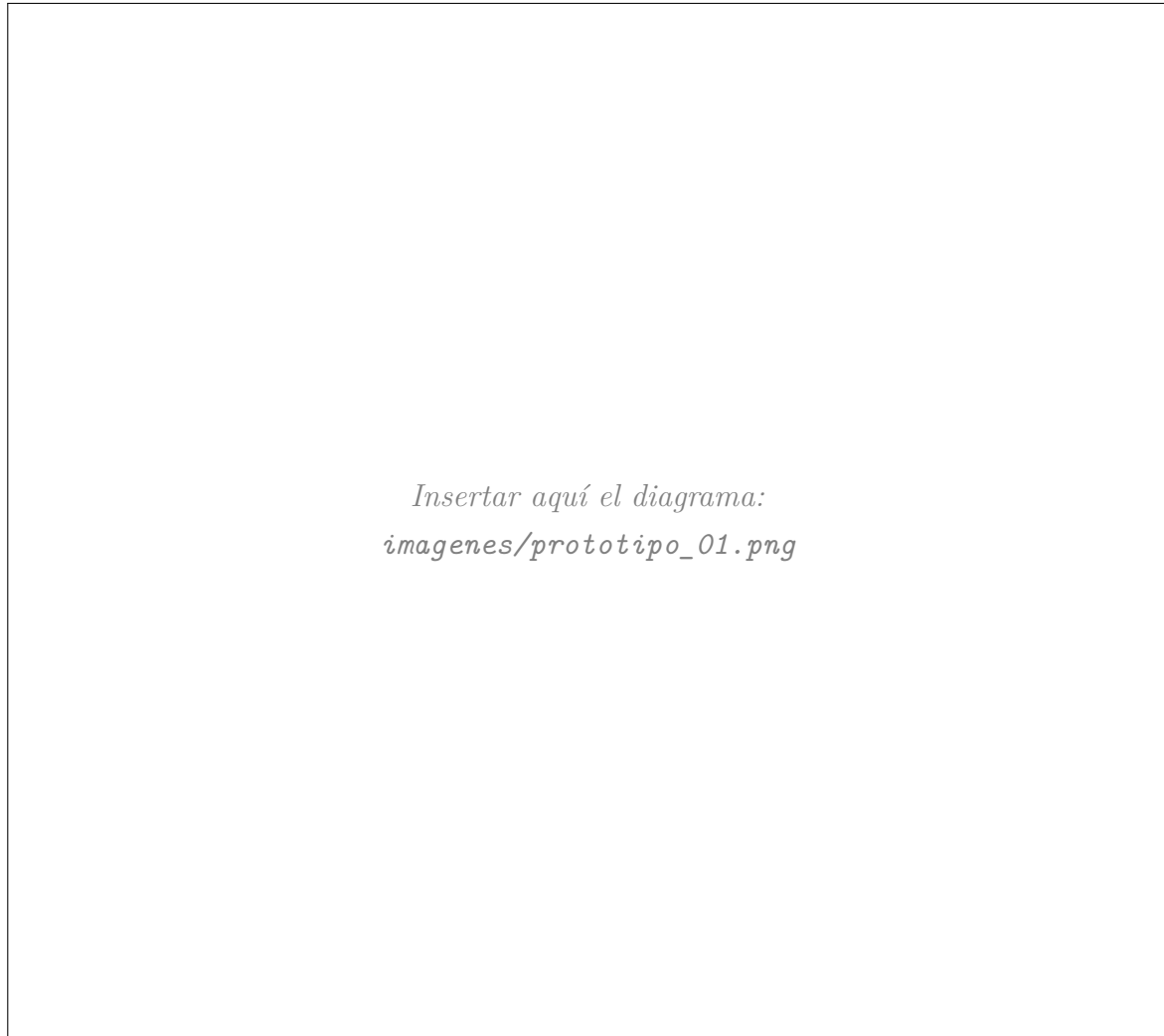


Figura A.2: Pantalla 01 – [Nombre de la pantalla]

Descripción: [Breve descripción de la pantalla y su relación con el(los) caso(s) de uso correspondiente(s).]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

A.1.4. Pantalla 02 – [Nombre de la pantalla]

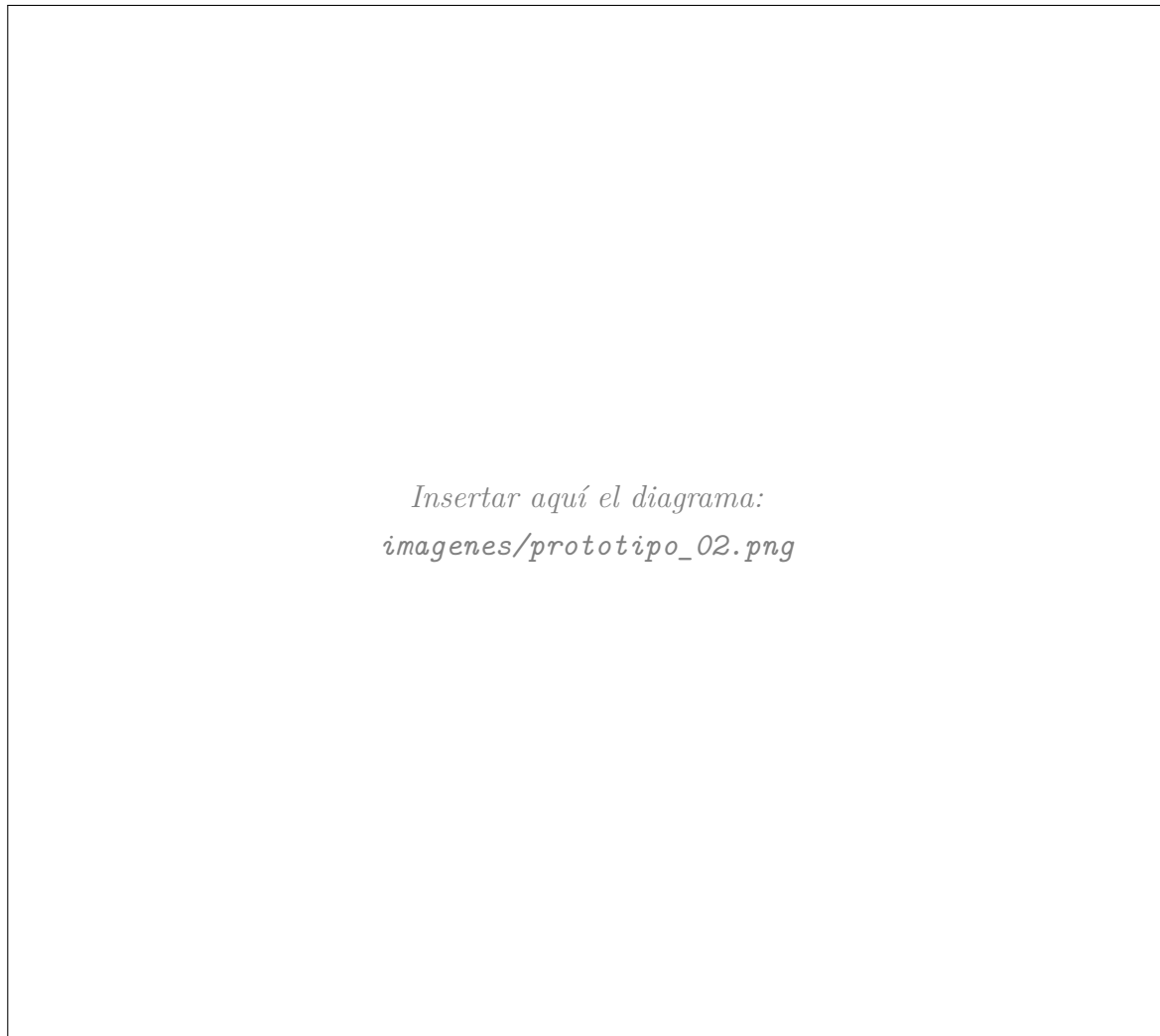


Figura A.3: Pantalla 02 – [Nombre de la pantalla]

Descripción: [Descripción de la pantalla.]

Herramienta de modelado utilizada: [Ej. Visual Paradigm, StarUML, Draw.io]

Nota: Duplique el bloque anterior para cada pantalla relevante del prototipo de alta fidelidad.

A.2. Acta de Conformidad del Anexo (ERS y Prototipo)

Importante: Al igual que en el cuerpo principal del documento, este anexo requiere un acta de conformidad firmada por el representante del cliente. Su ausencia también implica la penalidad máxima según la rúbrica.

Insertar aquí el acta de conformidad firmada correspondiente al ERS mejorado y al prototipo de alta fidelidad.
(Archivo esperado: `imagenes/acta_conformidad_ers.pdf`)